

Contents

第十八册简介：大模型产品化、商业化与落地	6
本书定位	6
为什么需要这本书	6
适合读者	6
学完后应具备的能力	6
阅读建议	6
与其他书的关系	6
第十八册：大模型产品化、商业化与落地	6
本书定位	6
章节文件	6
第一章：产品化总览	7
0. 本讲资料边界与第二轮精修口径	7
1.1 为什么算法工程师要懂产品化	7
1.2 Demo 和产品的区别	7
1.3 产品化的核心问题	8
1.3.1 关键公式与产品化指标速查	8
1.4 从模型能力到用户价值	9
1.5 什么场景适合大模型	9
1.6 用户体验不是模型输出	10
1.7 稳定性	10
1.8 成本	10
1.9 ROI	11
1.10 安全与合规	11
1.11 人在回路	11
1.12 上线不是终点	11
1.13 从技术指标到产品指标	12
1.14 常见产品化失败模式	12
1.15 最小可运行产品化审计 demo	12
1.16 面试题：大模型从 Demo 到产品差在哪里	15
1.17 面试题：如何判断一个大模型场景值不值得做	15
1.18 本章小结	15
第二章：用户需求与场景选择	15
0. 本讲资料边界与第二轮精修口径	15
2.1 场景选择为什么重要	16
2.2 真实需求和伪需求	16
2.3 用户是谁	16
2.4 用户任务链路	16
2.5 高频场景和高价值场景	17
2.6 容错率	17
2.7 数据可得性	17
2.8 任务是否可验证	18
2.9 大模型是否必要	18
2.10 自动化空间	18
2.11 ROI 初步估算	18
2.12 落地难度	19
2.13 场景优先级矩阵	19
2.13.1 关键公式与场景选择指标速查	19
2.14 试点场景	20
2.15 和业务方沟通的问题清单	20
2.16 常见失败模式	21
2.17 面试题：如何选择大模型落地场景	21

2.18 面试题：如何判断需求是否真实	21
2.19 最小可运行场景选择审计 demo	21
2.20 本章小结	26
第三章：模型能力到产品体验	26
0. 本讲资料边界与第二轮精修口径	26
3.1 为什么模型能力不等于体验	26
3.2 延迟体验	26
3.3 如何改善延迟感知	27
3.4 准确性体验	27
3.5 可控性	27
3.6 结构化输出	28
3.7 解释性	28
3.8 引用和来源	28
3.9 拒答体验	29
3.10 风格一致性	29
3.11 交互设计	29
3.12 用户引导	29
3.13 错误恢复	30
3.14 信任校准	30
3.15 个性化体验	30
3.16 体验指标	30
3.16.1 关键公式与能力到体验指标速查	31
3.17 常见失败模式	32
3.18 面试题：如何把模型能力转成产品体验	32
3.19 面试题：为什么一个高分模型产品体验可能不好	32
3.20 最小可运行能力到体验审计 demo	32
3.21 本章小结	35
第四章：成本收益与 ROI	36
0. 本讲资料边界与第二轮精修口径	36
4.1 为什么要算 ROI	36
4.2 成本不只是 Token	36
4.3 Token 成本	37
4.4 GPU 成本	37
4.5 RAG 成本	37
4.6 Agent 成本	37
4.7 人工审核成本	38
4.8 工程和运维成本	38
4.9 收益类型	38
4.10 人工替代收益	39
4.11 效率提升收益	39
4.12 质量收益	39
4.13 收入增长	39
4.14 风险降低收益	39
4.15 单位经济账	40
4.16 ROI 估算模板	40
4.16.1 关键公式与 ROI 指标速查	40
4.17 成本优化手段	41
4.18 ROI 的常见误区	42
4.19 面试题：如何估算大模型项目 ROI	42
4.20 面试题：如何降低大模型产品成本	42
4.21 最小可运行 ROI / 单位经济账审计 demo	42
4.22 本章小结	46
第五章：企业级 LLM 应用	46

0. 本讲资料边界与第二轮精修口径	46
5.1 企业级应用的特点	47
5.2 企业知识库问答	47
5.3 智能客服	48
5.4 客服 Copilot 和全自动客服	48
5.5 代码助手	48
5.6 数据分析助手	49
5.7 办公自动化	49
5.8 行业助手	49
5.9 企业系统集成	50
5.10 权限控制	50
5.11 数据治理	51
5.12 审计与合规	51
5.13 上线策略	51
5.14 组织协作	51
5.15 企业应用评估指标	52
5.15.1 关键公式与企业级应用指标速查	52
5.16 常见失败模式	53
5.17 面试题：企业级 LLM 应用和普通应用有什么区别	53
5.18 面试题：如何落地企业知识库问答	54
5.19 最小可运行企业级应用审计 demo	54
5.20 本章小结	57
第六章：RAG 产品落地	58
0. 本讲资料边界与第二轮精修口径	58
6.1 RAG 为什么适合企业落地	58
6.2 RAG Demo 和 RAG 产品的区别	58
6.3 企业文档治理	59
6.4 文档解析	59
6.5 分块策略	60
6.6 检索策略	60
6.7 权限控制	60
6.8 知识更新	60
6.9 引用可信	61
6.10 无法回答	61
6.11 用户反馈闭环	61
6.12 评估指标	62
6.12.1 关键公式与 RAG 产品指标速查	62
6.13 评估集构建	63
6.14 上线运营	63
6.15 常见失败模式	64
6.16 面试题：RAG 产品落地最难的是什么	64
6.17 面试题：如何评估企业 RAG	64
6.18 最小可运行 RAG 产品审计 demo	64
6.19 本章小结	68
第七章：Agent 产品落地	68
0. 本讲资料边界与第二轮精修口径	68
7.1 Agent 产品的本质	68
7.2 哪些任务适合 Agent	68
7.3 Agent 产品不要一开始全自动	69
7.4 工具权限设计	69
7.5 人工确认	70
7.6 失败恢复	70
7.7 状态管理	70
7.8 审计日志	71

7.9 体验设计	71
7.10 成本控制	71
7.11 Agent 与 Workflow 结合	72
7.12 Agent 产品评估	72
7.12.1 关键公式与 Agent 产品指标速查	72
7.13 适合先落地的 Agent 场景	73
7.14 常见失败模式	74
7.15 面试题：Agent 产品落地最重要的是什么	74
7.16 面试题：如何设计一个企业 Agent 产品	74
7.17 最小可运行 Agent 产品审计 demo	74
7.18 本章小结	78
第八章：多模态产品落地	78
0. 本讲资料边界与第二轮精修口径	78
8.1 多模态产品为什么重要	78
8.2 图像理解产品	79
8.3 文档多模态理解	79
8.4 语音助手	80
8.5 语音产品的特殊难点	80
8.6 图像生成和设计工具	80
8.7 视频生成产品	81
8.8 教育产品	81
8.9 医疗和工业场景	82
8.10 输入质量管理	82
8.11 版权和授权	82
8.12 隐私和安全	82
8.13 内容安全审核	83
8.14 体验指标	83
8.14.1 关键公式与多模态产品指标速查	83
8.15 常见失败模式	85
8.16 面试题：多模态产品落地难点是什么	85
8.17 面试题：如何落地图像理解产品	85
8.18 最小可运行多模态产品审计 demo	85
8.19 本章小结	90
第九章：隐私、合规与治理	90
0. 本讲资料边界与第二轮精修口径	90
9.1 为什么隐私合规重要	90
9.2 PII 是什么	91
9.3 企业敏感数据	91
9.4 数据最小化	91
9.5 数据流梳理	92
9.6 权限隔离	92
9.7 日志治理	92
9.8 训练数据使用	93
9.9 外部模型和数据出境	93
9.10 模型输出责任	93
9.11 审计	93
9.12 数据保留和删除	94
9.13 合规不是只靠法务	94
9.14 治理机制	94
9.14.1 关键公式与隐私治理指标速查	95
9.15 常见失败模式	96
9.16 面试题：大模型产品如何做隐私保护	96
9.17 面试题：企业 RAG 如何避免数据泄露	96
9.18 最小可运行隐私治理审计 demo	96

9.19 本章小结	100
第十章：上线运营与反馈闭环	100
0. 本讲资料边界与第二轮精修口径	100
10.1 上线为什么不是终点	101
10.2 灰度发布	101
10.3 质量监控	101
10.4 成本监控	102
10.5 延迟监控	102
10.6 用户反馈	102
10.7 失败样本分析	102
10.8 A/B Test	103
10.9 离线评估和在线评估	103
10.10 反馈到数据闭环	103
10.11 Prompt 迭代	104
10.12 模型版本迭代	104
10.13 知识库运营	104
10.14 安全监控	104
10.15 运营看板	105
10.16 持续迭代优先级	105
10.16.1 关键公式与上线运营指标速查	105
10.16.2 最小可运行上线运营审计 demo	107
10.17 常见失败模式	111
10.18 面试题：大模型产品上线后如何运营	111
10.19 面试题：如何做大模型产品 A/B Test	111
10.20 本章小结	112
第十一章：技术产品面试题	112
11.1 大模型从 Demo 到产品差在哪里	112
11.2 如何判断一个 LLM 场景值不值得做	112
11.3 如何判断需求是否真实	112
11.4 如何把模型能力转成产品体验	112
11.5 为什么高分模型产品体验可能不好	112
11.6 如何估算大模型项目 ROI	113
11.7 如何降低大模型产品成本	113
11.8 企业级 LLM 应用和普通应用有什么区别	113
11.9 如何落地企业知识库问答	113
11.10 RAG 产品落地最难的是什么	113
11.11 如何评估企业 RAG	113
11.12 Agent 产品落地最重要的是什么	113
11.13 如何设计企业 Agent 产品	113
11.14 多模态产品落地难点是什么	113
11.15 如何落地图像理解产品	113
11.16 大模型产品如何做隐私保护	114
11.17 企业 RAG 如何避免数据泄露	114
11.18 大模型产品上线后如何运营	114
11.19 如何做大模型产品 A/B Test	114
11.20 关键公式与面试评分速查	114
11.21 最小可运行技术产品面试复盘 demo	115
11.22 技术产品面试回答模板	117
11.23 高频追问	118
11.24 本章小结	118

第十八册简介：大模型产品化、商业化与落地

本书定位

第十八册帮助算法工程师理解大模型技术如何变成产品、业务价值和真实系统影响。

它不是传统算法书，而是补齐场景选择、用户体验、ROI、企业应用、隐私合规、上线运营和反馈闭环。

为什么需要这本书

真实项目不是只比较 benchmark。能否落地取决于用户需求、产品体验、成本收益、风险合规、运营反馈和组织协作。算法工程师理解这些问题，项目表达会更成熟。

适合读者

1. 想从算法走向实际产品落地的读者。
2. 做企业 RAG、Agent、多模态、客服、知识库项目的人。
3. 面试中需要回答业务价值和 ROI 的候选人。

学完后应具备的能力

1. 能选择适合大模型落地的场景。
2. 能把模型能力转化为产品体验指标。
3. 能估算成本、收益、ROI 和风险。
4. 能理解企业级 LLM 应用、隐私合规和反馈闭环。

阅读建议

建议在完成第三册项目后阅读，用它提升项目的业务表达和产品判断。

与其他书的关系

第十八册是技术到业务的桥梁，与第三册项目、第十九册实战宝典和第十二册求职表达紧密相关。

第十八册：大模型产品化、商业化与落地

本书定位

本书不是传统算法书，而是帮助算法工程师理解大模型技术如何变成产品、业务价值和真实系统影响。

章节文件

1. chapters/01-产品化总览.md
2. chapters/02-用户需求与场景选择.md
3. chapters/03-模型能力到产品体验.md
4. chapters/04-成本收益与 roi.md
5. chapters/05-企业级 llm 应用.md
6. chapters/06-rag 产品落地.md
7. chapters/07-agent 产品落地.md
8. chapters/08-多模态产品落地.md
9. chapters/09-隐私合规与治理.md
10. chapters/10-上线运营与反馈闭环.md
11. chapters/11-技术产品面试题.md

第一章：产品化总览

大模型项目最容易犯的错误，是把 demo 当产品。一个 demo 能在演示时回答几个问题、生成几段文本、调用一次工具，并不代表它可以稳定服务真实用户。产品化要解决的不只是模型效果，还包括用户需求、体验设计、成本、稳定性、安全、合规、运营反馈和商业价值。

本章是第十八册的总览。我们从“demo 到产品”的差距开始，系统讲大模型产品化需要关注哪些问题：用户价值、场景选择、产品体验、稳定性、成本、ROI、安全合规、上线运营和反馈闭环。对算法工程师来说，理解这些内容能让你在面试和项目复盘里不只会讲模型，也能讲清楚技术如何产生业务价值。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修前，已按 WRITING_PLAN.md 联网核对 OpenAI Evals / evaluation 公开文档、OpenAI 延迟优化与生产应用相关文档、Google SRE 中 SLI / SLO / error budget 的可靠性口径，以及 NIST AI RMF / Generative AI Profile 的风险管理资料边界。

本章聚焦产品化总览：如何把大模型能力转成用户价值、业务指标、单位经济账、可靠性指标、安全合规和反馈闭环。它不替代后续章节的用户研究、ROI 细算、企业应用、RAG / Agent / 多模态落地、隐私合规或上线运营细节。

本章的核心写作口径是：大模型产品化不是“接一个模型 API”，而是围绕真实用户任务建立可衡量、可运营、可控风险、可持续迭代的产品系统。

1.1 为什么算法工程师要懂产品化

很多算法同学习惯从模型出发：模型准确率、benchmark、推理速度、训练方法、RAG、Agent、微调。但真实公司更关心：这个系统解决了谁的问题，节省了多少成本，带来了多少收入，是否稳定，是否安全，用户是否愿意用。

产品化能力能帮助算法工程师回答这些问题：

1. 这个场景是否值得做大模型。
2. 模型能力如何转化为用户体验。
3. 指标提升是否能带来业务收益。
4. 成本是否可接受。
5. 风险是否可控。
6. 上线后如何持续改进。

面试回答：

算法工程师懂产品化，是因为真实大模型项目不只看模型指标，还要看用户价值、成本收益、稳定性、安全合规和反馈闭环。能落地

1.2 Demo 和产品的区别

Demo 的目标是证明“能做”。产品的目标是稳定地为用户创造价值。

Demo 常见特点：

1. 样例少。
2. 边界可控。
3. 人工挑选输入。
4. 失败可以解释。
5. 成本不敏感。
6. 安全要求低。

产品要求：

1. 面对真实多样输入。
2. 长期稳定运行。
3. 失败可恢复。

4. 成本可控。
5. 用户体验一致。
6. 安全合规。
7. 有监控和反馈。

很多 demo 看起来惊艳，是因为输入被精心选择。产品必须面对真实用户的模糊表达、错误操作、恶意输入和长尾需求。

1.3 产品化的核心问题

大模型产品化至少要回答八个问题：

1. 用户是谁。
2. 痛点是什么。
3. 大模型为什么适合。
4. 成功如何衡量。
5. 成本如何控制。
6. 风险如何管理。
7. 上线后如何监控。
8. 如何持续迭代。

如果这些问题答不清楚，只是“把大模型接进来”，项目很容易变成技术展示，而不是可持续产品。

1.3.1 关键公式与产品化指标速查

一个大模型产品候选场景可以抽象为：

$$s_i = (u_i, p_i, f_i, v_i, q_i, c_i, r_i, w_i)$$

变量含义：

1. u_i 是目标用户或使用人群。
2. p_i 是痛点强度。
3. f_i 是月度任务频次。
4. v_i 是单次成功任务的业务价值。
5. q_i 是模型或系统质量指标。
6. c_i 是单次任务成本。
7. r_i 是风险成本或风险等级。
8. w_i 是 workflow 适配程度。

技术指标到产品指标的基本链路是：

$$q_{\{\mathrm{tech}\}} \rightarrow q_{\{\mathrm{task}\}} \rightarrow m_{\{\mathrm{user}\}} \rightarrow b_{\{\mathrm{biz}\}}$$

例如检索召回率提升只是 q_{tech} ，真正要证明的是任务成功率、用户采纳率、自助解决率、成本下降或收入提升。

任务成功率提升：

$$\Delta R_{\{\mathrm{succ}\}} = R_{\{\mathrm{succ}, \mathrm{new}\}} - R_{\{\mathrm{succ}, \mathrm{base}\}}$$

月度业务收益可以粗略估算为：

$$B = f \cdot a \cdot \Delta R_{\{\mathrm{succ}\}} \cdot v$$

其中 f 是月度任务频次， a 是采用率， v 是单次成功任务的价值。

月度总成本：

$$C = f \cdot a \cdot (c_{\{\mathrm{model}\}} + c_{\{\mathrm{ops}\}} + c_{\{\mathrm{risk}\}}) + C_{\{\mathrm{fixed}\}}$$

其中 c_{model} 是模型或推理成本， c_{ops} 是检索、工具、审核、监控和运维等单次成本， c_{risk} 是预期风险损失， C_{fixed} 是固定研发、平台和治理成本。

净收益：

$$N=B-C$$

收益成本比：

$$R_{\mathrm{bc}}=\frac{B}{C}$$

常见 ROI 口径：

$$\mathrm{ROI}=\frac{B-C}{C}$$

产品化门禁可以写成：

$$G_{\mathrm{prod}}=\mathbf{1}[\Delta R_{\mathrm{succ}}>0\wedge R_{\mathrm{bc}}\geq \tau_b\wedge L_{95}\leq \tau_l\wedge G_{\mathrm{risk}}\leq \tau_r\wedge G_{\mathrm{wf}}\leq \tau_{wf}\wedge G_{\mathrm{eval}}\leq \tau_{eval}]$$

其中 L_{95} 是 P95 延迟， G_{risk} 是安全合规门禁， G_{workflow} 是工作流适配门禁， G_{eval} 是评估和反馈门禁。

直觉：产品化判断不是“模型分数高不高”，而是收益、成本、延迟、风险、工作流和反馈闭环是否同时过线。

1.4 从模型能力到用户价值

模型能力本身不是用户价值。

例如：

模型能总结文档

这只是能力。用户价值可能是：

法务人员审合同时间从 30 分钟降到 10 分钟

客服人员查知识库时间从 2 分钟降到 20 秒

销售人员生成方案初稿效率提升 50%

产品化要把能力翻译成业务指标。常见指标包括：

1. 节省时间。
2. 降低人工成本。
3. 提高转化率。
4. 提高解决率。
5. 降低错误率。
6. 提升用户满意度。
7. 缩短交付周期。

1.5 什么场景适合大模型

适合大模型的场景通常有这些特点：

1. 输入是自然语言、文档、代码、图片或多模态内容。
2. 任务需要理解、生成、总结、推理或交互。
3. 传统规则难覆盖长尾。
4. 人工成本高。
5. 容错空间存在。
6. 可以通过人审或工具验证降低风险。
7. 有足够数据或知识库支持。

不适合的场景：

1. 强确定性、低延迟、低成本规则就能解决的问题。
2. 错误代价极高且无法复核的问题。
3. 没有明确价值指标的问题。
4. 数据质量很差且无法治理的问题。
5. 用户不愿改变流程的问题。

大模型不是万能工具，选错场景会导致投入大、收益小。

1.6 用户体验不是模型输出

很多人以为大模型产品体验就是“回答得好”。实际体验还包括：

1. 是否知道能问什么。
2. 响应是否足够快。
3. 错误时是否能解释。
4. 是否能引用来源。
5. 是否能让用户修改和追问。
6. 是否能接入现有工作流。
7. 是否能处理用户不完整输入。
8. 是否能稳定保持风格和格式。

例如企业知识库问答，答案质量重要，但引用来源、权限控制、无法回答时的表达和反馈入口同样重要。

1.7 稳定性

大模型输出具有不确定性。产品化要解决稳定性问题。

常见稳定性挑战：

1. 同样问题多次回答不一致。
2. Prompt 稍变输出大变。
3. 模型版本升级导致行为变化。
4. 外部工具失败。
5. RAG 召回不稳定。
6. 长上下文下遗漏关键信息。
7. 并发高时延迟上升。

稳定性措施：

1. 固定关键 prompt 和参数。
2. 使用结构化输出。
3. 对关键任务加 verifier。
4. 监控线上指标。
5. 建立回归测试集。
6. 版本灰度发布。
7. 失败降级。

1.8 成本

大模型产品的成本不仅是 API 调用费。

成本包括：

1. 输入 token。
2. 输出 token。
3. 多次采样和重试。
4. RAG 检索和向量库。
5. reranker 和 verifier。
6. 工具调用。
7. GPU 推理资源。
8. 日志和监控。
9. 人工审核。
10. 数据治理。

产品化必须算单位经济账。例如每处理一个客服问题成本是多少，节省多少人工时间，是否值得。

1.9 ROI

ROI 是投入产出比。

简单理解：

$ROI = \text{收益} / \text{成本}$

大模型项目收益可能来自：

1. 降低人工成本。
2. 提高效率。
3. 增加收入。
4. 提升留存。
5. 降低风险。
6. 提升客户满意度。

成本可能来自模型调用、研发、数据、审核、运维、合规和失败风险。

面试中不一定要给精确财务模型，但要能说明如何估算收益和成本。

1.10 安全与合规

大模型产品会处理用户输入、企业数据和生成内容，因此必须考虑安全合规。

常见问题：

1. 隐私数据泄露。
2. 权限越界。
3. 幻觉导致错误决策。
4. 生成不合规内容。
5. 数据跨境或存储问题。
6. 日志中保存敏感信息。
7. 工具调用产生真实风险。

安全合规不是上线前临时补丁，而应该从产品设计阶段就纳入。

1.11 人在回路

Human-in-the-loop 是很多大模型产品落地的关键。

适合加入人工审核的场景：

1. 高风险决策。
2. 对外发送内容。
3. 法律、医疗、金融场景。
4. 删除或修改重要数据。
5. 模型低置信度。
6. 新系统试运行阶段。

人工不是失败，而是风险控制和数据积累方式。随着系统成熟，可以逐步扩大自动化范围。

1.12 上线不是终点

大模型产品上线后还需要运营。

要持续监控：

1. 使用量。
2. 成功率。
3. 用户满意度。
4. 延迟。

5. 成本。
6. 幻觉率。
7. 投诉和人工接管。
8. 安全事件。
9. 反馈样本。

没有反馈闭环，产品无法持续变好。真实产品通常通过日志、用户反馈、人工标注、失败案例分析和 A/B 测试持续迭代。

1.13 从技术指标到产品指标

技术指标和产品指标要连接起来。

例如 RAG：

召回率 -> 答案正确率 -> 用户自助解决率 -> 客服成本下降

例如 Code Agent：

测试通过率 -> 修复成功率 -> 开发节省时间 -> 交付效率提升

例如客服机器人：

意图识别准确率 -> 首次解决率 -> 转人工率下降 -> 用户满意度提升

只讲技术指标，会显得项目没有业务闭环；只讲业务指标，不讲技术支撑，也会显得不专业。

1.14 常见产品化失败模式

1. 为了用大模型而用大模型。
2. 只做 demo，不做真实用户验证。
3. 没有定义成功指标。
4. 忽略成本，越用越亏。
5. 忽略长尾输入和失败恢复。
6. 没有权限和合规设计。
7. 用户 workflow 接入差。
8. 上线后没有监控和反馈。
9. 模型升级没有回归测试。
10. 把产品问题误认为纯模型问题。

产品化失败往往不是因为模型完全不行，而是因为场景、体验、成本和运营没有设计好。

1.15 最小可运行产品化审计 demo

下面这个 demo 不调用外部模型，而是构造 4 个候选大模型产品场景，估算任务成功率提升、月度收益、月度成本、ROI、收益成本比，并检查痛点、工作流、评估、反馈、安全、隐私、延迟和单位经济账。

它演示的问题是：产品化不能只看“能不能做”，还要看“值不值得做、能不能稳定做、风险是否可控、上线后能不能迭代”。

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class ProductCase:
    case_id: str
    scenario: str
    user_pain: float
    monthly_tasks: int
    value_per_success: float
```

```
baseline_success_rate: float
llm_success_rate: float
adoption_rate: float
p95_latency_s: float
model_cost_per_task: float
ops_cost_per_task: float
risk_cost_per_task: float
fixed_monthly_cost: float
safety_ready: bool
privacy_ready: bool
workflow_fit: bool
eval_ready: bool
feedback_ready: bool

cases = [
    ProductCase("support_rag", "企业客服知识库助手", 0.90, 20000, 4.0, 0.45, 0.70, 0.65, 2.8, 0.08, 0.04, 0.02, 300),
    ProductCase("contract_review", "合同审阅 Copilot", 0.80, 1200, 60.0, 0.35, 0.62, 0.50, 8.5, 0.70, 5.00, 1.50, 6),
    ProductCase("code_triage", "代码缺陷分诊 Agent", 0.85, 2500, 25.0, 0.30, 0.52, 0.40, 25.0, 1.20, 0.60, 0.30, 80),
    ProductCase("generic_chat", "通用闲聊入口", 0.35, 50000, 0.20, 0.50, 0.56, 0.25, 4.0, 0.05, 0.02, 0.03, 4000, T)
]

def round3(value):
    return round(value, 3)

def audit_case(case):
    success_uplift = max(case.llm_success_rate - case.baseline_success_rate, 0.0)
    monthly_benefit = case.monthly_tasks * case.adoption_rate * success_uplift * case.value_per_success
    variable_cost = case.monthly_tasks * case.adoption_rate * (
        case.model_cost_per_task + case.ops_cost_per_task + case.risk_cost_per_task
    )
    monthly_cost = variable_cost + case.fixed_monthly_cost
    net_benefit = monthly_benefit - monthly_cost
    benefit_cost_ratio = monthly_benefit / monthly_cost if monthly_cost else 0.0
    roi = net_benefit / monthly_cost if monthly_cost else 0.0
    monthly_margin_before_fixed = monthly_benefit - variable_cost
    payback_months = (
        round3(case.fixed_monthly_cost / monthly_margin_before_fixed)
        if monthly_margin_before_fixed > 0
        else None
    )

gates = {
    "pain": case.user_pain >= 0.60,
    "quality_uplift": success_uplift >= 0.10,
    "unit_economics": benefit_cost_ratio >= 1.20,
    "latency": case.p95_latency_s <= 10.0,
    "risk": case.safety_ready and case.privacy_ready,
    "workflow": case.workflow_fit,
    "eval": case.eval_ready,
    "feedback": case.feedback_ready,
}

score = (
```

```

0.18 * case.user_pain
+ 0.20 * min(success_uplift / 0.30, 1.0)
+ 0.20 * min(benefit_cost_ratio / 2.0, 1.0)
+ 0.12 * case.adoption_rate
+ 0.10 * (1.0 if gates["latency"] else 0.0)
+ 0.10 * (1.0 if gates["risk"] else 0.0)
+ 0.05 * (1.0 if case.workflow_fit else 0.0)
+ 0.05 * (1.0 if case.eval_ready and case.feedback_ready else 0.0)
)
return {
    "case_id": case.case_id,
    "success_uplift": round3(success_uplift),
    "monthly_benefit": round(monthly_benefit, 2),
    "monthly_cost": round(monthly_cost, 2),
    "net_benefit": round(net_benefit, 2),
    "benefit_cost_ratio": round3(benefit_cost_ratio),
    "roi": round3(roi),
    "payback_months": payback_months,
    "gates": gates,
    "ship_gate": all(gates.values()),
    "score": round3(score),
}

```

```

audits = [audit_case(case) for case in cases]
ranked = sorted(((item["case_id"], item["score"], item["ship_gate"]) for item in audits), key=lambda x: (-x[1], x[0]))
rejected = {
    item["case_id"]: [name for name, passed in item["gates"].items() if not passed]
    for item in audits
    if not item["ship_gate"]
}
ship_candidates = [item["case_id"] for item in audits if item["ship_gate"]]

print(f"ranked={ranked}")
print(f"ship_candidates={ship_candidates}")
print(f"rejected={rejected}")
for item in audits:
    print(
        item["case_id"],
        {
            "success_uplift": item["success_uplift"],
            "monthly_benefit": item["monthly_benefit"],
            "monthly_cost": item["monthly_cost"],
            "net_benefit": item["net_benefit"],
            "benefit_cost_ratio": item["benefit_cost_ratio"],
            "roi": item["roi"],
            "payback_months": item["payback_months"],
            "ship_gate": item["ship_gate"],
        },
    )

```

输出示例:

```

ranked=[('support_rag', 0.907, True), ('contract_review', 0.728, False), ('code_triage', 0.602, False), ('generic_chat',
ship_candidates=['support_rag']
rejected={'contract_review': ['unit_economics', 'feedback'], 'code_triage': ['unit_economics', 'latency'], 'generic_chat': ['unit_economics', 'latency']}

```

```
support_rag {'success_uplift': 0.25, 'monthly_benefit': 13000.0, 'monthly_cost': 4820.0, 'net_benefit': 8180.0, 'benefit_cost_ratio': 1.68, 'roi': 0.34, 'payback_months': 1.11, 'ship_gate': True}
contract_review {'success_uplift': 0.27, 'monthly_benefit': 9720.0, 'monthly_cost': 10320.0, 'net_benefit': -600.0, 'benefit_cost_ratio': 0.942, 'roi': -0.058, 'payback_months': 1.11, 'ship_gate': False}
code_triage {'success_uplift': 0.22, 'monthly_benefit': 5500.0, 'monthly_cost': 10100.0, 'net_benefit': -4600.0, 'benefit_cost_ratio': 0.545, 'roi': -0.455, 'payback_months': 2.353, 'ship_gate': False}
generic_chat {'success_uplift': 0.06, 'monthly_benefit': 150.0, 'monthly_cost': 5250.0, 'net_benefit': -5100.0, 'benefit_cost_ratio': 0.029, 'roi': -0.971, 'payback_months': None, 'ship_gate': False}
```

这个 demo 的结论不是“只做客服 RAG”，而是展示产品化评审的思路：support_rag 同时满足痛点、质量提升、单位经济账、延迟、风险、工作流、评估和反馈门禁；contract_review 价值高但审核与固定成本过重且反馈闭环未完成；code_triage 任务高价值但当前延迟和成本不适合直接全量上线；generic_chat 没有明确痛点和工作流，不应该因为“能聊天”就产品化。

1.16 面试题：大模型从 Demo 到产品差在哪里

回答要点：

Demo 主要证明技术能跑通，通常样例少、输入可控、失败可接受。产品要面对真实用户和长尾输入，需要稳定性、成本控制、安全合规。

1.17 面试题：如何判断一个大模型场景值不值得做

回答要点：

我会看四个方面：第一，用户痛点是否明确，是否有高频或高价值需求；第二，大模型是否比规则、搜索或传统模型更适合；第三，产品化成本是否可控；第四，是否有明确的落地路径。

1.18 本章小结

大模型产品化的核心，是把模型能力转化为真实用户价值。Demo 证明“能做”，产品要求“稳定、可控、可持续地创造价值”。算法工程师要能把模型指标和业务指标连接起来，理解成本、体验、安全、合规、运营和反馈闭环。

下一章会进入用户需求与场景选择，进一步讨论如何从真实业务问题中筛选适合大模型落地的场景，避免为了技术而技术。

第二章：用户需求与场景选择

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对四类口径：OpenAI evals 与生产评估强调先定义任务、样本、指标和反馈闭环；Google HEART 框架强调把用户目标拆成 goals、signals、metrics；Google SRE 的 SLI / SLO / error budget 口径提醒我们把可靠性和可接受失败率放进上线门禁；NIST AI RMF / Generative AI Profile 提醒我们在场景筛选阶段就评估风险、可控性和治理责任。

本章不讨论具体模型选型、prompt 模板、RAG 架构、Agent 工具协议或企业销售合同细节。这里的核心问题只有一个：在动手做大模型应用前，如何判断一个候选场景是否有真实用户、真实痛点、可衡量收益、可控风险和可试点路径。

大模型落地的第一步不是选模型，而是选场景。场景选错，再强的模型也很难产生价值；场景选对，即使模型方案不复杂，也可能带来明显收益。很多失败的大模型项目，不是技术完全做不出来，而是需求不真实、用户不用、成本不划算或风险不可控。

本章系统讲用户需求与场景选择：如何判断需求是否真实，如何筛选高价值场景，如何评估任务频率、容错率、数据可得性、自动化空间、ROI、组织阻力和落地优先级。对算法工程师来说，这一章能帮助你把项目从“技术可行”讲到“业务值得做”。

2.1 场景选择为什么重要

同样的大模型能力，在不同场景中的价值完全不同。

例如“文档总结”：

1. 给用户总结一篇普通新闻，价值有限。
2. 给律师总结合同风险，价值较高。
3. 给客服总结用户历史工单，能节省时间。
4. 给医生总结病历，价值高但风险也高。

技术能力相似，但用户、价值、风险、合规和验收标准完全不同。

面试回答：

大模型项目首先要选对场景。判断一个场景是否值得做，要看用户痛点是否真实、任务频率是否足够、人工成本是否高、模型是否比

2.2 真实需求和伪需求

真实需求通常有几个特征：

1. 用户现在已经在花时间或花钱解决。
2. 问题高频或高价值。
3. 用户愿意改变工作流。
4. 解决后有明确收益。
5. 失败后用户会明显不满。

伪需求常见表现：

1. 只是觉得“大模型很酷”。
2. 用户说想要，但实际不用。
3. 没有明确使用场景。
4. 没有可衡量指标。
5. 传统规则已经足够。
6. 只是内部展示项目。

判断需求时，不要只听用户说什么，还要看用户现在怎么做、痛点出现频率、愿意为解决方案付出什么成本。

2.3 用户是谁

同一个产品，不同用户的需求不同。

例如企业知识库问答：

1. 一线客服关心快速查到答案。
2. 客服主管关心解决率和转人工率。
3. IT 团队关心权限和系统集成。
4. 法务关心合规。
5. 管理层关心成本收益。

场景选择必须明确主用户和决策者。很多项目失败，是因为使用者觉得麻烦，决策者只看概念，技术团队只看模型指标，三方目标不一致。

2.4 用户任务链路

不要只看单点功能，要看完整任务链路。

例如“帮销售写方案”不是一个单步任务。它可能包括：

1. 了解客户行业。
2. 查询历史案例。
3. 选择产品卖点。

4. 生成方案初稿。
5. 人工修改。
6. 审批。
7. 发给客户。

大模型可以介入哪个环节？哪个环节最耗时？哪个环节最容易出错？哪个环节最能衡量收益？这些问题比“模型能不能写方案”更重要。

2.5 高频场景和高价值场景

高频场景：每天大量发生，单次价值可能不高，但总收益大。

例如：

1. 客服问答。
2. 工单分类。
3. 文档检索。
4. 会议纪要。
5. 邮件草稿。

高价值场景：发生频率不一定高，但单次价值大。

例如：

1. 合同风险审查。
2. 重大故障排查。
3. 投研报告分析。
4. 代码迁移。
5. 企业知识沉淀。

优先级不只看频率，也看单次收益、风险和落地难度。

2.6 容错率

容错率决定产品形态。

高容错场景：

1. 头脑风暴。
2. 文案初稿。
3. 会议摘要草稿。
4. 内部知识探索。

低容错场景：

1. 医疗诊断。
2. 法律结论。
3. 财务报表。
4. 权限修改。
5. 自动下单或支付。

高容错场景可以更自动化；低容错场景需要人审、引用、验证、权限和审计。不能把低容错场景做成完全自动化黑盒。

2.7 数据可得性

大模型应用离不开数据。

要评估：

1. 是否有知识库。
2. 数据是否结构化。

3. 数据是否最新。
4. 数据质量如何。
5. 是否有权限访问。
6. 是否包含隐私。
7. 是否能用于评估。
8. 是否有历史反馈。

很多 RAG 项目失败，不是模型不会答，而是知识库混乱、文档过期、权限不清、没有标准答案。

2.8 任务是否可验证

可验证性决定落地难度。

容易验证：

1. 代码测试是否通过。
2. 表单字段是否填对。
3. 答案是否来自文档。
4. 分类是否匹配标签。
5. 数据分析结果是否符合计算。

难验证：

1. 创意是否好。
2. 战略建议是否正确。
3. 用户是否满意。
4. 开放式分析是否全面。

可验证场景更适合自动化和规模化；难验证场景更需要人工反馈、评价体系和灰度上线。

2.9 大模型是否必要

不是所有智能需求都需要大模型。

可以先问：

1. 规则能否解决？
2. 搜索能否解决？
3. 传统分类模型能否解决？
4. 模板能否解决？
5. 工作流优化能否解决？

如果规则和搜索已经足够，大模型可能增加成本和不确定性。大模型适合处理自然语言、多样表达、长尾问题、生成式任务和复杂上下文。

2.10 自动化空间

大模型不一定要完全自动化。可以分层：

1. Assist：辅助用户生成草稿。
2. Recommend：给出建议和候选。
3. Review：帮助检查和发现问题。
4. Automate with approval：自动执行但需确认。
5. Fully automate：完全自动执行。

很多场景适合先做辅助，而不是一开始就全自动。随着数据和信任积累，再逐步提高自动化程度。

2.11 ROI 初步估算

场景选择要做粗略 ROI 估算。

可以这样估：

月收益 = 月任务量 * 单次节省时间 * 人力成本 或 单次增量收入

月成本 = 模型调用 + 检索 + 工具 + 人审 + 运维 + 研发摊销

如果收益远小于成本，就算技术可行也不值得优先做。

ROI 不是只算钱，也可以算风险降低、客户满意度、交付速度。但最终要能连接到业务目标。

2.12 落地难度

落地难度来自多个方面：

1. 数据治理难度。
2. 系统集成难度。
3. 用户习惯改变。
4. 权限和合规要求。
5. 评估标准不清。
6. 延迟要求。
7. 成本约束。
8. 组织协作成本。

一个高价值但落地极难的场景，不一定适合作为第一个项目。早期可以选择价值中等但可快速验证的场景建立信任。

2.13 场景优先级矩阵

可以用二维矩阵评估：

业务价值高 + 落地难度低：优先做

业务价值高 + 落地难度高：拆分试点

业务价值低 + 落地难度低：可做 demo 或辅助功能

业务价值低 + 落地难度高：不做

更细一点，可以打分：

1. 用户痛点：1 到 5 分。
2. 任务频率：1 到 5 分。
3. 单次价值：1 到 5 分。
4. 数据可得性：1 到 5 分。
5. 可验证性：1 到 5 分。
6. 风险可控性：1 到 5 分。
7. 集成难度：反向 1 到 5 分。
8. 成本可控性：1 到 5 分。

评分不是为了假装精确，而是帮助团队把隐含判断显性化。

2.13.1 关键公式与场景选择指标速查

为了让场景选择不只停留在主观讨论，可以把每个候选场景记成一个结构化样本：

$s_i = (u_i, j_i, p_i, f_i, v_i, d_i, e_i, l_i, k_i, w_i, a_i, h_i)$

其中， u_i 是用户和利益相关方， j_i 是任务链路， p_i 是痛点强度， f_i 是月任务频次， v_i 是单次任务价值， d_i 是数据可得性， e_i 是结果可验证性， l_i 是大模型适配度， k_i 是剩余风险， w_i 是工作流适配度， a_i 是自动化层级， h_i 是落地难度。面试中你不一定要背这个 tuple，但要体现“我会把场景拆成用户、任务、价值、数据、验证、风险和工作流”。

真实需求不是一句“用户说想要”，而是痛点、当前替代方案和组织承诺共同成立。可以把痛点粗略归一化为：

$P_i = 0.4p_i + 0.3q_i + 0.3r_i$

其中, p_i 是问题严重度, q_i 是任务紧急度, r_i 是当前方案的不满意程度, 三者都归一化到 $[0, 1]$ 。如果用户没有当前替代方案, 也没有愿意投入的时间、数据或预算, 即使口头兴趣很强, 也要降低真实需求判断。

月度收益可以先做粗估:

$$B_i = f_i(h_i c_i + v_i g_i)$$

其中, f_i 是月任务量, h_i 是单次节省小时数, c_i 是小时成本或机会成本, v_i 是单次任务价值, g_i 是成功率、质量或转化率提升带来的增量比例。这个公式的直觉是: 收益来自“省时间”和“提升结果”两部分。

大模型适配度可以写成一个加权分:

$$L_i = 0.35n_i + 0.25c_i + 0.20g_i + 0.20o_i - 0.30r_i$$

其中, n_i 表示自然语言表达变化程度, c_i 表示上下文依赖, g_i 表示生成、总结、改写或推理需求, o_i 表示长尾输入比例, r_i 表示规则、搜索或模板已经足够的程度。这个分数提醒你: 不是所有智能需求都该上 LLM; 如果规则和搜索稳定解决, LLM 可能只是增加成本和不确定性。

为了把“业务价值”和“落地可行性”放在一起, 可以把收益压到 $[0, 1]$:

$$V_i = \min(1, \frac{B_i}{B_{\text{ref}}})$$

其中, B_{ref} 是团队约定的参考收益阈值。然后给候选场景一个优先级分:

$$S_i = 0.25P_i + 0.20V_i + 0.15L_i + 0.15D_i + 0.10E_i + 0.10W_i + 0.05A_i - 0.15H_i - 0.10K_i$$

其中, D_i 是数据准备度, E_i 是可验证性, W_i 是工作流适配度, A_i 是自动化层级合理性, H_i 是落地难度, K_i 是剩余风险。权重不是固定真理, 真实项目可以按行业调整; 关键是讲成“为什么优先做这个场景”可讨论、可复盘的判断。

试点门禁可以写成:

$$G_{\{\text{pilot}\}, i} = \mathbb{1}(P_i \geq \tau_p, V_i \geq \tau_v, L_i \geq \tau_L, D_i \geq \tau_d, E_i \geq \tau_e, W_i \geq \tau_k)$$

其中, $\tau_p, \tau_v, \tau_L, \tau_d, \tau_e, \tau_k$ 是团队预先设定的阈值。它表达的不是“分数高就一定成功”, 而是“痛点、收益、LLM 适配度、数据、验证和风险至少要同时过基本线, 才适合进入试点”。

2.14 试点场景

大模型产品通常适合先试点。

好的试点场景:

1. 用户明确。
2. 数据可得。
3. 任务边界清楚。
4. 风险可控。
5. 指标可衡量。
6. 能在短周期看到效果。
7. 成功后可以扩展。

试点不是做玩具 demo, 而是用受控范围验证真实价值。

2.15 和业务方沟通的问题清单

和业务方沟通时, 可以问:

1. 现在这个任务谁在做?
2. 每天或每月发生多少次?
3. 单次大约耗时多久?
4. 当前错误率或投诉点是什么?
5. 成功后希望提升哪个指标?
6. 可接受的错误是什么, 不可接受的错误是什么?
7. 是否有历史数据和标准答案?

8. 是否需要审批或合规评估？
9. 用户是否愿意改变工作流？
10. 如果只能辅助一部分，最希望辅助哪一步？

这些问题比直接问“你想要一个 AI 功能吗”更有效。

2.16 常见失败模式

1. 没有真实用户。
2. 痛点不够痛。
3. 任务频率太低。
4. 数据不可用或质量差。
5. 风险不可控。
6. 评估指标不清。
7. 用户工作流接不进去。
8. 成本高于收益。
9. 一开始就追求全自动。
10. 技术团队和业务团队目标不一致。

场景选择阶段发现问题，是好事。越早发现不值得做，越能节省成本。

2.17 面试题：如何选择大模型落地场景

回答要点：

我会从用户痛点、任务频率、单次价值、数据可得性、可验证性、容错率、成本和风险几个维度评估。优先选择高频或高价值、数据可得性高的场景。

2.18 面试题：如何判断需求是否真实

回答要点：

我不会只听用户说想要什么，而会看他们现在是否已经花时间或成本解决这个问题，任务发生频率如何，现有方案痛点在哪里，成功概率如何。

2.19 最小可运行场景选择审计 demo

下面的 0 依赖 demo 演示一个教学版 scenario selection audit：输入若干候选场景，输出真实需求分、月度收益、大模型适配度、优先级分、推荐动作和被拒原因。它不是生产评分器，而是帮助你把场景选择从“拍脑袋”变成“可解释表格”。

```
def clamp01(x):  
    return max(0.0, min(1.0, x))
```

```
AUTOMATION_SCORE = {  
    "assist": 0.55,  
    "review": 0.65,  
    "approval": 0.75,  
    "full": 0.90,  
}
```

```
SCENARIOS = [  
    {  
        "name": "support_rag",  
        "user_pain": 0.88,  
        "monthly_frequency": 4200,  
        "time_saved_hours": 0.07,
```

```

    "hourly_cost": 35,
    "value_per_task": 4,
    "quality_uplift": 0.25,
    "language_variability": 0.90,
    "context_need": 0.85,
    "generation_need": 0.65,
    "long_tail": 0.90,
    "rule_sufficiency": 0.25,
    "data_readiness": 0.82,
    "verifiability": 0.78,
    "risk_control": 0.86,
    "workflow_fit": 0.88,
    "automation_level": "assist",
    "implementation_difficulty": 0.42,
    "current_workaround": True,
    "stakeholder_alignment": 0.88,
  },
  {
    "name": "contract_review",
    "user_pain": 0.90,
    "monthly_frequency": 90,
    "time_saved_hours": 1.8,
    "hourly_cost": 140,
    "value_per_task": 320,
    "quality_uplift": 0.25,
    "language_variability": 0.80,
    "context_need": 0.90,
    "generation_need": 0.55,
    "long_tail": 0.55,
    "rule_sufficiency": 0.35,
    "data_readiness": 0.62,
    "verifiability": 0.66,
    "risk_control": 0.52,
    "workflow_fit": 0.64,
    "automation_level": "review",
    "implementation_difficulty": 0.76,
    "current_workaround": True,
    "stakeholder_alignment": 0.72,
  },
  {
    "name": "meeting_notes",
    "user_pain": 0.52,
    "monthly_frequency": 900,
    "time_saved_hours": 0.12,
    "hourly_cost": 45,
    "value_per_task": 1.2,
    "quality_uplift": 0.10,
    "language_variability": 0.70,
    "context_need": 0.45,
    "generation_need": 0.65,
    "long_tail": 0.50,
    "rule_sufficiency": 0.50,
    "data_readiness": 0.76,
    "verifiability": 0.42,
    "risk_control": 0.86,
  },

```

```

        "workflow_fit": 0.72,
        "automation_level": "assist",
        "implementation_difficulty": 0.25,
        "current_workaround": True,
        "stakeholder_alignment": 0.52,
    },
    {
        "name": "refund_automation",
        "user_pain": 0.82,
        "monthly_frequency": 1300,
        "time_saved_hours": 0.06,
        "hourly_cost": 30,
        "value_per_task": 8,
        "quality_uplift": 0.18,
        "language_variability": 0.45,
        "context_need": 0.55,
        "generation_need": 0.35,
        "long_tail": 0.45,
        "rule_sufficiency": 0.75,
        "data_readiness": 0.58,
        "verifiability": 0.72,
        "risk_control": 0.38,
        "workflow_fit": 0.51,
        "automation_level": "full",
        "implementation_difficulty": 0.68,
        "current_workaround": True,
        "stakeholder_alignment": 0.50,
    },
    {
        "name": "generic_chatbot",
        "user_pain": 0.32,
        "monthly_frequency": 180,
        "time_saved_hours": 0.05,
        "hourly_cost": 35,
        "value_per_task": 2,
        "quality_uplift": 0.04,
        "language_variability": 0.65,
        "context_need": 0.40,
        "generation_need": 0.45,
        "long_tail": 0.60,
        "rule_sufficiency": 0.65,
        "data_readiness": 0.35,
        "verifiability": 0.30,
        "risk_control": 0.62,
        "workflow_fit": 0.34,
        "automation_level": "assist",
        "implementation_difficulty": 0.42,
        "current_workaround": False,
        "stakeholder_alignment": 0.35,
    },
]

```

```

def reality_score(s):
    observed = 1.0 if s["current_workaround"] else 0.0

```

```

freq_score = min(1.0, s["monthly_frequency"] / 1000.0)
score = (
    0.35 * s["user_pain"]
    + 0.25 * freq_score
    + 0.20 * s["workflow_fit"]
    + 0.10 * observed
    + 0.10 * s["stakeholder_alignment"]
)
return round(score, 3)

def monthly_benefit(s):
    time_value = s["time_saved_hours"] * s["hourly_cost"]
    quality_value = s["value_per_task"] * s["quality_uplift"]
    return round(s["monthly_frequency"] * (time_value + quality_value), 2)

def llm_fit_score(s):
    score = (
        0.35 * s["language_variability"]
        + 0.25 * s["context_need"]
        + 0.20 * s["generation_need"]
        + 0.20 * s["long_tail"]
        - 0.30 * s["rule_sufficiency"]
    )
    return round(clamp01(score), 3)

def audit(s, benefit_ref=20000.0):
    benefit = monthly_benefit(s)
    benefit_score = clamp01(benefit / benefit_ref)
    llm_fit = llm_fit_score(s)
    automation = AUTOMATION_SCORE[s["automation_level"]]
    residual_risk = 1.0 - s["risk_control"]
    priority = (
        0.25 * s["user_pain"]
        + 0.20 * benefit_score
        + 0.15 * llm_fit
        + 0.15 * s["data_readiness"]
        + 0.10 * s["verifiability"]
        + 0.10 * s["workflow_fit"]
        + 0.05 * automation
        - 0.15 * s["implementation_difficulty"]
        - 0.10 * residual_risk
    )
    priority = round(clamp01(priority), 3)
    real_demand = (
        reality_score(s) >= 0.60
        and s["current_workaround"]
        and s["monthly_frequency"] * s["value_per_task"] > 0
    )

    rejected_reasons = []
    if not real_demand:
        rejected_reasons.append("real_demand")

```



```

if s["user_pain"] < 0.60:
    rejected_reasons.append("pain")
if benefit_score < 0.18:
    rejected_reasons.append("business_value")
if llm_fit < 0.55:
    rejected_reasons.append("llm_fit")
if s["data_readiness"] < 0.55:
    rejected_reasons.append("data")
if s["verifiability"] < 0.55:
    rejected_reasons.append("verifiability")
if s["risk_control"] < 0.60:
    rejected_reasons.append("risk_control")
if s["workflow_fit"] < 0.55:
    rejected_reasons.append("workflow")
if s["implementation_difficulty"] > 0.70:
    rejected_reasons.append("difficulty")

if not real_demand or s["risk_control"] < 0.45:
    recommendation = "reject"
elif not rejected_reasons and priority >= 0.70:
    recommendation = "pilot"
elif priority >= 0.58 and s["user_pain"] >= 0.65 and benefit_score >= 0.35:
    recommendation = "split_pilot"
elif priority >= 0.43:
    recommendation = "defer"
else:
    recommendation = "reject"

return {
    "real_demand": real_demand,
    "monthly_benefit": benefit,
    "benefit_score": round(benefit_score, 3),
    "llm_fit": llm_fit,
    "priority_score": priority,
    "recommendation": recommendation,
    "rejected_reasons": rejected_reasons,
}

```

```

audits = {s["name"]: audit(s) for s in SCENARIOS}
ranked = sorted(
    ((name, a["priority_score"], a["recommendation"]) for name, a in audits.items()),
    key=lambda item: item[1],
    reverse=True,
)

print("ranked=", ranked)
print("pilot_candidates=", [name for name, a in audits.items() if a["recommendation"] == "pilot"])
print("split_pilots=", [name for name, a in audits.items() if a["recommendation"] == "split_pilot"])
print("rejected=", {name: a["rejected_reasons"] for name, a in audits.items() if a["recommendation"] == "reject"})
for name in [item[0] for item in ranked]:
    print(name, audits[name])

```

这段 demo 的关键不是某个权重一定正确，而是训练你形成面试表达：先证明需求真实，再估算收益，再判断 LLM 是否必要，最后把数据、验证、风险、工作流和试点边界合起来做决策。输出中 support_rag 适合直接试点，contract_review 价值高但风险和难度较高，适合拆成受控试点；meeting_notes 暂缓；refund_automation 和

generic_chatbot 不能直接推进。

2.20 本章小结

大模型产品化首先要选对场景。好的场景通常有真实用户、明确痛点、高频或高价值、数据可得、风险可控、结果可验证，并且能接入现有 workflow。场景选择不是拍脑袋，而是对用户、任务、数据、成本、风险和 ROI 的综合判断。

下一章会进入“模型能力到产品体验”，讨论如何把模型的理解、生成、检索、推理和工具调用能力，转化为用户真正感受到的产品体验。

第三章：模型能力到产品体验

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对四类口径：OpenAI evals / evaluation best practices 强调上线前要把任务、样本、指标和回归集定义清楚；OpenAI latency optimization 和 structured outputs 资料提示延迟、流式返回、缓存、结构化格式会直接影响体验；Google HEART 框架强调从用户目标拆出可观测 signals 和 metrics；Google SRE 的 SLI / SLO / error budget 口径提醒我们不能只看平均延迟，要看尾延迟和可接受失败率；NIST AI RMF / Generative AI Profile 提醒产品体验还要包含透明性、风险提示、人审和治理边界。

本章不展开具体 prompt 模板、RAG 检索算法、Agent 工具协议、前端设计系统或线上 A/B 实验平台。这里的核心问题是：如何把模型的理解、生成、检索、推理和工具能力，转成用户能感知的“快、准、稳、可控、可信、可恢复”的产品体验。

模型能力不等于产品体验。一个模型 benchmark 很高，但如果响应太慢、回答不稳定、无法引用来源、格式不可控、拒答策略混乱、用户不知道怎么追问，产品体验依然会很差。大模型产品化的关键，是把理解、生成、检索、推理和工具调用能力，转化为用户可感知、可信任、可持续使用的体验。

本章系统讲模型能力到产品体验的转化：延迟、准确性、可控性、解释性、拒答、风格、交互设计、引用、置信度、用户反馈、体验指标和常见失败模式。

3.1 为什么模型能力不等于体验

模型能力是技术侧概念，产品体验是用户侧感受。

例如模型有很强的总结能力，但用户体验可能受这些因素影响：

1. 等待时间太长。
2. 摘要太长或太短。
3. 重点不符合用户 workflow。
4. 无法追溯原文。
5. 风格不符合业务场景。
6. 结果每次不一致。
7. 用户不知道如何修改输出。

面试回答：

模型能力需要通过产品设计转化为体验。用户感知到的不是 benchmark，而是任务是否更快完成、答案是否可信、交互是否顺畅、错误

3.2 延迟体验

延迟是用户最直接感知的指标之一。

大模型产品的延迟来自：

1. 模型推理。
2. RAG 检索。

3. reranker。
4. 工具调用。
5. 多轮 Agent 循环。
6. 网络传输。
7. 排队和限流。

延迟不是越低越好，而是要和任务价值匹配。闲聊需要快；复杂报告可以慢一点；高价值分析可以等待，但要给用户进度反馈。

3.3 如何改善延迟感知

产品上可以降低用户对延迟的负面感受。

方法：

1. 流式输出。
2. 显示进度。
3. 先返回摘要，再补充细节。
4. 后台执行长任务。
5. 给用户可取消选项。
6. 缓存常见结果。
7. 对简单问题走快速路径。

例如 Agent 执行复杂任务时，可以展示“正在检索资料、正在生成草稿、正在检查引用”，而不是让用户空等。

3.4 准确性体验

用户感知的准确性不只是最终答案对不对。

还包括：

1. 是否理解了用户意图。
2. 是否覆盖关键约束。
3. 是否引用正确来源。
4. 是否承认不确定性。
5. 是否避免编造。
6. 是否能被用户验证。
7. 是否在追问中保持一致。

对于企业应用，准确性往往比“回答得流畅”更重要。流畅但错误的答案会损害信任。

3.5 可控性

可控性指用户能否影响输出结果。

常见控制维度：

1. 长度。
2. 风格。
3. 格式。
4. 语气。
5. 受众。
6. 数据范围。
7. 引用来源。
8. 是否保守或发散。

大模型默认输出可能不符合用户预期。产品可以提供模板、参数、选项、示例和编辑入口，让用户更容易控制结果。

3.6 结构化输出

结构化输出能显著提升体验和稳定性。

例如：

结论：...

依据：...

风险：...

下一步建议：...

结构化输出适合：

1. 报告生成。
2. 工单总结。
3. 合同审查。
4. 客服回复。
5. 数据分析。
6. 面试反馈。

结构化输出的价值是降低用户理解成本，也便于后续系统处理。

3.7 解释性

用户需要知道答案为什么可信。

解释可以包括：

1. 依据来源。
2. 推理摘要。
3. 关键假设。
4. 不确定点。
5. 反例或限制。
6. 工具验证结果。

但解释不是越长越好。产品中更重要的是可验证解释，而不是完整内部推理。对于 RAG 产品，引用原文通常比长篇自我解释更有价值。

3.8 引用和来源

引用是建立信任的重要机制。

好的引用应该：

1. 指向具体文档或段落。
2. 支持对应结论。
3. 标明时间或版本。
4. 区分事实和推断。
5. 允许用户跳转查看原文。

坏引用包括：

1. 引用不相关文档。
2. 引用只支持部分结论。
3. 编造来源。
4. 用过期文档支持新结论。

引用准确率是很多企业 RAG 产品的重要指标。

3.9 拒答体验

拒答不是简单说“不行”。好的拒答应该说明原因，并给出可行替代方案。

例如：

我不能直接给出医疗诊断，但可以帮你整理症状、列出需要咨询医生的问题，并提醒你尽快就医。

拒答体验要处理：

1. 安全限制。
2. 权限不足。
3. 信息不足。
4. 任务超出能力。
5. 数据不可访问。
6. 高风险建议。

拒答太少会有风险；拒答太多会降低可用性。产品需要平衡安全和帮助性。

3.10 风格一致性

风格是产品体验的一部分。

不同产品需要不同风格：

1. 客服：礼貌、简洁、可执行。
2. 法务：谨慎、严谨、保留条件。
3. 编程助手：直接、技术准确。
4. 教育产品：循序渐进、有引导。
5. 销售助手：有说服力但不能夸大。

风格一致性可以通过系统 prompt、模板、示例、后处理和评估来保证。

3.11 交互设计

大模型产品不一定只有聊天框。

常见交互形态：

1. 聊天式问答。
2. 文档侧边栏助手。
3. 表单自动填写。
4. 一键生成草稿。
5. 批量处理。
6. workflow 节点。
7. IDE 插件。
8. 浏览器插件。
9. 报告生成器。

聊天框灵活，但不一定最高效。很多企业场景更适合把模型嵌入现有流程，而不是让用户专门打开一个聊天页面。

3.12 用户引导

用户需要知道如何使用 AI 功能。

可以提供：

1. 示例问题。
2. 推荐任务。
3. Prompt 模板。
4. 输入字段。
5. 可选参数。

6. 结果编辑入口。
7. 反馈按钮。

不要假设用户会写好 prompt。好的产品应该把 prompt 工程隐藏在体验设计里。

3.13 错误恢复

模型错误不可避免，关键是能否恢复。

恢复方式：

1. 允许用户追问。
2. 提供重新生成。
3. 支持修改约束。
4. 显示引用让用户检查。
5. 给出“哪里不对”的反馈入口。
6. 对低置信答案提醒用户。
7. 高风险场景转人工。

产品体验中，错误恢复比假装永远正确更重要。

3.14 信任校准

信任校准是让用户既不过度信任，也不过度怀疑。

方法：

1. 标注不确定性。
2. 展示来源。
3. 区分事实和建议。
4. 对高风险内容提醒复核。
5. 不夸大能力。
6. 对无法完成的任务明确说明。

如果产品总是自信地给错答案，用户会失去信任；如果产品总是保守拒答，用户也不会继续使用。

3.15 个性化体验

大模型可以根据用户角色和偏好调整体验。

例如：

1. 给新手更多解释。
2. 给专家更短结论。
3. 给销售输出客户化话术。
4. 给工程师输出代码和命令。
5. 根据企业规范调整格式。

个性化要注意隐私和可控性。用户应该知道哪些偏好被记住，并能修改或删除。

3.16 体验指标

常见体验指标：

1. 任务完成率。
2. 首次成功率。
3. 平均完成时间。
4. 用户满意度。
5. 追问率。
6. 重新生成率。

7. 人工接管率。
8. 结果采纳率。
9. 引用点击率。
10. 留存率。

技术指标要和体验指标关联。例如延迟下降是否提升留存，引用准确率是否提升采纳率。

3.16.1 关键公式与能力到体验指标速查

为了避免只说“模型更强所以体验更好”，可以把一次产品交互记成结构化样本：

$$e_i = (x_i, u_i, m_i, \hat{y}_i, y_i, z_i, \ell_i, c_i, r_i, h_i)$$

其中， x_i 是用户输入， u_i 是用户角色或场景， m_i 是产品形态， $\hat{y}_i$ 是模型输出， y_i 是期望结果或人工标注， z_i 是可用证据或工具结果， ℓ_i 是端到端延迟， c_i 是成本， r_i 是风险标签， h_i 是用户反馈或人工审核结果。

模型能力可以先抽象成一个能力向量：

$$a_i = (U_i, G_i, R_i, Q_i, T_i, F_i, S_i)$$

其中， U_i 是意图理解， G_i 是生成质量， R_i 是检索和引用能力， Q_i 是复杂推理能力， T_i 是工具调用能力， F_i 是格式稳定性， S_i 是安全和拒答能力。这个向量本身还不是体验，它必须映射到用户可观测指标。

任务成功率是最基础的体验指标：

$$R_{\{\mathrm{succ}\}} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\hat{y}_i \in \{\mathrm{passes}\}) V_i$$

其中， V_i 是任务验收函数，可以是单元测试、人工 rubric、引用检查、业务规则或用户采纳。面试中要强调：验收函数必须贴近真实任务，而不是只看模型输出是否流畅。

延迟满意度可以按 SLO 粗略归一化：

$$L_i = \max(0, 1 - \frac{\ell_{p95,i}}{\tau_i})$$

其中， $\ell_{p95,i}$ 是 P95 端到端延迟， τ_i 是该产品形态的可接受延迟阈值。闲聊、客服、IDE 补全和长报告的 τ_i 不一样；复杂任务可以慢，但需要进度反馈和可取消机制。

引用支持率衡量“用户为什么敢信”：

$$C_{\{\mathrm{cite}\}} = \frac{\sum_i n_{\{\mathrm{sup}\},i}}{\sum_i n_{\{\mathrm{claim}\},i}}$$

其中， $n_{\mathrm{sup},i}$ 是被证据支持的关键结论数， $n_{\mathrm{claim},i}$ 是需要证据支持的关键结论数。对 RAG 和企业知识助手来说，引用支持率通常比“回答很像正确答案”更重要。

结构化输出稳定率衡量产品能否接入 workflow：

$$F_{\{\mathrm{fmt}\}} = \frac{N_{\{\mathrm{valid}\}}}{N}$$

其中， N_{valid} 是符合 JSON、表格、字段或业务模板要求的输出数。结构化输出失败会导致后续系统无法消费，即使文本内容看起来不错，也会破坏体验。

错误恢复率衡量用户遇到错误后能否继续完成任务：

$$R_{\{\mathrm{rec}\}} = \frac{N_{\{\mathrm{recovered}\}}}{N_{\{\mathrm{failed}\}}}$$

其中， N_{failed} 是用户发现或系统检测到的失败次数， $N_{\mathrm{recovered}}$ 是通过追问、重新生成、修改约束、人工接管或工具校验恢复成功的次数。好产品不是永远不犯错，而是错误可见、可解释、可恢复。

信任校准可以用预测置信度和实际正确率的差距表示：

$$K_{\{\mathrm{cal}\}} = 1 - \frac{1}{B} \sum_{b=1}^B |p_b - a_b|$$

其中， p_b 是第 b 个置信度桶的平均预测置信度， a_b 是该桶真实正确率。过度自信会让用户误信错误答案；过度保守会造成过度拒答和低采用率。

综合体验分可以写成：

$$S_{\mathrm{ux}}=0.25R_{\mathrm{succ}}+0.15L+0.15C_{\mathrm{cite}}+0.15F_{\mathrm{fmt}}+0.10C_{\mathrm{ctrl}}+0.10R_{\mathrm{over}}-0.20R_{\mathrm{unsafe}}$$

其中, C_{ctrl} 是用户可控性满意度, R_{over} 是过度拒答率, R_{unsafe} 是不安全或不合规输出率。权重可以按产品调整, 核心思想是: 体验必须同时看质量、延迟、可信、格式、可控、恢复和风险。

上线门禁可以写成:

$$G_{\mathrm{ux}}=\mathbb{1}(R_{\mathrm{succ}}\geq \tau_s, \ell_{\mathrm{p95}}\leq \tau_l, C_{\mathrm{cite}}\geq \tau_c, F_{\mathrm{fmt}}\leq \tau_f, R_{\mathrm{over}}\leq \tau_o, R_{\mathrm{unsafe}}\leq \tau_{\mathrm{unsafe}})$$

这个门禁提醒面试回答不要只说“换更强模型”。如果 P95 延迟、引用支持、格式稳定、安全和恢复不过线, 高分模型也不该直接进入生产体验。

3.17 常见失败模式

1. 模型能力强, 但交互入口复杂。
2. 回答准确, 但太慢。
3. 输出格式不稳定, 无法接入流程。
4. 拒答策略过度保守或过度宽松。
5. 没有引用, 用户不敢信。
6. 错误后没有反馈入口。
7. 风格不符合业务场景。
8. 用户不会写 prompt。
9. 产品只追求炫技, 不解决任务。
10. 体验指标和模型指标脱节。

产品体验是模型、系统、交互和业务流程共同作用的结果。

3.18 面试题: 如何把模型能力转成产品体验

回答要点:

我会先明确用户任务和成功标准, 再把模型能力映射到体验指标。例如理解能力对应意图识别和任务完成率, 生成能力对应草稿质量。

3.19 面试题: 为什么一个高分模型产品体验可能不好

回答要点:

因为 benchmark 只覆盖部分能力, 而产品体验还受延迟、稳定性、可控性、交互设计、引用、拒答、风格、成本和错误恢复影响。一

3.20 最小可运行能力到体验审计 demo

下面的 0 依赖 demo 演示一个教学版 experience audit: 输入若干产品形态的 toy 观测指标, 输出任务成功、P95 延迟、引用支持、格式稳定、错误恢复、信任校准、综合体验分、失败原因和体验门禁。它的目的不是替代真实 A/B test, 而是帮助你把“模型能力”翻译成“用户体验是否过线”。

```
def safe_div(num, den, default=0.0):
    return default if den == 0 else num / den
```

```
def clamp01(x):
    return max(0.0, min(1.0, x))
```

```
PRODUCTS = [
    {
        "name": "support_rag_sidebar",
        "interactions": 120,
        "task_success": 101,
```



```

    "adopted": 82,
    "p95_latency_ms": 3100,
    "latency_slo_ms": 4000,
    "claims": 190,
    "supported_claims": 174,
    "structured_outputs": 115,
    "control_satisfied": 96,
    "failures": 19,
    "recovered": 14,
    "confidence_bins": [(0.55, 0.58, 38), (0.75, 0.72, 44), (0.90, 0.88, 38)],
    "over_refusals": 3,
    "unsafe_outputs": 0,
    "avg_cost": 0.19,
  },
  {
    "name": "code_copilot_inline",
    "interactions": 90,
    "task_success": 62,
    "adopted": 58,
    "p95_latency_ms": 740,
    "latency_slo_ms": 900,
    "claims": 0,
    "supported_claims": 0,
    "structured_outputs": 82,
    "control_satisfied": 70,
    "failures": 23,
    "recovered": 16,
    "confidence_bins": [(0.60, 0.62, 26), (0.78, 0.76, 38), (0.92, 0.80, 26)],
    "over_refusals": 1,
    "unsafe_outputs": 0,
    "avg_cost": 0.07,
  },
  {
    "name": "analyst_agent_report",
    "interactions": 60,
    "task_success": 43,
    "adopted": 31,
    "p95_latency_ms": 11800,
    "latency_slo_ms": 8000,
    "claims": 160,
    "supported_claims": 118,
    "structured_outputs": 49,
    "control_satisfied": 36,
    "failures": 17,
    "recovered": 8,
    "confidence_bins": [(0.55, 0.50, 20), (0.78, 0.68, 22), (0.93, 0.62, 18)],
    "over_refusals": 4,
    "unsafe_outputs": 1,
    "avg_cost": 1.85,
  },
  {
    "name": "generic_chat_entry",
    "interactions": 80,
    "task_success": 38,
    "adopted": 24,

```

```

        "p95_latency_ms": 1800,
        "latency_slo_ms": 2500,
        "claims": 90,
        "supported_claims": 42,
        "structured_outputs": 45,
        "control_satisfied": 30,
        "failures": 42,
        "recovered": 9,
        "confidence_bins": [(0.50, 0.44, 30), (0.75, 0.48, 28), (0.90, 0.42, 22)],
        "over_refusals": 10,
        "unsafe_outputs": 2,
        "avg_cost": 0.12,
    },
]

```

```

def calibration_score(bins):
    total = sum(count for _, _, count in bins)
    error = sum(abs(pred - actual) * count for pred, actual, count in bins)
    return round(clamp01(1.0 - safe_div(error, total)), 3)

```

```

def audit(product):
    n = product["interactions"]
    success = safe_div(product["task_success"], n)
    adoption = safe_div(product["adopted"], n)
    latency_score = clamp01(1.0 - product["p95_latency_ms"] / product["latency_slo_ms"])
    citation = 1.0 if product["claims"] == 0 else safe_div(product["supported_claims"], product["claims"])
    format_stability = safe_div(product["structured_outputs"], n)
    controllability = safe_div(product["control_satisfied"], n)
    recovery = safe_div(product["recovered"], product["failures"], default=1.0)
    calibration = calibration_score(product["confidence_bins"])
    over_refusal = safe_div(product["over_refusals"], n)
    unsafe_rate = safe_div(product["unsafe_outputs"], n)
    ux_score = (
        0.25 * success
        + 0.15 * latency_score
        + 0.15 * citation
        + 0.15 * format_stability
        + 0.10 * controllability
        + 0.10 * recovery
        + 0.10 * calibration
        + 0.05 * adoption
        - 0.15 * over_refusal
        - 0.20 * unsafe_rate
    )
    metrics = {
        "success_rate": round(success, 3),
        "adoption_rate": round(adoption, 3),
        "latency_score": round(latency_score, 3),
        "citation_support": round(citation, 3),
        "format_stability": round(format_stability, 3),
        "controllability": round(controllability, 3),
        "recovery_rate": round(recovery, 3),
        "calibration": calibration,
    }

```

```

    "over_refusal_rate": round(over_refusal, 3),
    "unsafe_rate": round(unsafe_rate, 3),
    "ux_score": round(clamp01(ux_score), 3),
}

failed = []
if metrics["success_rate"] < 0.72:
    failed.append("task_success")
if product["p95_latency_ms"] > product["latency_slo_ms"]:
    failed.append("p95_latency")
if metrics["citation_support"] < 0.80:
    failed.append("citation")
if metrics["format_stability"] < 0.82:
    failed.append("format")
if metrics["controllability"] < 0.65:
    failed.append("control")
if metrics["recovery_rate"] < 0.55:
    failed.append("recovery")
if metrics["calibration"] < 0.82:
    failed.append("calibration")
if metrics["over_refusal_rate"] > 0.08:
    failed.append("over_refusal")
if metrics["unsafe_rate"] > 0.0:
    failed.append("unsafe")

metrics["experience_gate"] = not failed and metrics["ux_score"] >= 0.72
metrics["failed_gates"] = failed
return metrics

```

```

audits = {product["name"]: audit(product) for product in PRODUCTS}
ranked = sorted(
    ((name, result["ux_score"], result["experience_gate"]) for name, result in audits.items()),
    key=lambda item: item[1],
    reverse=True,
)

print("ranked=", ranked)
print("ship_experience=", [name for name, result in audits.items() if result["experience_gate"]])
print("needs_rework=", {name: result["failed_gates"] for name, result in audits.items() if not result["experience_gate"]})
for name in [item[0] for item in ranked]:
    print(name, audits[name])

```

这段 demo 体现了本章核心判断：support_rag_sidebar 不是因为模型最强，而是因为成功率、引用、格式、恢复和延迟都过线，所以体验门禁通过；code_copilot_inline 延迟很好，但成功率还不够；analyst_agent_report 价值可能高，但 P95 延迟、引用和校准不足；generic_chat_entry 看起来灵活，但缺少明确任务、引用、控制和恢复，体验分最低。

3.21 本章小结

大模型产品体验不是模型能力的简单外显，而是模型、系统和交互设计共同塑造的结果。延迟、准确性、可控性、解释性、引用、拒答、风格、错误恢复和用户引导，都会决定用户是否愿意持续使用。

下一章会进入成本收益与 ROI，系统讨论大模型项目如何估算成本、收益、单位经济账和投入产出比。

第四章：成本收益与 ROI

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对四类口径：OpenAI 官方 pricing / Batch / prompt caching / latency optimization 资料提醒我们，模型调用成本由输入、输出、缓存命中、批处理、工具和延迟策略共同决定，而且价格会变化；OpenAI evals / evaluation best practices 强调 ROI 不能脱离任务成功率和回归评估；Google SRE 的 SLO / error budget 口径提醒我们成本优化不能破坏可靠性和尾延迟；FinOps Foundation 的云成本管理口径强调成本要按单位经济账、责任归属和持续优化来管理。

本章不写任何长期固定的 API 价格，也不替代企业财务模型、采购合同或真实云账单分析。正文中的价格都是 toy 参数，用来说明算法工程师在面试和项目复盘如何把 token、RAG、工具、人审、固定研发摊销、风险成本和业务收益放进同一张账。

大模型项目不能只问“效果好不好”，还要问“值不值得”。如果一个系统每次回答都很准确，但调用成本太高、人工审核太重、延迟太长、收益无法覆盖投入，就很难成为可持续产品。成本收益与 ROI，是大模型产品化绕不开的问题。

本章系统讲大模型项目的成本和收益：token 成本、GPU 成本、工程成本、数据成本、人工审核成本、运维成本；收益侧包括人工替代、效率提升、质量提升、收入增长和风险降低；最后讲 ROI 估算、单位经济账、成本优化和面试表达。

4.1 为什么要算 ROI

ROI，投入产出比，回答的是一个项目是否值得持续投入。

一个大模型 demo 可以不算 ROI，但产品必须算。原因是：

1. 模型调用有持续成本。
2. 用户规模扩大后成本会线性甚至超线性增长。
3. 人工审核和运维不能忽略。
4. 模型效果提升不一定带来业务收益。
5. 业务方需要判断优先级。

面试回答：

大模型项目必须算 ROI，因为模型能力只有转化为业务收益才可持续。ROI 要同时看收益和成本：收益包括节省人工、提升效率、增加

4.2 成本不只是 Token

很多人只算 token 成本，这是不够的。

完整成本包括：

1. 模型调用成本。
2. GPU 推理成本。
3. RAG 检索成本。
4. 向量库和存储成本。
5. reranker 和 verifier 成本。
6. Agent 工具调用成本。
7. 人工审核成本。
8. 数据清洗和标注成本。
9. 工程开发成本。
10. 运维监控成本。
11. 合规和安全成本。
12. 失败和事故成本。

产品化要看全生命周期成本，而不是单次模型调用报价。

4.3 Token 成本

Token 成本来自输入和输出。

影响因素：

1. Prompt 长度。
2. 用户输入长度。
3. RAG 拼接文档长度。
4. 历史上下文长度。
5. 输出长度。
6. 重试次数。
7. 多样本采样次数。
8. Agent 多轮调用。

例如普通问答一次调用可能只消耗几千 token；Agentic RAG 可能多轮检索、多次阅读、多次生成，token 成本显著更高。

4.4 GPU 成本

如果自部署模型，要考虑 GPU 成本。

包括：

1. GPU 采购或租用。
2. 推理服务部署。
3. 显存占用。
4. 吞吐和并发。
5. 量化和加速优化。
6. 空闲资源浪费。
7. 运维人员成本。

自部署不一定比 API 便宜。只有当调用量足够大、模型大小合适、团队有推理优化能力时，自部署才可能有成本优势。

4.5 RAG 成本

RAG 的成本包括：

1. 文档解析。
2. 分块。
3. embedding。
4. 向量库存储。
5. 检索。
6. reranking。
7. 文档更新。
8. 权限过滤。
9. 引用校验。

RAG 不是免费增强。企业知识库越大、权限越复杂、更新越频繁，成本越高。

4.6 Agent 成本

Agent 成本通常比普通问答高。

来源：

1. 多轮模型调用。
2. 工具调用。
3. 搜索和检索。

4. 代码执行或浏览器操作。
5. 失败重试。
6. trace 日志存储。
7. 安全检查。
8. 人工确认。

Agent 适合高价值任务，不适合所有简单请求默认开启。否则成本和延迟都会失控。

4.7 人工审核成本

很多大模型产品需要人在回路。

人工审核成本包括：

1. 审核时间。
2. 审核人员培训。
3. 质检流程。
4. 复核争议样本。
5. 处理用户投诉。
6. 标注反馈数据。

如果模型节省了 5 分钟，但审核要花 4 分钟，实际收益就很有限。人机协同设计要尽量让人只审核关键点，而不是重新做一遍。

4.8 工程和运维成本

产品上线后需要持续维护。

工程成本包括：

1. 前端和后端开发。
2. 模型服务接入。
3. 数据管道。
4. 权限系统。
5. 监控告警。
6. 日志和审计。
7. 回归评估。
8. 灰度发布。
9. 线上问题排查。

大模型产品不是接一个 API 就结束。越接近企业级应用，工程和运维成本越重要。

4.9 收益类型

大模型项目收益可以分为几类：

1. 节省人工。
2. 提升效率。
3. 提高质量。
4. 增加收入。
5. 降低风险。
6. 提升用户体验。
7. 加速知识沉淀。
8. 扩大服务覆盖。

不同收益的可量化程度不同。节省人工最容易算；用户满意度和知识沉淀较难直接量化，但仍然可以通过代理指标衡量。

4.10 人工替代收益

最常见收益是节省人工时间。

例如：

每月 10000 个客服问题

每个问题原来人工处理 5 分钟

AI 能自动解决 40%

每小时人工成本 60 元

粗略收益：

$10000 * 40\% * 5 / 60 * 60 = 20000$ 元/月

这里还要扣除模型成本、系统成本和人审成本。

4.11 效率提升收益

有些场景不是替代人工，而是提高效率。

例如：

1. 销售写方案从 2 小时降到 30 分钟。
2. 工程师查文档从 15 分钟降到 3 分钟。
3. 法务初审合同从 1 小时降到 20 分钟。
4. 数据分析师生成初稿从半天降到 1 小时。

效率提升不一定减少人数，但可以提升产能、缩短交付周期、支持更多客户。

4.12 质量收益

大模型也可能提升质量。

例如：

1. 减少漏检风险。
2. 提高回复一致性。
3. 提升文档完整性。
4. 降低新人上手成本。
5. 让专家经验可复用。

质量收益较难量化，但可以通过错误率、投诉率、返工率、审核通过率等指标间接衡量。

4.13 收入增长

大模型也可能带来收入增长。

例如：

1. 提高销售转化率。
2. 提升用户留存。
3. 增加付费功能。
4. 支持更多客户同时服务。
5. 提升客单价。

收入增长类项目要特别注意归因。转化率提升可能来自多个因素，不能轻易把所有增长都归因给大模型。

4.14 风险降低收益

有些收益来自降低风险。

例如：

1. 合同风险提示。
2. 内容合规审核。
3. 安全告警总结。
4. 财务异常检测辅助。
5. 客诉预警。

风险降低的收益可能不高频，但单次事故代价大。评估时可以看历史事故成本、风险暴露数量和人工审核覆盖率。

4.15 单位经济账

单位经济账是看单次任务是否划算。

例如每处理一个客服问题：

模型成本：0.05 元
 检索成本：0.01 元
 人审成本：0.20 元
 总成本：0.26 元
 节省人工成本：0.80 元
 净收益：0.54 元

如果任务量很大，单次净收益很关键。小小的成本差异会被规模放大。

4.16 ROI 估算模板

一个简单模板：

月任务量 = N
 自动化比例 = A
 单次节省时间 = T 小时
 人工时薪 = W
 月收益 = N * A * T * W

月成本 = 模型成本 + 检索成本 + 人审成本 + 运维成本 + 研发摊销
 ROI = 月收益 / 月成本

如果是收入增长场景，可以把月收益换成增量收入；如果是风险降低场景，可以用预期风险损失下降来估算。

4.16.1 关键公式与 ROI 指标速查

为了把 ROI 讲清楚，可以把候选产品化场景记成一个账本样本：

$b_i = (N_i, A_i, Q_i, T_i, W_i, V_i, P_i, C_i, F_i, R_i)$

其中， N_i 是月任务量， A_i 是采用率或覆盖率， Q_i 是任务成功率提升， T_i 是单次节省时间， W_i 是小时成本， V_i 是单次业务价值， P_i 是当前单次失败或返工概率， C_i 是单次可变成本， F_i 是月固定成本， R_i 是预期风险成本或风险下降收益。

模型调用成本可以先按输入和输出 token 拆开：

$C_{\{\mathrm{model}\},i} = M_i \left(\frac{I_i p_{\{\mathrm{in}\}}}{K} (1 - H_i) + \frac{O_i p_{\{\mathrm{out}\}}}{K} \right)$

其中， M_i 是单任务模型调用次数， I_i 是平均输入 token， O_i 是平均输出 token， p_{in} 和 p_{out} 是每 K 个 token 的 toy 单价， H_i 是缓存命中带来的输入折扣比例。这里故意写成变量，因为真实价格、缓存折扣和批处理折扣要随官方计费口径更新。

单次任务的全可变成本可以写成：

$C_{\{\mathrm{var}\},i} = C_{\{\mathrm{model}\},i} + C_{\{\mathrm{rag}\},i} + C_{\{\mathrm{tool}\},i} + C_{\{\mathrm{review}\},i} + C_{\{\mathrm{retry}\},i}$

其中, C_{rag} 包含 embedding、检索、rerank、向量库和引用校验, C_{tool} 是工具调用, C_{review} 是人工审核, C_{retry} 是重试, C_{risk} 是预期失败成本。

月度总成本是:

$$C_{\{\mathrm{month}\},i} = N_i A_i C_{\{\mathrm{var}\},i} + F_i$$

其中, F_i 包括研发摊销、运维、监控、日志、合规、安全评估、数据更新和固定基础设施。很多 ROI 误判来自只算 C_{model} , 不算 F_i 。

单次收益可以粗略拆成节省时间、质量提升、收入增长和风险下降:

$$B_{\{\mathrm{task}\},i} = A_i (T_i W_i + Q_i V_i + G_i) + R_i$$

其中, G_i 是单次增量收入, R_i 是单次任务对应的预期风险损失下降。注意 A_i 是采用率: 模型生成了结果但用户不采用, 不能直接算成收益。

月度收益是:

$$B_{\{\mathrm{month}\},i} = N_i B_{\{\mathrm{task}\},i}$$

净收益、收益成本比和净 ROI 分别是:

$$P_{\{\mathrm{net}\},i} = B_{\{\mathrm{month}\},i} - C_{\{\mathrm{month}\},i}$$

$$R_{\{\mathrm{bc}\},i} = \frac{B_{\{\mathrm{month}\},i}}{C_{\{\mathrm{month}\},i}}$$

$$R_{\{\mathrm{roi}\},i} = \frac{B_{\{\mathrm{month}\},i} - C_{\{\mathrm{month}\},i}}{C_{\{\mathrm{month}\},i}}$$

面试中要先说明自己采用哪个口径: R_{bc} 是收益成本比, R_{roi} 是扣除成本后的净 ROI。两者都可以用, 但不能混着讲。

如果有一性投入 U_i , 回本周期可以写成:

$$P_{\{\mathrm{back}\},i} = \frac{U_i}{\max(P_{\{\mathrm{net}\},i}, \epsilon)}$$

其中, ϵ 是避免除零的极小正数。真实表达时如果净收益小于等于 0, 就不要报一个看似精确的回本月数, 而应该说“当前假设下无法回本”。

盈亏平衡任务量可以写成:

$$N_{\{\mathrm{be}\},i} = \frac{F_i}{\max(B_{\{\mathrm{task}\},i} - A_i C_{\{\mathrm{var}\},i}, \epsilon)}$$

如果单次毛利为负, 规模越大亏得越多; 这时不能靠“用户量增长”解决问题, 必须先降成本、提高采用率或重新选场景。

敏感性分析至少要看三个变量:

$$\Delta P_{\{\mathrm{net}\}} = \frac{\partial P_{\{\mathrm{net}\}}}{\partial A} \Delta A + \frac{\partial P_{\{\mathrm{net}\}}}{\partial C} \Delta C + \frac{\partial P_{\{\mathrm{net}\}}}{\partial B} \Delta B$$

直觉是: ROI 最容易被采用率、质量提升和可变成本打穿。一个看似盈利的方案, 如果采用率下降、重试率上升或人审成本增加, 很快就会变成亏损。

上线门禁可以写成:

$$G_{\{\mathrm{roi}\},i} = \mathbb{1}(P_{\{\mathrm{net}\},i} > 0, R_{\{\mathrm{bc}\},i} \geq \tau_b, P_{\{\mathrm{back}\},i} \leq \tau_p, C_{\{\mathrm{month}\},i} \leq \tau_c)$$

它表达的是: 月度净收益要为正, 收益成本比要过阈值, 回本周期要可接受, 且单次任务本身不能越做越亏。

4.17 成本优化手段

常见优化:

1. 小模型处理简单任务。
2. 大模型只处理复杂任务。
3. 缓存高频问题。
4. 控制上下文长度。
5. 优化 RAG top-k。

6. 减少无效重试。
7. 使用结构化输出减少后处理。
8. 对低价值请求降级。
9. 批处理离线任务。
10. 推理加速和量化。

成本优化不能牺牲关键体验和安全。要看 quality-cost trade-off。

4.18 ROI 的常见误区

1. 只算 API 成本，不算工程和人审。
2. 只算节省时间，不算错误成本。
3. 夸大自动化比例。
4. 忽略用户采用率。
5. 忽略长尾和失败处理。
6. 不做 A/B 测试。
7. 把所有业务增长都归因于 AI。
8. 没有持续监控成本。

ROI 估算是决策工具，不是包装项目的数字游戏。

4.19 面试题：如何估算大模型项目 ROI

回答要点：

我会先明确收益来源，是节省人工、提升效率、增加收入还是降低风险。然后估算任务量、自动化比例、单次节省时间或增量收入，再

4.20 面试题：如何降低大模型产品成本

回答要点：

可以从模型、上下文、调用策略和系统架构优化。简单任务用小模型或规则，大模型只处理复杂任务；控制 prompt 和 RAG 上下文长度

4.21 最小可运行 ROI / 单位经济账审计 demo

下面的 0 依赖 demo 演示一个教学版 ROI audit: 输入 toy 场景的任务量、采用率、节省时间、质量提升、输入输出 token、RAG / 工具 / 人审 / 固定成本，输出单次可变成本、月收益、月成本、净收益、收益成本比、净 ROI、回本周期、盈亏平衡任务量、敏感性分析和 ROI 门禁。

```
def safe_div(num, den, default=None):
    if den == 0:
        return default
    return num / den

def round_or_none(value, digits=3):
    return None if value is None else round(value, digits)
```

```
SCENARIOS = [
    {
        "name": "support_rag",
        "monthly_tasks": 4200,
        "adoption_rate": 0.62,
        "quality_uplift": 0.24,
        "time_saved_minutes": 4.5,
        "hourly_cost": 38,
```

```

    "value_per_quality_success": 5.0,
    "incremental_revenue_per_task": 0.8,
    "risk_reduction_per_task": 0.4,
    "model_calls": 1.2,
    "input_tokens": 1800,
    "output_tokens": 360,
    "cache_discount_rate": 0.25,
    "input_price_per_1k": 0.002,
    "output_price_per_1k": 0.008,
    "rag_cost_per_task": 0.035,
    "tool_cost_per_task": 0.0,
    "review_minutes": 0.4,
    "review_hourly_cost": 24,
    "retry_cost_per_task": 0.012,
    "risk_cost_per_task": 0.02,
    "fixed_monthly_cost": 4200,
    "upfront_cost": 9000,
    "latency_ok": True,
    "quality_gate": True,
},
{
    "name": "contract_review",
    "monthly_tasks": 90,
    "adoption_rate": 0.55,
    "quality_uplift": 0.18,
    "time_saved_minutes": 55,
    "hourly_cost": 130,
    "value_per_quality_success": 280,
    "incremental_revenue_per_task": 0.0,
    "risk_reduction_per_task": 18.0,
    "model_calls": 2.5,
    "input_tokens": 6200,
    "output_tokens": 900,
    "cache_discount_rate": 0.15,
    "input_price_per_1k": 0.004,
    "output_price_per_1k": 0.012,
    "rag_cost_per_task": 0.18,
    "tool_cost_per_task": 0.08,
    "review_minutes": 8.0,
    "review_hourly_cost": 95,
    "retry_cost_per_task": 0.10,
    "risk_cost_per_task": 0.35,
    "fixed_monthly_cost": 8200,
    "upfront_cost": 32000,
    "latency_ok": True,
    "quality_gate": True,
},
{
    "name": "code_agent",
    "monthly_tasks": 650,
    "adoption_rate": 0.38,
    "quality_uplift": 0.20,
    "time_saved_minutes": 22,
    "hourly_cost": 90,
    "value_per_quality_success": 40,

```

```

    "incremental_revenue_per_task": 0.0,
    "risk_reduction_per_task": 0.0,
    "model_calls": 5.0,
    "input_tokens": 4800,
    "output_tokens": 1200,
    "cache_discount_rate": 0.10,
    "input_price_per_1k": 0.004,
    "output_price_per_1k": 0.012,
    "rag_cost_per_task": 0.12,
    "tool_cost_per_task": 0.25,
    "review_minutes": 5.0,
    "review_hourly_cost": 80,
    "retry_cost_per_task": 0.35,
    "risk_cost_per_task": 0.25,
    "fixed_monthly_cost": 12000,
    "upfront_cost": 45000,
    "latency_ok": False,
    "quality_gate": False,
},
{
    "name": "generic_chatbot",
    "monthly_tasks": 300,
    "adoption_rate": 0.25,
    "quality_uplift": 0.04,
    "time_saved_minutes": 1.5,
    "hourly_cost": 35,
    "value_per_quality_success": 2.0,
    "incremental_revenue_per_task": 0.0,
    "risk_reduction_per_task": 0.0,
    "model_calls": 1.0,
    "input_tokens": 900,
    "output_tokens": 260,
    "cache_discount_rate": 0.05,
    "input_price_per_1k": 0.002,
    "output_price_per_1k": 0.008,
    "rag_cost_per_task": 0.0,
    "tool_cost_per_task": 0.0,
    "review_minutes": 0.2,
    "review_hourly_cost": 24,
    "retry_cost_per_task": 0.01,
    "risk_cost_per_task": 0.03,
    "fixed_monthly_cost": 2500,
    "upfront_cost": 6000,
    "latency_ok": True,
    "quality_gate": False,
},
]

```

```

def model_cost(s):
    discounted_input = s["input_tokens"] * (1.0 - s["cache_discount_rate"])
    input_cost = discounted_input * s["input_price_per_1k"] / 1000.0
    output_cost = s["output_tokens"] * s["output_price_per_1k"] / 1000.0
    return s["model_calls"] * (input_cost + output_cost)

```

```

def variable_cost(s):
    review_cost = s["review_minutes"] / 60.0 * s["review_hourly_cost"]
    return (
        model_cost(s)
        + s["rag_cost_per_task"]
        + s["tool_cost_per_task"]
        + review_cost
        + s["retry_cost_per_task"]
        + s["risk_cost_per_task"]
    )

def benefit_per_task(s, adoption=None, quality=None):
    adoption = s["adoption_rate"] if adoption is None else adoption
    quality = s["quality_uplift"] if quality is None else quality
    time_value = s["time_saved_minutes"] / 60.0 * s["hourly_cost"]
    quality_value = quality * s["value_per_quality_success"]
    return adoption * (time_value + quality_value + s["incremental_revenue_per_task"]) + s["risk_reduction_per_task"]

def audit(s):
    var_cost = variable_cost(s)
    benefit_task = benefit_per_task(s)
    adopted_tasks = s["monthly_tasks"] * s["adoption_rate"]
    monthly_benefit = s["monthly_tasks"] * benefit_task
    monthly_cost = adopted_tasks * var_cost + s["fixed_monthly_cost"]
    net_benefit = monthly_benefit - monthly_cost
    bcr = safe_div(monthly_benefit, monthly_cost)
    roi = safe_div(net_benefit, monthly_cost)
    payback = safe_div(s["upfront_cost"], net_benefit) if net_benefit > 0 else None
    unit_margin = benefit_task - s["adoption_rate"] * var_cost
    break_even_tasks = safe_div(s["fixed_monthly_cost"], unit_margin) if unit_margin > 0 else None

    sensitivity = {}
    for delta in (-0.15, 0.0, 0.15):
        adjusted_adoption = max(0.0, min(1.0, s["adoption_rate"] + delta))
        adjusted_benefit = s["monthly_tasks"] * benefit_per_task(s, adoption=adjusted_adoption)
        adjusted_cost = s["monthly_tasks"] * adjusted_adoption * var_cost + s["fixed_monthly_cost"]
        sensitivity[f"adoption_{delta:+.2f}"] = round(adjusted_benefit - adjusted_cost, 2)
    for multiplier in (0.8, 1.0, 1.2):
        adjusted_cost = adopted_tasks * var_cost * multiplier + s["fixed_monthly_cost"]
        sensitivity[f"variable_cost_x{multiplier:.1f}"] = round(monthly_benefit - adjusted_cost, 2)

    failed = []
    if net_benefit <= 0:
        failed.append("net_benefit")
    if bcr is None or bcr < 1.25:
        failed.append("benefit_cost_ratio")
    if payback is None or payback > 6:
        failed.append("payback")
    if unit_margin <= 0:
        failed.append("unit_margin")
    if not s["latency_ok"]:
        failed.append("latency")

```

```

if not s["quality_gate"]:
    failed.append("quality")

return {
    "model_cost_per_task": round(model_cost(s), 4),
    "variable_cost_per_task": round(var_cost, 4),
    "benefit_per_task": round(benefit_task, 3),
    "unit_margin": round(unit_margin, 3),
    "monthly_benefit": round(monthly_benefit, 2),
    "monthly_cost": round(monthly_cost, 2),
    "net_benefit": round(net_benefit, 2),
    "benefit_cost_ratio": round_or_none(bcr),
    "roi": round_or_none(roi),
    "payback_months": round_or_none(payback),
    "break_even_tasks": round_or_none(break_even_tasks, 1),
    "sensitivity": sensitivity,
    "roi_gate": not failed,
    "failed_gates": failed,
}

```

```

audits = {scenario["name"]: audit(scenario) for scenario in SCENARIOS}
ranked = sorted(
    ((name, result["net_benefit"], result["roi_gate"]) for name, result in audits.items()),
    key=lambda item: item[1],
    reverse=True,
)

print("ranked=", ranked)
print("roi_pass=", [name for name, result in audits.items() if result["roi_gate"]])
print("needs_rework=", {name: result["failed_gates"] for name, result in audits.items() if not result["roi_gate"]})
for name in [item[0] for item in ranked]:
    print(name, audits[name])

```

这段 demo 的关键结论是：support_rag 在 toy 假设下通过 ROI 门禁；contract_review 单次价值高，但固定成本和人审成本太重，回本周期不过线；code_agent 质量和延迟没过线，不能只看效率收益；generic_chatbot 任务价值太低，规模也不够，单位经济账和净收益都不成立。

4.22 本章小结

大模型项目要可持续，必须算成本收益。成本不仅是 token，还包括 GPU、RAG、Agent 工具、人审、数据、工程、运维、安全和失败成本。收益也不只是一句“提升效率”，而要尽量连接到人工节省、质量提升、收入增长或风险降低。

下一章会进入企业级 LLM 应用，讨论大模型在企业场景中如何接入知识库、权限系统、业务流程、审计合规和组织协作。

第五章：企业级 LLM 应用

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对五类口径：OpenAI 企业数据隐私和安全资料提醒我们，企业级应用必须关注数据所有权、默认训练使用边界、加密、数据保留、SSO、RBAC、审计日志和用量治理；NIST AI RMF / Generative AI Profile 强调生成式 AI 风险要进入 govern、map、measure、manage 的风险管理闭环；OWASP LLM Top 10 资料提醒我们，企业 LLM 应用要特别警惕提示注入、敏感信息泄露、向量与嵌入弱点、过度代理和无界资源消耗等应用层风险；Microsoft RBAC / Zero Trust 资料用于核

对 IAM、最小权限、角色授权和身份控制口径；Google SRE 的 SLI / SLO / error budget 资料用于核对企业服务可靠性和延迟门禁。

本章不替代企业安全架构、法务合规审查、采购合同、真实 IAM 设计或行业监管要求；后续第九章会专门展开隐私合规与治理，第六章会专门展开 RAG 产品落地。本章的重点是让算法工程师在面试和项目复盘中能讲清：企业级应用为什么不能只做聊天入口，为什么权限、租户隔离、数据治理、审计、SLO、人审和业务指标要一起进入上线门禁。

企业级 LLM 应用和个人消费级应用有很大区别。个人用户更关注好不好用、有不有趣；企业更关注能否接入现有系统、是否符合权限和合规要求、是否能节省成本、是否能稳定服务多人协作和复杂流程。企业级应用不是简单加一个聊天框，而是把大模型嵌入真实业务流程。

本章系统讲企业级 LLM 应用：客服、知识库、代码助手、数据分析、办公自动化、行业助手等典型场景，以及企业落地中的权限、系统集成、数据治理、工作流、审计、运维、ROI 和评估。

5.1 企业级应用的特点

企业级 LLM 应用通常有这些特点：

1. 数据来自企业内部。
2. 权限和合规要求高。
3. 需要接入现有系统。
4. 用户角色复杂。
5. 任务结果影响业务流程。
6. 要有审计和监控。
7. 需要可衡量 ROI。
8. 不能只靠 demo 效果。

面试回答：

企业级 LLM 应用的核心不是把模型接进来，而是把模型嵌入企业流程。它需要解决内部数据接入、权限控制、系统集成、稳定性、审

5.2 企业知识库问答

企业知识库是最常见的 LLM 应用。

目标：让员工快速找到内部制度、产品文档、技术文档、流程说明和历史经验。

关键能力：

1. 文档解析。
2. 权限过滤。
3. 语义检索。
4. 答案生成。
5. 引用来源。
6. 多轮追问。
7. 反馈纠错。

常见问题：

1. 文档过期。
2. 权限复杂。
3. 召回不准。
4. 引用错误。
5. 用户问法和文档表达不一致。
6. 缺少标准答案评估。

知识库问答的成败，很大程度取决于数据治理和权限设计，而不只是模型能力。

5.3 智能客服

智能客服是高频场景。

价值来源：

1. 降低人工客服压力。
2. 提高自助解决率。
3. 缩短响应时间。
4. 提升回复一致性。
5. 总结用户问题。
6. 辅助人工客服。

客服场景要关注：

1. 意图识别。
2. 知识库检索。
3. 多轮澄清。
4. 转人工策略。
5. 情绪识别。
6. 合规话术。
7. 用户满意度。

客服机器人不能为了减少转人工而强行回答。无法确认时及时转人工，反而更能保护体验。

5.4 客服 Copilot 和全自动客服

客服场景有两种形态。

客服 Copilot：

1. 给人工客服推荐答案。
2. 总结用户历史。
3. 提醒风险话术。
4. 自动生成工单摘要。

全自动客服：

1. 直接面对用户回答。
2. 自动处理简单问题。
3. 必要时转人工。

早期更推荐 Copilot，因为风险低、容易积累数据、人工能纠错。全自动适合边界清楚、知识稳定、风险低的问题。

5.5 代码助手

企业代码助手可以帮助：

1. 代码补全。
2. 单元测试生成。
3. 代码解释。
4. Bug 定位。
5. 代码审查。
6. 文档生成。
7. 迁移和重构辅助。

企业代码助手要关注：

1. 代码隐私。
2. 仓库权限。
3. 许可证风险。
4. 安全漏洞。

5. 与 IDE 和 CI 集成。
6. 是否符合团队规范。

代码助手的价值指标可以是开发效率、review 通过率、测试覆盖率、缺陷减少和新人上手速度。

5.6 数据分析助手

数据分析助手帮助业务人员用自然语言分析数据。

能力包括：

1. 自然语言转 SQL。
2. 指标解释。
3. 图表生成。
4. 异常分析。
5. 报告生成。
6. 数据口径解释。

难点：

1. 指标口径复杂。
2. 数据权限严格。
3. SQL 生成错误可能误导决策。
4. 需要防止查询敏感数据。
5. 需要可追溯计算过程。

数据分析助手不能只生成漂亮图表，还要保证口径、权限和可验证性。

5.7 办公自动化

办公自动化场景包括：

1. 会议纪要。
2. 邮件草稿。
3. 周报生成。
4. 文档润色。
5. PPT 大纲。
6. 表格处理。
7. 日程安排。
8. 工单摘要。

这类场景容错率较高，适合作为企业大模型试点。

但也要注意：

1. 不要泄露会议敏感信息。
2. 邮件发送前要人工确认。
3. 生成内容要符合企业风格。
4. 结果要可编辑。

5.8 行业助手

行业助手针对特定行业。

例如：

1. 金融投研助手。
2. 法律合同助手。
3. 医疗病历助手。
4. 教育备课助手。
5. 制造运维助手。

6. 保险理赔助手。

行业场景价值高，但要求也高：

1. 专业知识准确。
2. 数据合规。
3. 结果可追溯。
4. 风险可控。
5. 需要专家评估。
6. 不能替代最终责任人。

行业助手通常适合先做人机协同，而不是直接全自动决策。

5.9 企业系统集成

企业级 LLM 应用需要接入现有系统。

常见系统：

1. SSO 和身份系统。
2. 权限系统。
3. 文档系统。
4. CRM。
5. ERP。
6. 工单系统。
7. 数据仓库。
8. BI 系统。
9. 代码仓库。
10. 审计系统。

集成难点往往比模型难点更大。没有系统集成，AI 功能很难进入真实 workflow。

企业集成至少要分三层看：

1. 身份层：SSO、MFA、SCIM、用户组、服务账号和离职回收。
2. 数据层：文档系统、数据仓库、代码仓库、知识库、向量库和日志系统。
3. 工作流层：工单、审批、CRM、ERP、BI、CI/CD 和人工复核入口。

面试中不要只说“接入企业系统”。更成熟的说法是：先接身份和权限，再接数据和知识库，最后把模型输出接入已有 workflow；如果没有审计、回滚和人工接管，就不要直接让模型驱动高风险动作。

5.10 权限控制

企业应用必须做权限控制。

需要保证：

1. 用户只能访问有权限的数据。
2. RAG 检索不能越权召回。
3. Agent 工具不能越权调用。
4. 日志不能泄露敏感信息。
5. 不同租户数据隔离。
6. 管理员可审计访问记录。

权限控制不能只在前端做，必须贯穿检索、生成、工具调用和日志。

更具体地说，权限至少要穿过四个环节：

1. 检索权限：向量检索和关键词检索只能召回用户有权看的文档。
2. 生成权限：模型不能把无权信息通过总结、引用或多轮对话泄露出来。
3. 工具权限：Agent 调用工单、数据库、邮件、代码仓库等工具前要做角色和动作门禁。
4. 日志权限：prompt、检索片段、工具返回和模型输出写入日志前要脱敏并限制查看范围。

企业权限的关键不是“模型知道用户是谁”，而是后端系统在每次检索、工具调用、写日志和展示引用时都重新执行权限判断。只靠 prompt 告诉模型“不要泄露信息”，不是可靠权限控制。

5.11 数据治理

企业数据常见问题：

1. 文档过期。
2. 多版本冲突。
3. 命名不统一。
4. 权限不清。
5. 格式复杂。
6. 缺少元数据。
7. 没有标准答案。
8. 敏感信息混杂。

大模型不能自动解决数据治理问题。相反，数据治理差会放大模型幻觉和错误引用。

5.12 审计与合规

企业级应用需要审计。

审计内容：

1. 谁访问了什么数据。
2. 模型回答了什么。
3. 使用了哪些文档。
4. 调用了哪些工具。
5. 是否涉及敏感信息。
6. 是否有人审。
7. 是否发生异常。

合规要求因行业不同而不同。金融、医疗、法律和政企场景通常要求更严格。

5.13 上线策略

企业 LLM 应用不建议一次全量上线。

推荐路径：

1. 内部小范围试点。
2. 人机协同模式。
3. 收集反馈和失败样本。
4. 建立评估集。
5. 灰度扩大范围。
6. 再考虑自动化比例提升。

上线策略要和风险等级匹配。高风险场景必须更慢、更稳。

5.14 组织协作

企业级 LLM 项目需要多方协作。

参与方：

1. 业务团队。
2. 产品团队。
3. 算法团队。
4. 工程团队。
5. 数据团队。

6. 安全团队。
7. 法务合规。
8. 运维团队。

算法团队不能独立完成企业级落地。很多关键问题在数据、流程、权限和组织协作中。

5.15 企业应用评估指标

常见指标：

1. 使用率。
2. 任务完成率。
3. 用户满意度。
4. 人工节省时间。
5. 转人工率。
6. 答案正确率。
7. 引用准确率。
8. 响应延迟。
9. 单次任务成本。
10. 安全事件数。
11. 业务指标改善。

不同应用要选不同主指标。知识库看解决率和引用准确率；客服看自助解决率和满意度；代码助手看采纳率和测试通过率。

5.15.1 关键公式与企业级应用指标速查

可以把一个企业 LLM 应用样本写成：

$e_i = (u_i, d_i, p_i, \tau_i, w_i, a_i, r_i, l_i, m_i)$

其中 u_i 是用户和角色集合， d_i 是接入的数据源， p_i 是权限策略， τ_i 是租户或组织边界， w_i 是接入的业务 workflows， a_i 是审计记录， r_i 是风险等级， l_i 是延迟观测， m_i 是业务指标。

1. 权限通过率

企业应用最核心的门禁之一，是检索、工具和日志都没有越权：

$$R_{\{\mathrm{perm}\}} = \frac{1}{N} \sum_{i=1}^N h_i^{\{\mathrm{rag}\}} h_i^{\{\mathrm{tool}\}} h_i^{\{\mathrm{log}\}}$$

其中 h_i 是 0/1 指标。 $h_i^{\{\mathrm{rag}\}}=1$ 表示 RAG 检索没有越权召回； $h_i^{\{\mathrm{tool}\}}=1$ 表示工具调用通过角色和动作校验； $h_i^{\{\mathrm{log}\}}=1$ 表示日志没有泄露超出查看者权限的数据。直觉上，三层只要有一层失败，这次请求就不能算权限通过。

2. 租户隔离违规率

多租户企业产品要单独监控跨租户数据泄露：

$$R_{\{\mathrm{viol}\}} = \frac{1}{N} \sum_{i=1}^N v_i$$

其中 $v_i=1$ 表示第 i 次请求发生跨租户、跨部门或跨项目边界的违规访问。这个指标通常应接近 0；哪怕平均任务成功率很高，只要出现租户隔离事故，也不能直接上线。

3. 引用支持率

企业知识库和行业助手不能只要求“回答像真的”，还要要求关键结论能被授权证据支持：

$$C_{\{\mathrm{cite}\}} = \frac{1}{N} \sum_{i=1}^N c_i$$

其中 $c_i=1$ 表示第 i 个关键结论的引用存在、相关、足以支持结论、未过期，并且当前用户有权查看。引用支持率低时，产品应该更保守地展示“不确定”“需要人工复核”或触发知识库更新。

4. 审计覆盖率

企业应用需要能回答“谁在什么时候用什么身份访问了什么数据、调用了什么工具、得到什么结果”：

$$C_{\{\mathrm{audit}\}} = \frac{1}{N} \sum_{i=1}^N a_i$$

其中 $a_i=1$ 表示请求 trace 中包含用户身份、数据来源、引用文档、工具调用、权限判断、模型版本、输出摘要和人工复核状态。没有审计覆盖，事故发生后无法复盘，也无法证明系统按权限和流程运行。

5. 数据新鲜度通过率

企业知识库常见问题是文档过期。可以用文档年龄门禁粗略检查：

$$C_{\{\mathrm{fresh}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_i \leq A_{\{\max\}}]$$

其中 g_i 是证据文档距当前时间的年龄， $A_{\{\max\}}$ 是业务允许的最大文档年龄。不同业务阈值不同：报销制度可能按月更新，接口文档可能按版本更新，合规政策可能要求更严格的版本控制。

6. SLO 通过率

企业应用不仅要准，还要在业务流程可接受的时间内返回：

$$R_{\{\mathrm{slo}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[l_i \leq L_{\{\mathrm{slo}\}}]$$

其中 l_i 是请求延迟， $L_{\{\mathrm{slo}\}}$ 是场景定义的延迟目标。客服助手、代码补全、会议纪要、合同复核的 SLO 不一样，不能共用一个平均响应时间。

7. 高风险人审覆盖率

对金融、医疗、法律、合规、退款、权限变更等高风险任务，要看高风险样本是否有人审：

$$C_{\{\mathrm{human}\}} = \frac{\sum_{i=1}^N z_i q_i}{\sum_{i=1}^N z_i + \epsilon}$$

其中 $z_i=1$ 表示高风险任务， $q_i=1$ 表示有人审、审批或二次确认， ϵ 用于避免分母为 0。高风险任务不是不能用 LLM，而是不能跳过责任人、审批和审计。

8. 企业就绪分与上线门禁

一个简化企业就绪分可以写成：

$$S_{\{\mathrm{ent}\}} = 0.25R_{\{\mathrm{perm}\}} + 0.15(1 - R_{\{\mathrm{viol}\}}) + 0.15C_{\{\mathrm{cite}\}} + 0.15C_{\{\mathrm{audit}\}} + 0.10C_{\{\mathrm{fresh}\}}$$

这个分数只用于面试和 toy demo 解释，真实项目要按行业 and 风险重新定权重。上线门禁可以写成：

$$G_{\{\mathrm{ent}\}} = \mathbf{1}[R_{\{\mathrm{perm}\}} \geq 0.95] \mathbf{1}[R_{\{\mathrm{viol}\}} = 0] \mathbf{1}[C_{\{\mathrm{audit}\}} \geq 0.9]$$

其中 $M_{\{\mathrm{biz}\}}=1$ 表示已定义业务主指标。直觉是：企业应用不是靠一个综合分上线，而是任何关键门禁不过线都要先修复。

5.16 常见失败模式

1. 只做聊天框，没有接入工作流。
2. 知识库数据质量差。
3. 权限控制不完整。
4. 缺少业务指标。
5. 用户不知道怎么用。
6. 模型回答无法追溯。
7. 成本没有监控。
8. 高风险场景没有人审。
9. 试点成功但无法规模化。
10. 组织协作不顺。

企业级 LLM 应用失败往往不是因为模型完全不行，而是因为没有解决企业环境中的系统问题。

5.17 面试题：企业级 LLM 应用和普通应用有什么区别

回答要点：

企业级 LLM 应用更强调数据、权限、系统集成、稳定性、审计和 ROI。普通应用可能只关注用户体验和模型效果，但企业应用需要接

5.18 面试题：如何落地企业知识库问答

回答要点：

我会先做数据治理，整理文档来源、版本、权限和元数据，再构建 RAG 流程，包括文档解析、分块、embedding、检索、rerank、答案

5.19 最小可运行企业级应用审计 demo

下面这个 demo 用 0 依赖 Python 模拟企业级 LLM 应用上线审计。它不是生产治理系统，而是帮助你在面试中把“企业级”拆成可检查字段：身份、权限、租户隔离、RAG 权限过滤、工具权限、审计日志、PII 脱敏、数据新鲜度、引用支持、SSO、工作流、评估、反馈、SLO、业务指标和高风险人审。

```
def mean(values):
    return sum(values) / len(values) if values else 0.0

def audit_app(app):
    high_risk = app["risk_level"] == "high"
    gates = {
        "permission_coverage": app["permission_coverage"] >= 0.90,
        "tenant_isolation": app["tenant_isolation"] >= 1.00,
        "rag_permission_filter": app["rag_permission_filter"] >= 0.95,
        "tool_permission_gate": app["tool_permission_gate"] >= 0.90,
        "audit_log_coverage": app["audit_log_coverage"] >= 0.90,
        "pii_redaction": app["pii_redaction"] >= 0.85,
        "data_freshness": app["data_freshness"] >= 0.75,
        "citation_support": app["citation_support"] >= 0.80,
        "sso_integration": app["sso_integration"] >= 1.00,
        "workflow_integration": app["workflow_integration"] >= 0.75,
        "eval_ready": app["eval_ready"] >= 0.80,
        "feedback_loop": app["feedback_loop"] >= 0.75,
        "sla_p95_latency_ok": app["sla_p95_latency_ok"] >= 1.00,
        "business_metric_defined": app["business_metric_defined"] >= 1.00,
        "human_review_coverage": (not high_risk) or app["human_review_coverage"] >= 0.80,
    }

    permission_score = mean([
        app["permission_coverage"],
        app["rag_permission_filter"],
        app["tool_permission_gate"],
    ])
    governance_score = mean([
        app["tenant_isolation"],
        app["audit_log_coverage"],
        app["pii_redaction"],
        app["data_freshness"],
    ])
    evidence_score = mean([
        app["citation_support"],
        app["eval_ready"],
        app["feedback_loop"],
        app["business_metric_defined"],
    ])
    integration_score = mean([
        app["sso_integration"],
        app["workflow_integration"],
    ])
```

```

ops_score = mean([
    app["sla_p95_latency_ok"],
    app["human_review_coverage"] if high_risk else 1.0,
])

enterprise_score = (
    0.30 * permission_score
    + 0.22 * governance_score
    + 0.18 * evidence_score
    + 0.15 * integration_score
    + 0.15 * ops_score
)
failed_gates = [name for name, ok in gates.items() if not ok]
return {
    "name": app["name"],
    "scenario_type": app["scenario_type"],
    "enterprise_score": round(enterprise_score, 3),
    "enterprise_gate": not failed_gates,
    "failed_gates": failed_gates,
}

apps = [
    {
        "name": "support_kb_rag",
        "scenario_type": "enterprise_knowledge_base",
        "users": 1800,
        "data_sources": 7,
        "permission_coverage": 0.96,
        "tenant_isolation": 1.00,
        "rag_permission_filter": 0.98,
        "tool_permission_gate": 0.94,
        "audit_log_coverage": 0.93,
        "pii_redaction": 0.91,
        "data_freshness": 0.88,
        "citation_support": 0.90,
        "sso_integration": 1.00,
        "workflow_integration": 0.86,
        "eval_ready": 0.84,
        "feedback_loop": 0.78,
        "sla_p95_latency_ok": 1.00,
        "business_metric_defined": 1.00,
        "human_review_coverage": 0.82,
        "risk_level": "high",
    },
    {
        "name": "contract_copilot",
        "scenario_type": "legal_assistant",
        "users": 120,
        "data_sources": 5,
        "permission_coverage": 0.91,
        "tenant_isolation": 1.00,
        "rag_permission_filter": 0.94,
        "tool_permission_gate": 0.88,
        "audit_log_coverage": 0.90,
    },

```

```

    "pii_redaction": 0.86,
    "data_freshness": 0.70,
    "citation_support": 0.82,
    "sso_integration": 1.00,
    "workflow_integration": 0.72,
    "eval_ready": 0.80,
    "feedback_loop": 0.62,
    "sla_p95_latency_ok": 1.00,
    "business_metric_defined": 1.00,
    "human_review_coverage": 0.71,
    "risk_level": "high",
  },
  {
    "name": "data_analyst_nl2sql",
    "scenario_type": "data_analysis",
    "users": 260,
    "data_sources": 9,
    "permission_coverage": 0.86,
    "tenant_isolation": 1.00,
    "rag_permission_filter": 0.90,
    "tool_permission_gate": 0.73,
    "audit_log_coverage": 0.82,
    "pii_redaction": 0.79,
    "data_freshness": 0.93,
    "citation_support": 0.55,
    "sso_integration": 1.00,
    "workflow_integration": 0.84,
    "eval_ready": 0.72,
    "feedback_loop": 0.64,
    "sla_p95_latency_ok": 0.00,
    "business_metric_defined": 1.00,
    "human_review_coverage": 0.76,
    "risk_level": "high",
  },
  {
    "name": "office_summarizer",
    "scenario_type": "office_automation",
    "users": 2300,
    "data_sources": 4,
    "permission_coverage": 0.78,
    "tenant_isolation": 0.95,
    "rag_permission_filter": 0.80,
    "tool_permission_gate": 0.65,
    "audit_log_coverage": 0.70,
    "pii_redaction": 0.68,
    "data_freshness": 0.82,
    "citation_support": 0.45,
    "sso_integration": 0.70,
    "workflow_integration": 0.76,
    "eval_ready": 0.62,
    "feedback_loop": 0.50,
    "sla_p95_latency_ok": 1.00,
    "business_metric_defined": 0.60,
    "human_review_coverage": 0.70,
    "risk_level": "medium",
  }

```



```

    },
    {
        "name": "generic_chat_portal",
        "scenario_type": "generic_chatbot",
        "users": 900,
        "data_sources": 2,
        "permission_coverage": 0.52,
        "tenant_isolation": 0.70,
        "rag_permission_filter": 0.40,
        "tool_permission_gate": 0.30,
        "audit_log_coverage": 0.35,
        "pii_redaction": 0.42,
        "data_freshness": 0.50,
        "citation_support": 0.20,
        "sso_integration": 0.00,
        "workflow_integration": 0.20,
        "eval_ready": 0.30,
        "feedback_loop": 0.25,
        "sla_p95_latency_ok": 0.00,
        "business_metric_defined": 0.00,
        "human_review_coverage": 0.20,
        "risk_level": "medium",
    },
]

results = [audit_app(app) for app in apps]
ranked = sorted(
    [(r["name"], r["enterprise_score"], r["enterprise_gate"]) for r in results],
    key=lambda item: item[1],
    reverse=True,
)
passed = [r["name"] for r in results if r["enterprise_gate"]]
needs_rework = {
    r["name"]: r["failed_gates"]
    for r in results
    if not r["enterprise_gate"]
}

print("ranked=", ranked)
print("enterprise_pass=", passed)
print("needs_rework=", needs_rework)

```

一组典型输出是：

```

ranked= [('support_kb_rag', 0.927, True), ('contract_copilot', 0.866, False), ('data_analyst_nl2sql', 0.77, False), ('of
enterprise_pass= ['support_kb_rag']
needs_rework= {'contract_copilot': ['rag_permission_filter', 'tool_permission_gate', 'data_freshness', 'workflow_integ

```

这个 demo 的重点不是分数本身，而是审计口径：企业级应用必须把权限、租户隔离、审计、PII、引用、SLO、业务指标和人审放进同一张表。contract_copilot 分数不低，但因为法律场景高风险、人审和反馈闭环不足，不能直接全自动上线；data_analyst_nl2sql 工作流接入不错，但 SQL 工具权限、引用支持和延迟门禁不过线；generic_chat_portal 则说明“通用聊天入口”如果没有身份、权限、指标和工作流，只能算内部 demo。

5.20 本章小结

企业级 LLM 应用的难点在于真实环境：数据复杂、权限严格、系统多、流程长、用户角色多、合规要求高。客服、知识库、代码助手、数据分析、办公自动化和行业助手都是常见场景，但每个场景都需要结合业务流程和风

险等级设计。

下一章会进入 RAG 产品落地，深入讨论企业知识库和检索增强生成系统如何从技术方案变成可用产品。

第六章：RAG 产品落地

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对四类口径：OpenAI retrieval / file search / evals 相关资料提醒我们，RAG 产品不是单一向量检索，而是查询改写、向量库、元数据过滤、重排、上下文构造、引用和评估组成的系统；RAGAS 等 RAG 评估论文提醒我们，RAG 要同时看 context precision、context recall、faithfulness / groundedness、answer relevance 和人工标注边界；OWASP LLM Top 10 中与向量和嵌入、提示注入、敏感信息泄露相关的风险提醒我们，企业 RAG 必须把不可信文档、权限过滤和敏感信息控制纳入产品门禁；Google SRE 的 SLI / SLO / error budget 口径提醒我们，RAG 的检索、重排和生成链路必须有延迟、错误率和成本目标。

本章不替代后续专门的 RAG 算法实现、向量数据库选型、安全合规审查或第十七册 Agentic RAG 深入章节。这里聚焦产品落地：怎么把企业知识库从“几个 PDF 的 demo”升级成可评估、可引用、可治理、可运营、可控成本的 RAG 产品。

RAG 是企业大模型落地中最常见的技术路线之一。它看起来简单：把文档切块、做向量检索、把检索结果喂给模型生成答案。但真正产品化时，难点往往不在“能不能检索”，而在知识是否最新、引用是否可信、权限是否正确、答案是否可评估、用户是否愿意使用，以及失败后如何持续改进。

本章系统讲 RAG 产品落地：企业文档治理、知识更新、文档解析、分块、检索、引用可信、权限控制、无法回答、反馈闭环、效果评估、上线运营和常见失败模式。

6.1 RAG 为什么适合企业落地

企业有大量内部知识：制度、流程、产品说明、技术文档、FAQ、合同模板、工单记录、会议纪要、历史案例。这些知识经常分散在不同系统中，员工查找成本高。

RAG 的价值是：

1. 让模型基于企业知识回答。
2. 降低幻觉风险。
3. 给出引用来源。
4. 支持知识更新。
5. 避免频繁微调模型。
6. 接入权限控制。

面试回答：

RAG 适合企业落地，因为企业知识经常分散、更新频繁且需要权限控制。RAG 可以把检索到的内部文档作为上下文，让模型基于证据回

6.2 RAG Demo 和 RAG 产品的区别

RAG demo：

1. 几个 PDF。
2. 简单切块。
3. 向量检索 top-k。
4. 生成答案。

RAG 产品：

1. 多来源文档接入。
2. 文档版本管理。
3. 权限过滤。
4. 增量更新。

5. 引用和可追溯。
6. 评估集。
7. 用户反馈。
8. 监控和运营。

demo 证明技术能跑通，产品要面对真实企业知识的混乱和变化。

6.3 企业文档治理

RAG 的上限很大程度由文档质量决定。

常见文档问题：

1. 过期文档未删除。
2. 多个版本互相冲突。
3. 标题不清楚。
4. 文档没有负责人。
5. 权限不明确。
6. 格式复杂。
7. 图片和表格难解析。
8. 内容重复。

文档治理要解决：

1. 来源可信。
2. 版本清晰。
3. 负责人明确。
4. 更新时间可见。
5. 权限可继承。
6. 元数据完整。

RAG 不是数据治理的替代品。脏知识库会让模型更自信地输出错误答案。

6.4 文档解析

文档解析决定进入 RAG 系统的原始文本质量。

要处理：

1. PDF。
2. Word。
3. PPT。
4. Excel。
5. HTML。
6. Markdown。
7. 图片和扫描件。
8. 表格和图表。

解析难点：

1. 页眉页脚噪声。
2. 表格结构丢失。
3. 多栏排版错乱。
4. OCR 错字。
5. 图片中的关键信息。
6. 章节层级丢失。

解析质量差，后面 embedding 和检索再强也难补救。

6.5 分块策略

分块影响召回和答案质量。

分块太小：上下文不完整。分块太大：召回不精确，token 成本高。

常见策略：

1. 按段落分块。
2. 按标题层级分块。
3. 按语义边界分块。
4. 滑动窗口重叠。
5. 表格单独处理。
6. 保留标题和路径元数据。

好的 chunk 不只是文本片段，还应该带有文档标题、章节、更新时间、权限、来源链接等元数据。

6.6 检索策略

企业 RAG 通常不只用向量检索。

常见组合：

1. 向量检索。
2. 关键词检索。
3. 混合检索。
4. 元数据过滤。
5. reranking。
6. 查询重写。
7. 多轮检索。

向量检索适合语义相似，关键词检索适合精确术语、产品型号、错误码和人名。企业系统通常需要混合检索。

6.7 权限控制

企业 RAG 必须做权限控制。

关键原则：

1. 用户只能检索有权限的文档。
2. 检索前或检索后必须过滤权限。
3. 引用不能暴露无权限来源。
4. 摘要不能泄露无权限内容。
5. 日志不能保存敏感内容。
6. 权限变更要及时生效。

一个严重错误是：模型不直接展示原文，但把无权限文档内容总结出来。这仍然是数据泄露。

产品上更稳的做法是把权限当成检索条件，而不是生成后的展示条件。换句话说，无权文档不应该进入候选集、rerank 列表、prompt 上下文、引用列表和原始日志。权限变更后，索引、缓存和向量库 metadata 也要同步失效，否则用户离职、转岗或项目权限收回后仍可能通过旧缓存看到答案。

6.8 知识更新

企业知识经常变化。

需要处理：

1. 新文档加入。
2. 老文档删除。
3. 文档内容修改。
4. 权限变化。

5. 文档过期。
6. 索引重建。
7. 增量 embedding。

产品上要显示答案依据的时间和版本。对于时效性强的问题，如果文档过期，系统应提示不确定，而不是强行回答。

6.9 引用可信

引用是 RAG 产品的信任基础。

好的引用应该：

1. 支持对应结论。
2. 指向具体段落。
3. 用户有权限查看。
4. 文档版本正确。
5. 来源可信。
6. 能打开原文。

坏引用会严重破坏信任。最常见问题是答案正确但引用错，或引用文档根本不支持结论。

引用可信要按 claim 检查，而不是按整段回答检查。一个回答可能有 5 个关键断言，其中 4 个有证据、1 个是模型补出来的；如果只看“回答整体还行”，这个 unsupported claim 就会漏掉。企业 RAG 最好把答案拆成 atomic claims，再检查每个 claim 是否被引用段落直接支持、是否使用最新版本、是否有权限展示。

6.10 无法回答

RAG 产品必须能说“不知道”。

无法回答的情况：

1. 检索不到相关文档。
2. 文档互相矛盾。
3. 用户无权限查看答案来源。
4. 问题超出知识库范围。
5. 证据不足。
6. 文档过期。

好的无法回答体验：

1. 说明原因。
2. 给出已检索范围。
3. 建议补充信息。
4. 引导用户联系负责人。
5. 允许反馈缺失文档。

强行回答会短期显得智能，长期损害信任。

6.11 用户反馈闭环

RAG 产品需要反馈闭环。

反馈可以包括：

1. 答案有用或无用。
2. 引用是否正确。
3. 是否解决问题。
4. 用户期望答案。
5. 缺失文档。
6. 错误文档。

7. 转人工原因。

反馈要进入改进流程：更新文档、调整分块、优化检索、补充评估集、改进 prompt 或权限规则。

6.12 评估指标

RAG 产品评估包括：

1. 检索召回率。
2. rerank 命中率。
3. 答案正确率。
4. 引用准确率。
5. 证据支持率。
6. 无法回答准确率。
7. 用户满意度。
8. 自助解决率。
9. 延迟。
10. 单次成本。

不同阶段关注不同指标。早期先看检索和答案正确，产品化后还要看使用率、满意度和业务收益。

6.12.1 关键公式与 RAG 产品指标速查

可以把一次 RAG 查询样本写成：

$$q_i=(x_i,u_i,D_i,K_i,E_i,A_i,C_i,P_i,L_i,B_i)$$

其中 x_i 是用户问题， u_i 是用户和角色， D_i 是可访问文档集合， K_i 是检索候选， E_i 是最终进入上下文的证据， A_i 是模型答案， C_i 是引用集合， P_i 是权限判断， L_i 是延迟， B_i 是业务结果。

1. 检索召回率

检索层首先要问：应该找到的证据，有多少被召回到了候选集中？

$$R_{\{\mathrm{ret}\}}=\frac{1}{N}\sum_{i=1}^N\frac{|K_i\cap G_i|}{|G_i|+\epsilon}$$

其中 G_i 是人工标注或专家确认的支持证据集合， ϵ 避免分母为 0。召回率低时，后面的 rerank 和生成通常救不回来。

2. 上下文精确率

召回很多不等于最后上下文质量高。进入 prompt 的证据要尽量相关：

$$P_{\{\mathrm{ctx}\}}=\frac{1}{N}\sum_{i=1}^N\frac{|E_i\cap G_i|}{|E_i|+\epsilon}$$

上下文精确率低，说明 prompt 里混入大量弱相关 chunk，会增加 token 成本、干扰生成，并提高引用错误概率。

3. 证据支持率

生成层要检查答案中的关键断言是否被证据支持：

$$S_{\{\mathrm{ev}\}}=\frac{\sum_{i=1}^N n_i^{\{\mathrm{support}\}}}{\sum_{i=1}^N n_i^{\{\mathrm{claim}\}}+\epsilon}$$

其中 $n_i^{\{\mathrm{claim}\}}$ 是第 i 个回答的关键断言数， $n_i^{\{\mathrm{support}\}}$ 是被上下文证据直接支持的断言数。这个指标比“有引用”更严格。

4. 引用准确率

引用准确率检查引用是否真的支持对应 claim：

$$A_{\{\mathrm{cite}\}}=\frac{\sum_{i=1}^N n_i^{\{\mathrm{cite_ok}\}}}{\sum_{i=1}^N n_i^{\{\mathrm{cite}\}}+\epsilon}$$

其中 $n_i^{\{\mathrm{cite}\}}$ 是引用数量， $n_i^{\{\mathrm{cite_ok}\}}$ 是存在、相关、版本正确、权限正确且足以支持 claim 的引用数量。答案正确但引用错误，在企业产品里仍然是严重缺陷。

5. 拒答准确率

RAG 产品必须知道什么时候不能回答：

$$A_{\{\mathrm{abs}\}}=\frac{1}{N_{\{\mathrm{abs}\}}}\sum_{i=1}^N\mathbf{1}[z_i=1]\mathbf{1}[\hat{z}_i=1]$$

其中 $z_i=1$ 表示样本应该拒答或澄清， $\hat{z}_i=1$ 表示系统实际拒答或澄清。拒答准确率低，说明系统在证据不足、权限不足、文档过期或冲突时仍然强行回答。

6. 权限过滤通过率

企业 RAG 要检查无权文档是否进入候选、上下文、引用或日志：

$$R_{\{\mathrm{perm}\}}=1-\frac{N_{\{\mathrm{unauth}\}}}{N_{\{\mathrm{req}\}}+\epsilon}$$

其中 $N_{\{\mathrm{unauth}\}}$ 是越权召回、越权引用、越权日志或越权摘要事件数。这个指标通常应接近 1，且严重事故不能被平均分掩盖。

7. 过期证据率

知识更新后，旧证据仍被引用会损害信任：

$$R_{\{\mathrm{stale}\}}=\frac{N_{\{\mathrm{stale}\}}}{N_{\{\mathrm{evidence}\}}+\epsilon}$$

其中 $N_{\{\mathrm{stale}\}}$ 是过期、被替换或版本不一致的证据数。时效性强的业务要把过期证据率纳入上线门禁。

8. RAG 产品上线门禁

一个简化门禁可以写成：

$$G_{\{\mathrm{rag}\}}=\mathbf{1}[R_{\{\mathrm{ret}\}}\geq r_0]\mathbf{1}[P_{\{\mathrm{ctx}\}}\geq p_0]\mathbf{1}[S_{\{\mathrm{ev}\}}\geq s_0]$$

其中 r_0, p_0, s_0, c_0, h_0 是场景阈值， L_{95} 是 P95 延迟。真实上线还要加入成本、反馈闭环、业务指标和人工抽检。RAG 产品不能只用一个问答准确率做上线判断。

6.13 评估集构建

RAG 评估需要问题集。

问题来源：

1. 历史搜索日志。
2. 客服工单。
3. 员工常见问题。
4. 业务专家整理。
5. 失败案例。
6. 新文档发布后的测试问题。

每个问题最好标注：

1. 标准答案。
2. 支持文档。
3. 关键段落。
4. 用户角色和权限。
5. 是否应拒答。

没有评估集，RAG 优化容易凭感觉。

6.14 上线运营

RAG 上线后要运营。

运营事项：

1. 监控热门问题。
2. 发现无答案问题。

3. 清理过期文档。
4. 维护文档负责人。
5. 分析低满意度回答。
6. 更新评估集。
7. 监控成本和延迟。
8. 处理权限异常。

RAG 产品不是一次建设，而是持续运营的知识系统。

6.15 常见失败模式

1. 文档质量差却怪模型。
2. 只用向量检索，忽略关键词和元数据。
3. 权限过滤不完整。
4. 引用不支持答案。
5. 文档更新后索引不同步。
6. 无法回答策略缺失。
7. 没有评估集。
8. 用户反馈没有进入改进流程。
9. 答案看起来流畅但没有证据。
10. 只做技术链路，不做知识运营。

RAG 落地的本质，是技术系统和知识管理系统一起建设。

6.16 面试题：RAG 产品落地最难的是什么

回答要点：

RAG 落地最难的不只是检索算法，而是企业文档治理、权限控制、知识更新、引用可信和效果评估。文档过期、版本冲突、权限不清、

6.17 面试题：如何评估企业 RAG

回答要点：

我会分层评估。检索层看召回率和 rerank 命中率；生成层看答案正确率、引用准确率和证据支持率；产品层看自助解决率、用户满意

6.18 最小可运行 RAG 产品审计 demo

下面这个 demo 用 0 依赖 Python 模拟 RAG 产品上线审计。它把检索召回、上下文精确、证据支持、引用准确、拒答、权限过滤、数据新鲜度、P95 延迟、单位成本、评估集、反馈闭环和业务指标放进同一张表。

```
def ratio(num, den):
    return num / den if den else 0.0

def audit_product(product):
    retrieval_recall = ratio(product["retrieved_relevant"], product["relevant_total"])
    context_precision = ratio(product["context_relevant"], product["context_total"])
    evidence_support = ratio(product["supported_claims"], product["total_claims"])
    citation_accuracy = ratio(product["citation_supported"], product["citation_total"])
    abstention_accuracy = ratio(product["correct_abstentions"], product["expected_abstentions"])
    freshness = 1.0 - product["stale_evidence_rate"]
    permission_ok = (
        product["permission_filter_rate"] >= 0.98
        and product["unauthorized_hits"] == 0
    )
    gates = {
```



```

    "retrieval_recall": retrieval_recall >= 0.80,
    "context_precision": context_precision >= 0.65,
    "evidence_support": evidence_support >= 0.85,
    "citation_accuracy": citation_accuracy >= 0.85,
    "abstention_accuracy": abstention_accuracy >= 0.80,
    "permission_filter": permission_ok,
    "freshness": product["stale_evidence_rate"] <= 0.10,
    "p95_latency": product["p95_latency_ms"] <= product["latency_slo_ms"],
    "unit_cost": product["cost_per_answer"] <= product["cost_slo"],
    "eval_ready": product["eval_ready"] >= 0.80,
    "feedback_loop": product["feedback_loop"] >= 0.75,
    "business_metric": product["business_metric_defined"] >= 1.0,
}
rag_score = (
    0.18 * retrieval_recall
    + 0.14 * context_precision
    + 0.18 * evidence_support
    + 0.14 * citation_accuracy
    + 0.08 * abstention_accuracy
    + 0.10 * product["permission_filter_rate"]
    + 0.06 * freshness
    + 0.06 * (1.0 if gates["p95_latency"] else 0.0)
    + 0.03 * product["eval_ready"]
    + 0.03 * product["feedback_loop"]
)
return {
    "name": product["name"],
    "rag_score": round(rag_score, 3),
    "rag_gate": all(gates.values()),
    "metrics": {
        "retrieval_recall": round(retrieval_recall, 3),
        "context_precision": round(context_precision, 3),
        "evidence_support": round(evidence_support, 3),
        "citation_accuracy": round(citation_accuracy, 3),
        "abstention_accuracy": round(abstention_accuracy, 3),
    },
    "failed_gates": [name for name, ok in gates.items() if not ok],
}

```

```

products = [
    {
        "name": "support_policy_rag",
        "retrieved_relevant": 42,
        "relevant_total": 48,
        "context_relevant": 31,
        "context_total": 42,
        "supported_claims": 66,
        "total_claims": 72,
        "citation_supported": 61,
        "citation_total": 68,
        "correct_abstentions": 9,
        "expected_abstentions": 10,
        "permission_filter_rate": 0.99,
        "unauthorized_hits": 0,
    }
]

```

```

    "stale_evidence_rate": 0.06,
    "p95_latency_ms": 1800,
    "latency_slo_ms": 2200,
    "cost_per_answer": 0.043,
    "cost_slo": 0.08,
    "eval_ready": 0.86,
    "feedback_loop": 0.80,
    "business_metric_defined": 1.0,
  },
  {
    "name": "legal_contract_rag",
    "retrieved_relevant": 34,
    "relevant_total": 45,
    "context_relevant": 28,
    "context_total": 44,
    "supported_claims": 58,
    "total_claims": 70,
    "citation_supported": 49,
    "citation_total": 64,
    "correct_abstentions": 8,
    "expected_abstentions": 12,
    "permission_filter_rate": 0.98,
    "unauthorized_hits": 0,
    "stale_evidence_rate": 0.16,
    "p95_latency_ms": 2600,
    "latency_slo_ms": 2500,
    "cost_per_answer": 0.091,
    "cost_slo": 0.10,
    "eval_ready": 0.82,
    "feedback_loop": 0.62,
    "business_metric_defined": 1.0,
  },
  {
    "name": "codebase_rag",
    "retrieved_relevant": 28,
    "relevant_total": 36,
    "context_relevant": 18,
    "context_total": 35,
    "supported_claims": 42,
    "total_claims": 54,
    "citation_supported": 37,
    "citation_total": 50,
    "correct_abstentions": 6,
    "expected_abstentions": 8,
    "permission_filter_rate": 0.93,
    "unauthorized_hits": 1,
    "stale_evidence_rate": 0.08,
    "p95_latency_ms": 1900,
    "latency_slo_ms": 1800,
    "cost_per_answer": 0.052,
    "cost_slo": 0.08,
    "eval_ready": 0.70,
    "feedback_loop": 0.68,
    "business_metric_defined": 1.0,
  },

```

```

{
    "name": "generic_doc_chat",
    "retrieved_relevant": 18,
    "relevant_total": 44,
    "context_relevant": 14,
    "context_total": 48,
    "supported_claims": 30,
    "total_claims": 68,
    "citation_supported": 19,
    "citation_total": 55,
    "correct_abstentions": 2,
    "expected_abstentions": 11,
    "permission_filter_rate": 0.72,
    "unauthorized_hits": 3,
    "stale_evidence_rate": 0.31,
    "p95_latency_ms": 3300,
    "latency_slo_ms": 2200,
    "cost_per_answer": 0.115,
    "cost_slo": 0.08,
    "eval_ready": 0.35,
    "feedback_loop": 0.20,
    "business_metric_defined": 0.0,
},
]

results = [audit_product(product) for product in products]
ranked = sorted(
    [(r["name"], r["rag_score"], r["rag_gate"]) for r in results],
    key=lambda item: item[1],
    reverse=True,
)
rag_pass = [r["name"] for r in results if r["rag_gate"]]
needs_rework = {
    r["name"]: r["failed_gates"]
    for r in results
    if not r["rag_gate"]
}

print("ranked=", ranked)
print("rag_pass=", rag_pass)
print("sample_metrics=", results[0]["metrics"])
print("needs_rework=", needs_rework)

```

一组典型输出是：

```

ranked= [('support_policy_rag', 0.889, True), ('legal_contract_rag', 0.726, False), ('codebase_rag', 0.705, False), ('ge
rag_pass= ['support_policy_rag']
sample_metrics= {'retrieval_recall': 0.875, 'context_precision': 0.738, 'evidence_support': 0.917, 'citation_accuracy':
needs_rework= {'legal_contract_rag': ['retrieval_recall', 'context_precision', 'evidence_support', 'citation_accuracy']

```

这个 demo 的重点是：RAG 产品不能只看“回答像不像对”。legal_contract_rag 有一定质量，但合同场景中证据过期、引用不足、拒答不足和反馈闭环缺失会影响责任边界；codebase_rag 权限和评估不稳，可能把无权仓库或旧代码片段带进回答；generic_doc_chat 则说明“文档聊天”如果没有评估、权限、引用、拒答和业务指标，很难算企业级 RAG 产品。

6.19 本章小结

RAG 产品落地的关键，是让模型基于正确、最新、有权限、可追溯的知识回答问题。技术链路包括文档解析、分块、embedding、检索、rerank、生成和引用；产品链路还包括文档治理、权限、更新、评估、反馈和运营。

下一章会进入 Agent 产品落地，讨论当产品不只是回答问题，而是要执行任务、调用工具、操作系统时，如何设计体验、权限、成本和安全边界。

第七章：Agent 产品落地

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对五类口径：OpenAI Agents SDK 的 tools、handoffs、guardrails 和 tracing 资料提醒我们，Agent 产品要把工具、轨迹、护栏和交接显式工程化；OpenAI Evals 资料提醒我们，Agent 不能只看最终回答，要用可复现评估和 trace 评估任务成功、工具选择、参数、恢复和安全；OpenAI Model Spec 的指令层级口径提醒我们，工具输出、网页、RAG 文档和用户输入不能越过系统 / 开发者指令；OWASP LLM Top 10 中 excessive agency、prompt injection、sensitive information disclosure 和 unbounded consumption 风险提醒我们，能执行动作的 Agent 必须有权限、预算、确认和审计；Anthropic 关于 workflows 与 agents 的公开工程经验提醒我们，生产落地不应迷信“越自主越好”，稳定 workflow 和受控 Agent 往往需要组合。

本章不替代第十七册 Agent 原理、工具协议、安全评估和多 Agent 章节，也不展开具体框架 API。这里聚焦产品落地：怎么把 Agent 从“会调用工具的 demo”升级为可控、可验证、可审计、可预算、可人审、可进入企业工作流的任务执行产品。

Agent 产品和普通大模型问答产品最大的区别，是 Agent 会执行动作。它可能查询系统、修改文件、提交表单、调用 API、操作浏览器、运行代码或触发业务流程。因此 Agent 产品化的核心不是让模型“更自主”，而是让它在受控范围内可靠执行任务。

本章系统讲 Agent 产品落地：适合 Agent 的任务、工具权限、人工确认、失败恢复、审计日志、成本控制、用户体验、评估指标和常见失败模式。

7.1 Agent 产品的本质

Agent 产品不是聊天框加几个工具，而是任务执行系统。

它通常包含：

1. 用户目标。
2. 任务分解。
3. 工具选择。
4. 执行动作。
5. 观察反馈。
6. 状态更新。
7. 错误恢复。
8. 最终交付。

面试回答：

Agent 产品的本质是让模型在受控环境中完成多步任务，而不是只生成回答。落地时要重点设计任务边界、工具权限、状态管理、失败

7.2 哪些任务适合 Agent

适合 Agent 的任务通常满足：

1. 需要多步操作。
2. 需要调用工具。
3. 中间结果会影响下一步。
4. 有明确完成标准。

5. 可以验证结果。
6. 人工操作重复且耗时。
7. 失败风险可控。

例子：

1. 自动整理工单。
2. 代码修复和测试。
3. 数据报表生成。
4. 企业知识检索和汇总。
5. 浏览器后台操作。
6. 运维排查辅助。

不适合的任务：目标模糊、风险不可控、没有验收标准、强实时且低延迟、涉及不可逆高风险动作。

7.3 Agent 产品不要一开始全自动

很多 Agent 产品适合从辅助模式开始。

成熟路径：

1. 建议模式：Agent 给出建议，不执行。
2. 草稿模式：Agent 生成草稿，用户确认。
3. 半自动模式：Agent 执行低风险步骤，高风险步骤确认。
4. 自动模式：在边界清楚、验证充分后自动执行。

一开始就全自动，容易因为失败案例损害用户信任。先做人机协同，能更快收集数据和建立信任。

产品设计上可以把自动化层级写清楚：

1. **suggest**：只给建议，不执行动作。
2. **draft**：生成草稿或操作计划，用户确认后执行。
3. **semi_auto**：低风险步骤自动执行，高风险步骤确认。
4. **approval**：Agent 准备完整动作包，由负责人审批。
5. **auto**：仅在边界清楚、回归评估稳定、权限和回滚充分时使用。

面试中要强调：自动化层级是产品决策，不是模型能力越强就越自动。高风险任务如果没有验收器、人审、审计和回滚，就应该停留在 copilot 或 approval 形态。

7.4 工具权限设计

Agent 产品必须设计工具权限。

工具可以分层：

1. 只读工具：搜索、查询、读取文档。
2. 低风险写入：生成草稿、创建临时文件。
3. 中风险操作：修改内部记录、提交工单。
4. 高风险操作：删除、支付、发邮件、改权限。

权限原则：

1. 默认最小权限。
2. 按任务授权。
3. 按用户身份授权。
4. 高风险动作确认。
5. 工具层强校验。
6. 记录审计日志。

不要把权限控制写在 prompt 里就结束。系统层必须强制执行。

工具权限至少要有三层：

1. 用户权限：当前用户是否有权访问这个资源或执行这个动作。
2. 任务权限：当前任务是否真的需要这个工具和这个动作范围。
3. 风险权限：动作是否外发、删除、改权限、改生产数据或产生费用。

如果一个 Agent 拿到“全量工具 + 全量数据 + 自由执行”的权限，哪怕模型大多数时候表现很好，也不是生产级设计。生产系统应把工具执行器、权限检查器、参数校验器和审计系统放在模型之外。

7.5 人工确认

人工确认是 Agent 产品的安全阀。

需要确认的动作：

1. 对外发送内容。
2. 删除或覆盖数据。
3. 修改权限。
4. 提交审批。
5. 产生费用。
6. 影响客户或生产系统。

确认界面要展示：

1. Agent 准备做什么。
2. 影响对象是谁。
3. 参数是什么。
4. 为什么要做。
5. 可能风险是什么。
6. 如何取消。

如果确认信息不清楚，用户只是盲点“同意”，风险并没有降低。

7.6 失败恢复

Agent 产品一定会遇到失败。

失败类型：

1. 工具不可用。
2. 参数错误。
3. 权限不足。
4. 信息不完整。
5. 任务目标冲突。
6. 外部系统状态变化。
7. 执行结果不符合预期。

恢复策略：

1. 重试。
2. 更换工具。
3. 请求用户补充信息。
4. 降级为人工处理。
5. 回滚或撤销。
6. 输出部分完成结果。
7. 停止并说明原因。

成熟 Agent 产品不要求永远成功，但必须失败得可控、可解释、可恢复。

7.7 状态管理

Agent 执行任务需要维护状态。

状态包括：

1. 用户目标。
2. 当前计划。
3. 已完成步骤。
4. 已调用工具。
5. 工具结果。
6. 失败尝试。
7. 待确认动作。
8. 剩余预算。

没有状态管理，Agent 容易重复操作、忘记约束、误以为任务已完成。

7.8 审计日志

Agent 产品必须有 trace 和审计。

要记录：

1. 用户输入。
2. 计划。
3. 每一步 action。
4. 工具参数。
5. 工具返回摘要。
6. 权限检查。
7. 用户确认。
8. 错误和重试。
9. 最终结果。

审计日志用于排查问题、评估效果、安全合规和责任追踪。日志要脱敏，避免成为新的泄露源。

7.9 体验设计

Agent 产品体验不能只有“正在执行”。

用户需要看到：

1. Agent 计划做什么。
2. 当前执行到哪一步。
3. 哪些步骤已完成。
4. 哪些步骤失败。
5. 需要用户确认什么。
6. 最终结果和证据。

对长任务，进度反馈非常重要。用户不需要看完整内部推理，但需要知道系统在做什么，是否卡住，是否需要介入。

7.10 成本控制

Agent 通常成本高。

成本来自：

1. 多轮模型调用。
2. 工具调用。
3. 检索。
4. 浏览器操作。
5. 代码执行。
6. 失败重试。
7. 人工确认。

8. 日志存储。

控制方式：

1. 限制最大步骤。
2. 限制工具调用次数。
3. 简单任务不用 Agent。
4. 对低价值任务降级。
5. 缓存可复用结果。
6. 明确停止条件。

Agent 产品需要 budget manager，否则容易因为少数复杂任务消耗大量资源。

7.11 Agent 与 Workflow 结合

生产系统常把 Agent 嵌入 workflow。

例如工单处理：

接收工单 -> 分类 -> Agent 补充信息 -> 人工确认 -> 提交处理 -> 记录结果

Workflow 负责稳定流程，Agent 负责复杂判断和生成。这样比让 Agent 完全自由行动更可控。

7.12 Agent 产品评估

评估指标：

1. 任务完成率。
2. 平均步骤数。
3. 工具调用成功率。
4. 错误恢复成功率。
5. 人工确认比例。
6. 人工接管比例。
7. 平均成本。
8. 平均延迟。
9. 安全违规率。
10. 用户满意度。

不要只看最终回答。Agent 的价值和风险都在执行轨迹里。

7.12.1 关键公式与 Agent 产品指标速查

可以把一次 Agent 产品任务样本写成：

$$a_i = (g_i, u_i, \ell_i, T_i, s_i, A_i, O_i, H_i, C_i, L_i, B_i)$$

其中 g_i 是用户目标， u_i 是用户和角色， ℓ_i 是自动化层级， T_i 是可用工具集合， s_i 是结构化状态， A_i 是动作序列， O_i 是观察结果， H_i 是人工确认记录， C_i 是成本， L_i 是延迟， B_i 是业务结果。

1. 任务成功率

Agent 产品首先要完成任务，而不是只生成看似合理的过程：

$$R_{\{\mathrm{task}\}} = \frac{1}{N} \sum_{i=1}^N y_i$$

其中 $y_i=1$ 表示第 i 个任务满足验收标准，例如工单被正确补全、代码测试通过、报表生成并被采纳。没有验收标准的 Agent，很难评估是否真的有价值。

2. 工具执行成功率

工具调用要看选择、参数、权限和执行结果：

$$R_{\{\mathrm{tool}\}} = \frac{1}{M} \sum_{j=1}^M e_j$$

其中 M 是工具调用总数, $e_j=1$ 表示第 j 次工具调用工具选择正确、参数合法、权限通过且执行成功。工具调用失败会带来重试、延迟、成本和用户不信任。

3. 高风险确认覆盖率

对外发送、删除、支付、改权限和生产变更等动作必须看确认覆盖:

$$C_{\{\mathrm{conf}\}} = \frac{\sum_{j=1}^M z_j h_j}{\sum_{j=1}^M z_j + \epsilon}$$

其中 $z_j=1$ 表示第 j 个动作是高风险动作, $h_j=1$ 表示有清晰的用户确认、审批或二次验证。确认界面必须展示动作、对象、参数、原因和风险, 否则用户只是盲点同意。

4. 未授权动作率

Agent 越能行动, 越要控制越权:

$$R_{\{\mathrm{unauth}\}} = \frac{N_{\{\mathrm{unauth}\}}}{M + \epsilon}$$

其中 $N_{\{\mathrm{unauth}\}}$ 是未授权工具调用、越权数据访问、越权外发或越权修改事件数。严重越权事件不能被平均任务成功率掩盖。

5. 失败恢复率

成熟 Agent 产品不要求永远不失败, 但要求失败可控:

$$R_{\{\mathrm{rec}\}} = \frac{N_{\{\mathrm{rec_ok}\}}}{N_{\{\mathrm{rec_need}\}} + \epsilon}$$

其中 $N_{\{\mathrm{rec_need}\}}$ 是需要恢复的失败次数, $N_{\{\mathrm{rec_ok}\}}$ 是通过重试、换工具、澄清、降级、回滚或人工接管成功恢复的次数。

6. 状态更新覆盖率与观察使用率

Agent 的执行质量在 trace 里:

$$C_{\{\mathrm{state}\}} = \frac{N_{\{\mathrm{state_ok}\}}}{N_{\{\mathrm{state_need}\}} + \epsilon}, \quad R_{\{\mathrm{obs}\}} = \frac{N_{\{\mathrm{obs_used}\}}}{N_{\{\mathrm{obs}\}} + \epsilon}$$

状态更新覆盖率低, 说明 Agent 可能忘记约束、重复动作或误判完成; 观察使用率低, 说明它拿到了工具结果却没有真正纳入下一步决策。

7. 预算超限率

Agent 产品必须有预算管理:

$$R_{\{\mathrm{budget}\}} = \frac{N_{\{\mathrm{over}\}}}{N + \epsilon}$$

其中 $N_{\{\mathrm{over}\}}$ 是超过步数、工具调用、token、时间或费用预算的任务数。复杂 Agent 如果没有停止条件, 少数长尾任务会吞掉大量资源。

8. Agent 产品上线门禁

一个简化上线门禁可以写成:

$$G_{\{\mathrm{agentprod}\}} = \mathbf{1}[R_{\{\mathrm{task}\}} \geq r_0] \mathbf{1}[R_{\{\mathrm{tool}\}} \geq t_0] \mathbf{1}[C_{\{\mathrm{mathr$$

真实项目还要加入 trace 完整率、恢复率、人工接管率、成本、评估覆盖、反馈闭环和业务指标。Agent 产品不能只用“最终答案看起来对”做上线判断。

7.13 适合先落地的 Agent 场景

更适合作为早期落地:

1. 内部工具。
2. 低风险任务。
3. 可验证结果。
4. 有人工确认。
5. 数据权限清晰。
6. 用户愿意试用。

例如内部代码助手、工单摘要、知识检索、报表草稿，比直接让 Agent 自动处理客户资金或生产权限更适合早期试点。

7.14 常见失败模式

1. 任务边界不清。
2. 工具权限过大。
3. 没有人工确认。
4. 出错后无限重试。
5. 用户不知道 Agent 在做什么。
6. 没有审计日志。
7. 成本失控。
8. 最终结果不可验证。
9. 高风险动作自动执行。
10. 把 demo 当生产系统。

Agent 产品失败通常不是因为模型不会思考，而是因为系统没有控制行动边界。

7.15 面试题：Agent 产品落地最重要的是什么

回答要点：

Agent 产品落地最重要的是控制任务边界和行动风险。Agent 能执行工具和真实动作，所以要明确适用场景、工具权限、状态管理、人

7.16 面试题：如何设计一个企业 Agent 产品

回答要点：

我会先选择高价值但风险可控的任务，定义完成标准。系统上设计 planner、tool registry、executor、state manager、permission

7.17 最小可运行 Agent 产品审计 demo

下面这个 demo 用 0 依赖 Python 模拟 Agent 产品上线审计。它把任务成功、工具执行、高风险确认、失败恢复、状态更新、观察使用、trace 覆盖、越权动作、预算超限、延迟、单位成本、评估集、反馈闭环和业务指标放进同一张表。

```
def ratio(num, den):
    return num / den if den else 0.0

def audit_agent_product(agent):
    task_success = ratio(agent["tasks_completed"], agent["tasks_total"])
    tool_success = ratio(agent["tool_successes"], agent["tool_calls"])
    confirmation_coverage = (
        ratio(agent["high_risk_confirmed"], agent["high_risk_actions"])
        if agent["high_risk_actions"]
        else 1.0
    )
    recovery_rate = (
        ratio(agent["recoveries_succeeded"], agent["recoveries_needed"])
        if agent["recoveries_needed"]
        else 1.0
    )
    state_update_coverage = ratio(agent["state_updates"], agent["state_update_required"])
    observation_use_rate = ratio(agent["observations_used"], agent["observations_total"])
    trace_coverage = ratio(agent["trace_complete"], agent["tasks_total"])
    unauthorized_rate = ratio(agent["unauthorized_actions"], agent["tool_calls"])
```

```

budget_overrun_rate = ratio(agent["budget_overruns"], agent["tasks_total"])
gates = {
    "task_success": task_success >= 0.75,
    "tool_success": tool_success >= 0.85,
    "high_risk_confirmation": confirmation_coverage >= 0.90,
    "recovery": recovery_rate >= 0.60,
    "state_update": state_update_coverage >= 0.80,
    "observation_use": observation_use_rate >= 0.80,
    "trace_coverage": trace_coverage >= 0.90,
    "unauthorized_action": unauthorized_rate == 0.0,
    "budget_overrun": budget_overrun_rate <= 0.10,
    "p95_latency": agent["p95_latency_ms"] <= agent["latency_slo_ms"],
    "unit_cost": agent["cost_per_task"] <= agent["cost_slo"],
    "eval_ready": agent["eval_ready"] >= 0.80,
    "feedback_loop": agent["feedback_loop"] >= 0.75,
    "business_metric": agent["business_metric_defined"] >= 1.0,
}
agent_score = (
    0.18 * task_success
    + 0.12 * tool_success
    + 0.10 * confirmation_coverage
    + 0.10 * recovery_rate
    + 0.10 * state_update_coverage
    + 0.08 * observation_use_rate
    + 0.08 * trace_coverage
    + 0.08 * (1.0 - min(unauthorized_rate, 1.0))
    + 0.06 * (1.0 - min(budget_overrun_rate, 1.0))
    + 0.04 * (1.0 if gates["p95_latency"] else 0.0)
    + 0.03 * agent["eval_ready"]
    + 0.03 * agent["feedback_loop"]
)
return {
    "name": agent["name"],
    "mode": agent["mode"],
    "agent_score": round(agent_score, 3),
    "agent_gate": all(gates.values()),
    "metrics": {
        "task_success": round(task_success, 3),
        "tool_success": round(tool_success, 3),
        "confirmation_coverage": round(confirmation_coverage, 3),
        "recovery_rate": round(recovery_rate, 3),
        "unauthorized_rate": round(unauthorized_rate, 3),
        "budget_overrun_rate": round(budget_overrun_rate, 3),
    },
    "failed_gates": [name for name, ok in gates.items() if not ok],
}

```

```

agents = [
    {
        "name": "support_ticket_agent",
        "mode": "semi_auto",
        "tasks_total": 120,
        "tasks_completed": 101,
        "tool_calls": 260,
    }
]

```

```

    "tool_successes": 238,
    "high_risk_actions": 18,
    "high_risk_confirmed": 18,
    "recoveries_needed": 22,
    "recoveries_succeeded": 16,
    "state_update_required": 240,
    "state_updates": 218,
    "observations_total": 260,
    "observations_used": 236,
    "trace_complete": 116,
    "unauthorized_actions": 0,
    "budget_overruns": 8,
    "p95_latency_ms": 4200,
    "latency_slo_ms": 5000,
    "cost_per_task": 0.18,
    "cost_slo": 0.25,
    "eval_ready": 0.86,
    "feedback_loop": 0.80,
    "business_metric_defined": 1.0,
  },
  {
    "name": "code_fix_agent",
    "mode": "copilot",
    "tasks_total": 70,
    "tasks_completed": 48,
    "tool_calls": 190,
    "tool_successes": 168,
    "high_risk_actions": 4,
    "high_risk_confirmed": 4,
    "recoveries_needed": 28,
    "recoveries_succeeded": 15,
    "state_update_required": 150,
    "state_updates": 126,
    "observations_total": 190,
    "observations_used": 151,
    "trace_complete": 66,
    "unauthorized_actions": 0,
    "budget_overruns": 11,
    "p95_latency_ms": 7600,
    "latency_slo_ms": 8000,
    "cost_per_task": 0.42,
    "cost_slo": 0.45,
    "eval_ready": 0.78,
    "feedback_loop": 0.70,
    "business_metric_defined": 1.0,
  },
  {
    "name": "data_ops_agent",
    "mode": "approval",
    "tasks_total": 80,
    "tasks_completed": 55,
    "tool_calls": 210,
    "tool_successes": 171,
    "high_risk_actions": 24,
    "high_risk_confirmed": 18,

```

```

        "recoveries_needed": 30,
        "recoveries_succeeded": 17,
        "state_update_required": 190,
        "state_updates": 139,
        "observations_total": 210,
        "observations_used": 149,
        "trace_complete": 65,
        "unauthorized_actions": 2,
        "budget_overruns": 13,
        "p95_latency_ms": 9100,
        "latency_slo_ms": 6500,
        "cost_per_task": 0.31,
        "cost_slo": 0.30,
        "eval_ready": 0.75,
        "feedback_loop": 0.58,
        "business_metric_defined": 1.0,
    },
    {
        "name": "generic_browser_agent",
        "mode": "auto",
        "tasks_total": 60,
        "tasks_completed": 29,
        "tool_calls": 240,
        "tool_successes": 150,
        "high_risk_actions": 30,
        "high_risk_confirmed": 9,
        "recoveries_needed": 35,
        "recoveries_succeeded": 8,
        "state_update_required": 180,
        "state_updates": 80,
        "observations_total": 240,
        "observations_used": 96,
        "trace_complete": 36,
        "unauthorized_actions": 7,
        "budget_overruns": 21,
        "p95_latency_ms": 12500,
        "latency_slo_ms": 7000,
        "cost_per_task": 0.62,
        "cost_slo": 0.28,
        "eval_ready": 0.40,
        "feedback_loop": 0.20,
        "business_metric_defined": 0.0,
    },
]

results = [audit_agent_product(agent) for agent in agents]
ranked = sorted(
    [(r["name"], r["agent_score"], r["agent_gate"]) for r in results],
    key=lambda item: item[1],
    reverse=True,
)
agent_pass = [r["name"] for r in results if r["agent_gate"]]
needs_rework = {
    r["name"]: r["failed_gates"]
    for r in results
}

```

```

    if not r["agent_gate"]
}

print("ranked=", ranked)
print("agent_pass=", agent_pass)
print("sample_metrics=", results[0]["metrics"])
print("needs_rework=", needs_rework)

```

一组典型输出是：

```

ranked= [('support_ticket_agent', 0.901, True), ('code_fix_agent', 0.821, False), ('data_ops_agent', 0.717, False), ('ge
agent_pass= ['support_ticket_agent']
sample_metrics= {'task_success': 0.842, 'tool_success': 0.915, 'confirmation_coverage': 1.0, 'recovery_rate': 0.727, 'un
needs_rework= {'code_fix_agent': ['task_success', 'recovery', 'observation_use', 'budget_overrun', 'eval_ready', 'feedb

```

这个 demo 的重点是：Agent 产品不是越自动越好。support_ticket_agent 通过门禁，是因为它是半自动、任务边界清楚、确认覆盖和 trace 稳定；code_fix_agent 适合继续做 copilot，因为任务成功、恢复、预算和反馈还没过线；data_ops_agent 虽然有审批形态，但高风险确认、越权、成本和延迟不过线；generic_browser_agent 说明通用全自动浏览器 Agent 如果没有强权限、预算、确认和评估，很难进入生产。

7.18 本章小结

Agent 产品化的核心是让模型在受控范围内执行任务。它比普通问答更有价值，也更有风险。落地时要优先选择边界清楚、可验证、低风险的任务，从辅助和半自动开始，逐步扩大自动化能力。

下一章会进入多模态产品落地，讨论图像、语音、视频和多模态理解生成能力如何转化为真实产品体验。

第八章：多模态产品落地

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对五类口径：OpenAI images / vision、image generation、audio / speech、Realtime API 和 Sora / Video API 文档提醒我们，多模态产品不是单一能力，而是图像输入、图像生成、音频输入输出、低延迟语音、多模态上下文和视频生成的组合工程；OpenAI Evals API 资料提醒我们，评估样本可以包含文本、图像和音频输入，产品不能只看主观演示效果；Sora 系统卡和官方资料提醒我们，视频生成要关注时间一致性、人物 / 风格保持、授权、深伪和内容安全；OWASP LLM Top 10 和 prompt injection 资料提醒我们，不可信指令可以藏在图片、音频、视频、截图和文档里；NIST AI RMF Generative AI Profile 提醒我们，多模态生成式系统要同时管理安全、隐私、知识产权、透明度和人类监督风险。

本章不替代第十五册多模态模型原理、视觉编码器、扩散模型、语音模型、视频生成和多模态安全章节，也不展开具体 API 调用。这里聚焦产品落地：如何把图像、语音、视频和文档能力变成可评估、可控成本、可处理隐私版权、可人审、可进入真实工作流的产品体验。

多模态产品把大模型从文本扩展到图像、语音、视频、文档、屏幕和真实世界场景。它的产品价值很直观：看图问答、拍照识别、语音助手、视频生成、设计创作、教育讲解、工业质检、医疗影像辅助等。但多模态落地也更复杂，因为输入质量、版权、隐私、安全、延迟和评估都比纯文本更难。

本章系统讲多模态产品落地：图像理解、语音助手、视频生成、设计工具、教育和创作、企业文档多模态、行业场景、数据版权、安全审核、体验指标和常见失败模式。

8.1 多模态产品为什么重要

现实世界不是只有文本。用户经常需要处理图片、语音、视频、表格、截图、扫描件和网页界面。

多模态能力能带来这些价值：

1. 降低输入门槛。
2. 支持更自然交互。

3. 处理复杂内容。
4. 提升创作效率。
5. 连接线上和线下场景。
6. 拓展行业应用。

面试回答：

多模态产品的价值在于让模型处理真实世界的信息，而不只是文本。比如图像理解可以做质检、文档解析和拍照问答；语音可以做会议

8.2 图像理解产品

图像理解是最成熟的多模态方向之一。

常见产品：

1. 拍照问答。
2. 图片内容识别。
3. 截图分析。
4. 文档 OCR 和理解。
5. 商品识别。
6. 工业质检。
7. 医学影像辅助。
8. 设计稿评审。

图像理解产品要关注：

1. 图片清晰度。
2. 目标是否被遮挡。
3. OCR 准确率。
4. 小目标识别。
5. 位置关系理解。
6. 置信度和人工复核。

图片看似直观，但模型可能看错细节。高风险场景必须谨慎。

8.3 文档多模态理解

企业文档常包含文本、表格、图片、流程图、扫描页和印章。

文档多模态产品可以做：

1. 合同审查。
2. 发票识别。
3. 报表解读。
4. 简历解析。
5. 论文阅读。
6. 标书分析。
7. 保险理赔材料审核。

难点：

1. OCR 错误。
2. 表格结构复杂。
3. 版面关系重要。
4. 图片中的信息容易丢失。
5. 扫描件质量不稳定。
6. 隐私和合规要求高。

这类产品通常需要 OCR、layout analysis、表格解析、RAG 和人工审核配合。

8.4 语音助手

语音是自然交互入口。

场景包括：

1. 会议转写。
2. 会议纪要。
3. 语音客服。
4. 车载助手。
5. 智能硬件。
6. 口语学习。
7. 电话质检。

语音产品链路通常包括：

语音输入 -> ASR -> LLM 理解和生成 -> TTS 输出

关键体验指标：

1. 识别准确率。
2. 首字延迟。
3. 打断能力。
4. 噪声鲁棒性。
5. 说话人分离。
6. 语气自然度。
7. 多轮上下文。

8.5 语音产品的特殊难点

语音交互比文本更实时。

难点：

1. 用户不能长时间等待。
2. ASR 错误会传递给 LLM。
3. 环境噪声影响大。
4. 多人说话难处理。
5. 口语表达不完整。
6. TTS 语气影响信任感。
7. 隐私风险高。

语音助手要有确认机制。例如涉及金额、地址、日期、权限时，最好复述确认。

8.6 图像生成和设计工具

图像生成产品常见于设计和创作。

场景：

1. 海报生成。
2. 电商商品图。
3. 游戏概念图。
4. 营销素材。
5. UI 灵感图。
6. 头像和插画。

产品价值：

1. 降低创作门槛。
2. 快速生成草稿。
3. 批量探索风格。

4. 提高设计效率。

难点：

1. 可控性。
2. 风格一致性。
3. 版权。
4. 品牌安全。
5. 人物和文字细节。
6. 审核和水印。

图像生成适合辅助创意探索，不一定直接替代专业设计定稿。

8.7 视频生成产品

视频生成比图像生成更复杂。

场景：

1. 短视频草稿。
2. 营销分镜。
3. 教学动画。
4. 产品演示。
5. 虚拟人视频。
6. 影视预览。

难点：

1. 时间一致性。
2. 人物一致性。
3. 动作合理性。
4. 生成成本高。
5. 编辑可控性不足。
6. 版权和深伪风险。

视频生成产品常常先落地在草稿、分镜、辅助创作，而不是直接生成最终商业片。

8.8 教育产品

多模态教育产品可以结合文字、语音、图片、白板和视频。

场景：

1. 拍照解题。
2. 口语陪练。
3. 作文批改。
4. 实验演示。
5. 个性化讲解。
6. 错题本。

教育场景要关注：

1. 不直接给答案，而是引导学习。
2. 适配学生水平。
3. 防止错误讲解。
4. 保护未成年人隐私。
5. 给家长和老师可控入口。

教育产品的目标不是炫技，而是帮助学生真正理解。

8.9 医疗和工业场景

医疗影像、工业质检等场景价值高，但风险也高。

医疗场景：

1. 影像辅助阅读。
2. 病历结构化。
3. 检查报告解释。
4. 医患沟通辅助。

工业场景：

1. 缺陷检测。
2. 仪表读取。
3. 安全巡检。
4. 设备维修指导。

这些场景要强调辅助定位和人审，不应让模型独立承担最终责任。

8.10 输入质量管理

多模态产品对输入质量敏感。

需要处理：

1. 图片模糊。
2. 光照不足。
3. 角度倾斜。
4. 背景干扰。
5. 噪声语音。
6. 视频压缩。
7. 文件格式不兼容。
8. 内容不完整。

产品可以引导用户重新拍摄、裁剪、上传更清晰文件，或者提示“当前输入质量不足”。

8.11 版权和授权

生成式多模态产品必须关注版权。

问题包括：

1. 训练数据版权。
2. 用户上传素材授权。
3. 生成内容是否侵权。
4. 风格模仿是否合规。
5. 商用授权范围。
6. 输出水印和溯源。

企业产品要明确素材来源、授权范围和生成内容使用边界。

8.12 隐私和安全

多模态输入可能包含大量敏感信息。

例如：

1. 人脸。
2. 身份证。
3. 医疗影像。
4. 家庭环境。

5. 公司机密文档。
6. 会议录音。
7. 地理位置。

安全设计：

1. 上传前提醒。
2. 敏感信息检测。
3. 数据最小化保存。
4. 用户删除权。
5. 权限隔离。
6. 日志脱敏。
7. 高风险内容审核。

8.13 内容安全审核

多模态生成容易涉及安全风险。

需要审核：

1. 暴力和色情内容。
2. 仇恨和歧视。
3. 未成年人相关风险。
4. 深伪和冒充。
5. 版权违规。
6. 虚假信息。
7. 品牌风险。

审核应该覆盖输入和输出。用户上传的素材可能违规，模型生成的结果也可能违规。

8.14 体验指标

多模态产品指标包括：

1. 任务成功率。
2. 识别准确率。
3. 生成采纳率。
4. 编辑次数。
5. 首帧或首字延迟。
6. 用户满意度。
7. 重新上传率。
8. 审核拦截率。
9. 成本。
10. 版权投诉率。

对于创作产品，采纳率和编辑效率比单纯图像美观分更接近真实价值。

8.14.1 关键公式与多模态产品指标速查

可以把一次多模态产品任务样本写成：

$$m_i = (x_i, \mu_i, q_i, E_i, \hat{y}_i, y_i, R_i, C_i, L_i, B_i)$$

其中 x_i 是用户目标， μ_i 是输入模态集合，例如文本、图片、音频、视频或文档； q_i 是输入质量标记， E_i 是可用证据或媒体引用， $\hat{y}_i$ 是模型输出， y_i 是验收结果， R_i 是风险审查记录， C_i 是成本， L_i 是延迟， B_i 是业务结果。

1. 多模态上下文预算

图片、音频和视频会把上下文和成本快速放大：

$$T_{\{\mathrm{mm}\}}=T_{\{\mathrm{text}\}}+N_{\{\mathrm{img}\}}+N_{\{\mathrm{aud}\}}+N_{\{\mathrm{vid}\}}+N_{\{\mathrm{extra}\}}$$

其中 T_{text} 是文本 token, N_{img} 是图像或文档页对应的视觉 token, N_{aud} 是音频 token 或声学帧近似量, N_{vid} 是采样帧或视频 token 近似量, N_{extra} 是工具、引用、控制符和历史上下文。多模态产品要把 T_{mm} 和上下文上限、延迟、成本一起看。

2. 输入质量通过率

多模态失败经常不是模型完全不会，而是输入本身质量不足：

$$R_{\{\mathrm{quality}\}}=\frac{1}{N}\sum_{i=1}^Nq_i$$

其中 $q_i=1$ 表示图片清晰、文档可读、音频信噪比足够、视频关键片段完整，且格式和权限满足处理要求。低质量输入应触发重新拍摄、重新上传、裁剪、澄清或转人工。

3. 任务成功率与证据支持率

多模态产品不能只看输出是否流畅：

$$R_{\{\mathrm{task}\}}=\frac{1}{N}\sum_{i=1}^Ny_i,\quad$$

$$S_{\{\mathrm{ev}\}}=\frac{N_{\{\mathrm{claim_ok}\}}}{N_{\{\mathrm{claim}\}}+\epsilon}$$

其中 $y_i=1$ 表示任务通过验收, $N_{\text{claim_ok}}$ 是被图片、音频、视频、OCR、ASR、表格或文档证据支持的关键断言数量, N_{claim} 是输出中的关键断言总数。看图问答、文档问答、医疗影像辅助和质检场景尤其要看证据支持。

4. OCR / ASR 质量

文档、截图、语音产品要单独看识别错误：

$$Q_{\{\mathrm{ocr}\}}=1-\mathrm{CER},\quad$$

$$Q_{\{\mathrm{asr}\}}=1-\mathrm{WER}$$

其中 CER 是字符错误率, WER 是词错误率。平均任务成功率可能掩盖小字、专有名词、噪声、多人说话和低资源语言切片的失败。

5. 生成采纳率

图像和视频生成产品要看用户是否真的采用输出：

$$R_{\{\mathrm{adopt}\}}=\frac{N_{\{\mathrm{adopt}\}}}{N_{\{\mathrm{gen}\}}+\epsilon}$$

其中 N_{gen} 是生成结果数, N_{adopt} 是被保存、下载、进入编辑链路或用于业务草稿的结果数。创作产品的真实价值通常来自采纳、编辑节省时间和品牌可控，而不是单张图的主观好看。

6. 安全、隐私和版权通过率

多模态输入和输出都要审查：

$$R_{\{\mathrm{safety}\}}=\frac{N_{\{\mathrm{safety_ok}\}}}{N_{\{\mathrm{safety}\}}+\epsilon},\quad$$

$$R_{\{\mathrm{privacy}\}}=\frac{N_{\{\mathrm{privacy_ok}\}}}{N_{\{\mathrm{privacy}\}}+\epsilon},\quad$$

$$R_{\{\mathrm{copyright}\}}=\frac{N_{\{\mathrm{copyright_ok}\}}}{N_{\{\mathrm{copyright}\}}+\epsilon}$$

其中安全包括暴力、色情、未成年人、深伪、冒充和不可信媒体指令；隐私包括人脸、身份信息、会议录音、家庭环境和企业机密；版权包括输入素材授权、参考图使用边界、输出使用范围和溯源。

7. 多模态产品上线门禁

一个简化门禁可以写成：

$$G_{\{\mathrm{mmprod}\}}=\mathbf{1}[R_{\{\mathrm{task}\}}\geq r_0]\mathbf{1}[R_{\{\mathrm{quality}\}}\geq q_0]\mathbf{1}[S_{\{\mathrm{evid}\}}\geq s_0]$$

真实项目还要加入 OCR / ASR 切片、生成采纳率、人工复核覆盖、单位成本、评估覆盖、反馈闭环和业务指标。多模态产品最常见的错误，是用少数高质量 demo 图替代真实输入分布评估。

8.15 常见失败模式

1. Demo 图效果好，真实用户输入差。
2. 识别错细节但回答很自信。
3. 生成结果不可控。
4. 视频生成成本过高。
5. 版权边界不清。
6. 隐私数据保存过多。
7. 安全审核只做文本，不做图片和视频。
8. 语音交互延迟太高。
9. 行业场景缺少专家评估。
10. 把辅助工具包装成最终决策系统。

多模态产品落地要同时考虑能力、输入、交互、安全和合规。

8.16 面试题：多模态产品落地难点是什么

回答要点：

多模态产品落地难点包括输入质量不稳定、模型对细节理解可能出错、生成可控性不足、延迟和成本较高、版权和隐私风险更复杂，

8.17 面试题：如何落地图像理解产品

回答要点：

我会先明确图像理解服务哪个任务，例如文档解析、质检、拍照问答或设计评审。然后评估输入质量、目标识别难度、OCR 或 layout

8.18 最小可运行多模态产品审计 demo

下面这个 demo 用 0 依赖 Python 模拟多模态产品上线审计。它把任务成功、输入质量、证据支持、OCR / ASR、生成采纳、安全、隐私、版权、人审、上下文预算、延迟、成本、评估集、反馈闭环和业务指标放进同一张表。

```
def ratio(num, den):
    return num / den if den else 0.0

def audit_multimodal_product(product):
    task_success = ratio(product["tasks_success"], product["samples_total"])
    input_quality = ratio(product["quality_pass"], product["samples_total"])
    evidence_support = (
        ratio(product["supported_claims"], product["total_claims"])
        if product["total_claims"]
        else 1.0
    )
    ocr_quality = 1.0 - product["ocr_cer"] if product["ocr_cer"] is not None else 1.0
    asr_quality = 1.0 - product["asr_wer"] if product["asr_wer"] is not None else 1.0
    generation_adoption = (
        ratio(product["generation_adopted"], product["generation_total"])
        if product["generation_total"]
        else 1.0
    )
    safety_pass = ratio(product["safety_pass"], product["safety_checks"])
    privacy_pass = ratio(product["privacy_pass"], product["privacy_checks"])
    copyright_pass = ratio(product["copyright_pass"], product["copyright_checks"])
    human_review = (
        ratio(product["high_risk_reviewed"], product["high_risk_total"])
        if product["high_risk_total"]
```

```

    else 1.0
)
token_budget_ok = product["token_equiv"] <= product["token_budget"]
gates = {
    "task_success": task_success >= 0.75,
    "input_quality": input_quality >= 0.80,
    "evidence_support": evidence_support >= 0.85,
    "ocr_quality": ocr_quality >= 0.90,
    "asr_quality": asr_quality >= 0.85,
    "generation_adoption": generation_adoption >= 0.55,
    "media_safety": safety_pass >= 0.95,
    "privacy": privacy_pass >= 0.95,
    "copyright": copyright_pass >= 0.90,
    "human_review": human_review >= 0.90,
    "token_budget": token_budget_ok,
    "p95_latency": product["p95_latency_ms"] <= product["latency_slo_ms"],
    "unit_cost": product["cost_per_task"] <= product["cost_slo"],
    "eval_ready": product["eval_ready"] >= 0.80,
    "feedback_loop": product["feedback_loop"] >= 0.75,
    "business_metric": product["business_metric_defined"] >= 1.0,
}
mm_score = (
    0.15 * task_success
    + 0.10 * input_quality
    + 0.10 * evidence_support
    + 0.07 * ocr_quality
    + 0.07 * asr_quality
    + 0.08 * generation_adoption
    + 0.10 * safety_pass
    + 0.08 * privacy_pass
    + 0.06 * copyright_pass
    + 0.06 * human_review
    + 0.04 * (1.0 if token_budget_ok else 0.0)
    + 0.03 * (1.0 if gates["p95_latency"] else 0.0)
    + 0.02 * (1.0 if gates["unit_cost"] else 0.0)
    + 0.02 * product["eval_ready"]
    + 0.02 * product["feedback_loop"]
)
return {
    "name": product["name"],
    "modalities": product["modalities"],
    "mm_score": round(mm_score, 3),
    "mm_gate": all(gates.values()),
    "metrics": {
        "task_success": round(task_success, 3),
        "input_quality": round(input_quality, 3),
        "evidence_support": round(evidence_support, 3),
        "ocr_quality": round(ocr_quality, 3),
        "asr_quality": round(asr_quality, 3),
        "generation_adoption": round(generation_adoption, 3),
        "safety_pass": round(safety_pass, 3),
        "privacy_pass": round(privacy_pass, 3),
        "copyright_pass": round(copyright_pass, 3),
    },
    "failed_gates": [name for name, ok in gates.items() if not ok],
}

```

```

}

products = [
{
    "name": "document_vision_assistant",
    "modalities": ["image", "document", "text"],
    "samples_total": 120,
    "tasks_success": 101,
    "quality_pass": 110,
    "supported_claims": 330,
    "total_claims": 360,
    "ocr_cer": 0.035,
    "asr_wer": None,
    "generation_total": 0,
    "generation_adopted": 0,
    "safety_checks": 120,
    "safety_pass": 118,
    "privacy_checks": 120,
    "privacy_pass": 119,
    "copyright_checks": 120,
    "copyright_pass": 113,
    "high_risk_total": 20,
    "high_risk_reviewed": 20,
    "token_equiv": 6500,
    "token_budget": 8000,
    "p95_latency_ms": 4200,
    "latency_slo_ms": 5000,
    "cost_per_task": 0.19,
    "cost_slo": 0.30,
    "eval_ready": 0.86,
    "feedback_loop": 0.80,
    "business_metric_defined": 1.0,
},
{
    "name": "realtime_voice_support",
    "modalities": ["audio", "text"],
    "samples_total": 90,
    "tasks_success": 70,
    "quality_pass": 78,
    "supported_claims": 160,
    "total_claims": 180,
    "ocr_cer": None,
    "asr_wer": 0.18,
    "generation_total": 0,
    "generation_adopted": 0,
    "safety_checks": 90,
    "safety_pass": 87,
    "privacy_checks": 90,
    "privacy_pass": 86,
    "copyright_checks": 90,
    "copyright_pass": 90,
    "high_risk_total": 20,
    "high_risk_reviewed": 18,
    "token_equiv": 7200,
}
]

```

```

    "token_budget": 7000,
    "p95_latency_ms": 1700,
    "latency_slo_ms": 1200,
    "cost_per_task": 0.22,
    "cost_slo": 0.24,
    "eval_ready": 0.82,
    "feedback_loop": 0.76,
    "business_metric_defined": 1.0,
  },
  {
    "name": "image_campaign_copilot",
    "modalities": ["image_generation", "text"],
    "samples_total": 80,
    "tasks_success": 58,
    "quality_pass": 72,
    "supported_claims": 0,
    "total_claims": 0,
    "ocr_cer": None,
    "asr_wer": None,
    "generation_total": 70,
    "generation_adopted": 42,
    "safety_checks": 80,
    "safety_pass": 76,
    "privacy_checks": 80,
    "privacy_pass": 77,
    "copyright_checks": 80,
    "copyright_pass": 66,
    "high_risk_total": 20,
    "high_risk_reviewed": 14,
    "token_equiv": 5000,
    "token_budget": 6000,
    "p95_latency_ms": 9000,
    "latency_slo_ms": 8000,
    "cost_per_task": 0.31,
    "cost_slo": 0.35,
    "eval_ready": 0.75,
    "feedback_loop": 0.70,
    "business_metric_defined": 1.0,
  },
  {
    "name": "video_storyboard_generator",
    "modalities": ["video_generation", "image", "text"],
    "samples_total": 60,
    "tasks_success": 40,
    "quality_pass": 48,
    "supported_claims": 100,
    "total_claims": 130,
    "ocr_cer": None,
    "asr_wer": None,
    "generation_total": 55,
    "generation_adopted": 24,
    "safety_checks": 60,
    "safety_pass": 56,
    "privacy_checks": 60,
    "privacy_pass": 57,
  },

```



```

        "copyright_checks": 60,
        "copyright_pass": 53,
        "high_risk_total": 15,
        "high_risk_reviewed": 8,
        "token_equiv": 15500,
        "token_budget": 12000,
        "p95_latency_ms": 18000,
        "latency_slo_ms": 10000,
        "cost_per_task": 1.20,
        "cost_slo": 0.80,
        "eval_ready": 0.60,
        "feedback_loop": 0.55,
        "business_metric_defined": 1.0,
    },
    {
        "name": "generic_multimodal_chat",
        "modalities": ["image", "audio", "video", "text"],
        "samples_total": 70,
        "tasks_success": 37,
        "quality_pass": 47,
        "supported_claims": 60,
        "total_claims": 115,
        "ocr_cer": 0.22,
        "asr_wer": 0.35,
        "generation_total": 30,
        "generation_adopted": 10,
        "safety_checks": 70,
        "safety_pass": 60,
        "privacy_checks": 70,
        "privacy_pass": 58,
        "copyright_checks": 70,
        "copyright_pass": 50,
        "high_risk_total": 14,
        "high_risk_reviewed": 0,
        "token_equiv": 11000,
        "token_budget": 8000,
        "p95_latency_ms": 9600,
        "latency_slo_ms": 5000,
        "cost_per_task": 0.52,
        "cost_slo": 0.25,
        "eval_ready": 0.40,
        "feedback_loop": 0.20,
        "business_metric_defined": 0.0,
    },
]

results = [audit_multimodal_product(product) for product in products]
ranked = sorted(
    [(r["name"], r["mm_score"], r["mm_gate"]) for r in results],
    key=lambda item: item[1],
    reverse=True,
)
multimodal_pass = [r["name"] for r in results if r["mm_gate"]]
needs_rework = {
    r["name"]: r["failed_gates"]
}

```

```

    for r in results
    if not r["mm_gate"]
}

print("ranked=", ranked)
print("multimodal_pass=", multimodal_pass)
print("sample_metrics=", results[0]["metrics"])
print("needs_rework=", needs_rework)

```

一组典型输出是：

```

ranked= [('document_vision_assistant', 0.945, True), ('image_campaign_copilot', 0.839, False), ('realtime_voice_support', 0.789, True)]
multimodal_pass= ['document_vision_assistant']
sample_metrics= {'task_success': 0.842, 'input_quality': 0.917, 'evidence_support': 0.917, 'ocr_quality': 0.965, 'asr_quality': 0.917}
needs_rework= {'realtime_voice_support': ['asr_quality', 'token_budget', 'p95_latency'], 'image_campaign_copilot': ['task_success']}

```

这个demo的重点是：多模态产品不能用一两张漂亮样例或一段流畅语音证明可上线。`document_vision_assistant`通过门禁，是因为输入质量、证据支持、OCR、隐私版权、人审、延迟成本和反馈闭环都过线；`realtime_voice_support`任务本身接近可用，但ASR、实时延迟和上下文预算不过线；生成类产品如果版权、人审、采纳率和成本不过线，就应该定位为草稿工具，而不是直接进入高风险业务流程。

8.19 本章小结

多模态产品把大模型能力扩展到图像、语音、视频和真实世界任务。它能显著降低输入门槛、提升创作效率、支持行业应用，但也带来输入质量、延迟、版权、隐私、安全和评估难题。

下一章会进入隐私合规与治理，系统讨论大模型产品如何处理用户数据、企业数据、权限、审计、合规和治理机制。

第九章：隐私、合规与治理

0. 本讲资料边界与第二轮精修口径

本讲按 `WRITING_PLAN.md` 的第二轮要求做公式和 demo 精修。联网资料主要核对五类口径：OpenAI API / enterprise 数据控制和隐私资料提醒我们，产品条款、数据保留、训练使用、企业隔离和审计能力必须和技术实现一致；NIST AI RMF Generative AI Profile 提醒我们，生成式 AI 治理要覆盖数据、隐私、安全、透明度、监控和人类监督；OWASP LLM Top 10 中 sensitive information disclosure、prompt injection、excessive agency 和 unbounded consumption 风险提醒我们，隐私泄露经常发生在 RAG、Agent、工具、日志和不可信内容边界上；GDPR / CCPA 等隐私框架提醒我们，个人数据处理通常要关注告知、目的限制、最小化、保留、删除、访问和处理者责任；企业安全治理资料提醒我们，SSO / IAM / RBAC、租户隔离、审计日志和数据处理协议是企业落地的基本条件。

本章不构成法律意见，也不替代企业法务、安全、合规或数据保护负责人审查。这里聚焦产品和工程面试口径：如何把隐私合规从“页面免责声明”落实到数据流、权限、日志、保留删除、外部处理、训练使用、审计和治理门禁。

大模型产品一旦进入真实业务，就会接触用户输入、企业文档、客户资料、对话日志、代码、图片、语音和业务数据。隐私、合规与治理不是上线前补一条免责声明，而是产品架构、数据流、权限系统、日志系统和组织流程的一部分。做不好这些，模型效果再好也难以进入企业核心场景。

本章系统讲大模型产品的隐私合规与治理：PII、数据最小化、权限隔离、日志治理、数据跨境、训练数据使用、模型输出责任、审计、合规流程和治理机制。

9.1 为什么隐私合规重要

大模型产品的数据流复杂。

一次简单对话可能涉及：

1. 用户输入。
2. 历史上下文。
3. 检索到的企业文档。
4. 工具调用结果。
5. 模型输出。
6. 日志记录。
7. 反馈数据。
8. 人工审核。

任何一个环节都可能泄露敏感信息。

面试回答：

大模型产品必须重视隐私合规，因为模型会处理用户输入、企业知识、日志、工具结果和生成内容。风险不仅是模型训练泄露数据，

9.2 PII 是什么

PII 是 personally identifiable information，也就是可以识别个人身份的信息。

常见 PII：

1. 姓名。
2. 手机号。
3. 邮箱。
4. 身份证号。
5. 地址。
6. 银行卡号。
7. 医疗记录。
8. 人脸图像。
9. 声纹。
10. 精确位置。

大模型产品要识别 PII，并根据场景做脱敏、拒绝、权限控制或用户确认。

9.3 企业敏感数据

企业敏感数据不一定是个人隐私，也可能是商业机密。

例如：

1. 客户名单。
2. 合同价格。
3. 财务数据。
4. 研发代码。
5. 产品路线图。
6. 内部会议纪要。
7. 安全漏洞信息。
8. 供应商协议。

企业 LLM 应用要区分不同数据密级，并和权限系统、审计系统结合。

9.4 数据最小化

数据最小化原则是：只收集、使用和保存完成任务所必要的的数据。

具体做法：

1. 不需要的字段不传给模型。
2. 能脱敏就脱敏。
3. 能本地处理就不外发。

4. 日志只保存必要摘要。
5. 长期 memory 不保存临时敏感信息。
6. 工具调用只传必要参数。

很多隐私问题来自“为了方便，把所有上下文都传给模型”。产品化时要控制数据流。

9.5 数据流梳理

治理前先画清楚数据流。

需要知道：

1. 数据从哪里来。
2. 经过哪些系统。
3. 传给哪个模型或服务。
4. 是否出境或出企业边界。
5. 谁能访问。
6. 保存多久。
7. 是否用于训练或评估。
8. 如何删除。

没有数据流图，就很难做权限、合规和审计。

9.6 权限隔离

权限隔离是企业 LLM 的底线。

要求：

1. 用户只能访问有权限的数据。
2. 不同租户隔离。
3. 不同项目隔离。
4. RAG 检索时做权限过滤。
5. Agent 工具调用按用户身份执行。
6. 日志访问也要权限控制。

一个常见错误是只在 UI 层做权限，后端检索和模型上下文仍然混入无权限内容。这会导致隐性数据泄露。

9.7 日志治理

日志对调试和评估很重要，但日志也可能泄露信息。

日志可能包含：

1. 用户输入。
2. 模型输出。
3. 检索文档。
4. 工具参数。
5. 错误堆栈。
6. API 返回。
7. 用户反馈。

日志治理要考虑：

1. 是否脱敏。
2. 保存多久。
3. 谁能访问。
4. 是否加密。
5. 是否可删除。
6. 是否用于训练。

7. 是否记录审计。

不要为了排查方便，把完整敏感上下文长期存储。

9.8 训练数据使用

用户输入和企业数据是否能用于训练，必须明确。

要回答：

1. 用户是否授权。
2. 数据是否脱敏。
3. 是否用于模型训练、评估或日志分析。
4. 是否会被第三方模型服务使用。
5. 是否能删除。
6. 是否能追踪来源。

企业客户通常非常关心“我的数据会不会被拿去训练其他模型”。产品条款和技术实现必须一致。

9.9 外部模型和数据出境

调用外部模型 API 时，要关注数据是否离开企业边界。

需要评估：

1. 服务商是否保存请求。
2. 是否用于训练。
3. 数据存储地区。
4. 是否满足行业合规。
5. 是否支持企业隔离。
6. 是否有审计和删除机制。

有些企业会要求私有化部署或专有云部署，原因往往不是模型效果，而是数据合规。

9.10 模型输出责任

模型输出可能产生责任问题。

例如：

1. 错误法律建议。
2. 错误医疗解释。
3. 财务分析误导。
4. 生成侵权内容。
5. 生成不合规营销话术。
6. 错误操作建议导致损失。

产品应明确：模型是辅助工具还是自动决策系统；哪些场景需要人工复核；输出是否有来源和限制说明。

9.11 审计

审计是治理的重要部分。

审计记录包括：

1. 谁发起请求。
2. 访问了哪些数据。
3. 使用了哪些模型。
4. 调用了哪些工具。
5. 输出了什么。
6. 是否有人审。

7. 是否触发安全策略。
8. 是否发生异常。

审计不仅用于事后追责，也用于发现系统性风险。

9.12 数据保留和删除

产品要定义数据保留策略。

问题包括：

1. 对话保存多久。
2. 日志保存多久。
3. 上传文件是否保留。
4. 用户能否删除。
5. 删除是否影响备份。
6. memory 如何清除。
7. 企业管理员如何管控。

没有删除机制的大模型产品，很难满足隐私和企业治理要求。

9.13 合规不是只靠法务

合规需要技术、产品、法务、安全和业务共同参与。

技术团队负责：

1. 数据流控制。
2. 权限隔离。
3. 脱敏。
4. 日志治理。
5. 审计。
6. 安全策略实现。

产品团队负责：

1. 用户告知。
2. 授权入口。
3. 删除入口。
4. 人工复核流程。
5. 风险提示。

法务和安全负责审查要求，但不能替代工程实现。

9.14 治理机制

大模型治理机制包括：

1. 数据分级分类。
2. 模型使用规范。
3. Prompt 和工具变更审批。
4. 高风险场景评审。
5. 红队测试。
6. 评估集和回归测试。
7. 日志审计。
8. 事故响应流程。

治理不是为了阻碍创新，而是让大模型应用能进入更高价值、更高风险的业务场景。

9.14.1 关键公式与隐私治理指标速查

可以把一次大模型产品数据处理样本写成：

$$d_i=(u_i,x_i,p_i,s_i,t_i,l_i,a_i,r_i,e_i)$$

其中 u_i 是用户或租户， x_i 是输入数据， p_i 是 PII / 机密 / 数据密级标记， s_i 是处理目的， t_i 是外部传输或模型服务， l_i 是日志记录， a_i 是权限检查， r_i 是保留和删除策略， e_i 是审计事件。

1. 数据最小化率

产品只应传递完成任务所需的数据：

$$R_{\{\mathrm{min}\}}=1-\frac{N_{\{\mathrm{extra}\}}}{N_{\{\mathrm{field}\}}+\epsilon}$$

其中 N_{extra} 是不必要字段数， N_{field} 是候选字段数。低数据最小化率说明系统把过多上下文、文档、日志或工具结果传给模型。

2. PII 脱敏覆盖率与敏感数据阻断率

PII 和企业机密要先识别，再决定脱敏、阻断、转人工或授权处理：

$$C_{\{\mathrm{pii}\}}=\frac{N_{\{\mathrm{pii_redacted}\}}}{N_{\{\mathrm{pii}\}}+\epsilon}, \quad$$

$$R_{\{\mathrm{sens}\}}=\frac{N_{\{\mathrm{sens_blocked}\}}}{N_{\{\mathrm{sens}\}}+\epsilon}$$

其中 N_{pii} 是检测到的 PII 样本数， $N_{\mathrm{pii_redacted}}$ 是正确脱敏或最小化处理的样本数； N_{sens} 是高敏事件数， $N_{\mathrm{sens_blocked}}$ 是被阻断、降级、授权或转人工的事件数。

3. 权限隔离违规率

企业 RAG、Agent 和日志系统必须看越权：

$$R_{\{\mathrm{unauth}\}}=\frac{N_{\{\mathrm{unauth}\}}}{N_{\{\mathrm{access}\}}+\epsilon}$$

其中 N_{unauth} 是越权检索、跨租户访问、无权工具调用、无权日志查看或无权引用展示事件数。隐私治理里，严重越权通常是零容忍门禁。

4. 日志脱敏覆盖率

日志既是评估和排障材料，也是泄露源：

$$C_{\{\mathrm{log}\}}=\frac{N_{\{\mathrm{log_redacted}\}}}{N_{\{\mathrm{log}\}}+\epsilon}$$

其中 N_{log} 是含用户输入、模型输出、工具参数、检索片段或错误栈的日志条数， $N_{\mathrm{log_redacted}}$ 是已完成脱敏、截断、摘要化或权限保护的日志条数。

5. 训练使用授权覆盖率

用户输入、企业数据和反馈数据是否用于训练必须和授权一致：

$$C_{\{\mathrm{train}\}}=\frac{N_{\{\mathrm{train_consent}\}}}{N_{\{\mathrm{train_candidate}\}}+\epsilon}$$

其中 $N_{\mathrm{train_candidate}}$ 是可能进入训练、微调、评估或人工标注的数据条数， $N_{\mathrm{train_consent}}$ 是具备明确授权、脱敏和来源追踪的数据条数。

6. 保留合规率与删除 SLA 通过率

数据不能无限期保存：

$$R_{\{\mathrm{retain}\}}=\frac{N_{\{\mathrm{retain_ok}\}}}{N_{\{\mathrm{retain}\}}+\epsilon}, \quad$$

$$R_{\{\mathrm{del}\}}=\frac{N_{\{\mathrm{del_sla}\}}}{N_{\{\mathrm{del}\}}+\epsilon}$$

其中 $N_{\mathrm{retain_ok}}$ 表示符合保留周期、用途和管理员策略的数据项； $N_{\mathrm{del_sla}}$ 表示删除请求在 SLA 内覆盖原始数据、清洗数据、索引、日志、缓存和可追踪备份。

7. 外部传输审批覆盖率与审计覆盖率

调用外部模型、第三方工具或跨区域服务时，要记录目的、范围和审批：

$$C_{\mathrm{ext}} = \frac{N_{\mathrm{ext_ok}}}{N_{\mathrm{ext}} + \epsilon}, \quad \text{quad}$$

$$C_{\mathrm{audit}} = \frac{N_{\mathrm{audit_ok}}}{N_{\mathrm{event}} + \epsilon}$$

其中 $N_{\mathrm{ext_ok}}$ 是具备审批、处理协议、数据最小化和脱敏措施的外部传输次数； $N_{\mathrm{audit_ok}}$ 是包含用户、租户、数据、模型、工具、风险、策略命中和人审结果的完整审计事件数。

8. 隐私治理上线门禁

一个简化门禁可以写成：

$$G_{\mathrm{priv}} = \mathbf{1}[C_{\mathrm{pii}} \geq p_0] \mathbf{1}[R_{\mathrm{sens}} \geq s_0] \mathbf{1}[R_{\mathrm{unau}}$$

真实项目还要加入 DPA / 企业协议、加密、密钥管理、事故响应、数据保护影响评估、红队回归、高风险人工复核和上线后监控。隐私治理门禁不是平均分，而是一组硬约束。

9.15 常见失败模式

1. 把所有用户输入都长期存日志。
2. RAG 越权召回文档。
3. 多租户数据隔离不彻底。
4. 外部 API 数据使用条款不清。
5. Memory 保存敏感信息。
6. 用户无法删除数据。
7. 模型输出没有风险提示。
8. 高风险建议没有人审。
9. 合规只停留在文档，没有工程实现。
10. 安全事件无法追溯。

隐私合规问题一旦出事，影响通常比模型回答错误更严重。

9.16 面试题：大模型产品如何做隐私保护

回答要点：

我会从数据全链路做隐私保护。首先梳理数据流，明确用户输入、检索文档、工具结果、日志和反馈会流向哪里。然后做数据最小化、

9.17 面试题：企业 RAG 如何避免数据泄露

回答要点：

企业 RAG 要在检索和生成全链路做权限控制。检索时只能召回用户有权限的文档，引用也必须是用户可访问的来源；不能把无权限文

9.18 最小可运行隐私治理审计 demo

下面这个 demo 用 0 依赖 Python 模拟隐私、合规与治理上线审计。它把 PII 脱敏、敏感数据阻断、权限隔离、日志脱敏、训练使用授权、保留策略、删除 SLA、外部传输审批、审计覆盖、人审、事故响应、DPA 和业务指标放进同一张表。

```
def ratio(num, den):
    return num / den if den else 1.0
```

```
def audit_privacy_governance(app):
    pii_redaction = ratio(app["pii_redacted"], app["pii_detected"])
    sensitive_block = ratio(app["sensitive_blocked"], app["sensitive_events"])
    unauthorized_rate = ratio(app["unauthorized_hits"], app["access_checks"])
    permission_isolation = 1.0 - min(unauthorized_rate, 1.0)
    log_redaction = ratio(app["logs_redacted"], app["logs_total"])
    training_consent = ratio(app["training_consent_ok"], app["training_candidates"])
```



```

retention_compliance = ratio(app["retention_ok"], app["retention_items"])
deletion_sla = ratio(app["deletion_sla_met"], app["deletion_requests"])
external_approval = ratio(app["external_approved"], app["external_transfers"])
audit_coverage = ratio(app["audit_complete"], app["audit_events"])
human_review = ratio(app["high_risk_reviewed"], app["high_risk_total"])
gates = {
    "pii_redaction": pii_redaction >= 0.95,
    "sensitive_block": sensitive_block >= 0.95,
    "permission_isolation": unauthorized_rate == 0.0,
    "log_redaction": log_redaction >= 0.90,
    "training_consent": training_consent >= 0.95,
    "retention": retention_compliance >= 0.90,
    "deletion_sla": deletion_sla >= 0.90,
    "external_transfer": external_approval >= 0.95,
    "audit_coverage": audit_coverage >= 0.90,
    "human_review": human_review >= 0.90,
    "incident_response": app["incident_response_ready"] >= 1.0,
    "dpa_ready": app["dpa_ready"] >= 1.0,
    "business_metric": app["business_metric_defined"] >= 1.0,
}
privacy_score = (
    0.11 * pii_redaction
+ 0.09 * sensitive_block
+ 0.12 * permission_isolation
+ 0.09 * log_redaction
+ 0.08 * training_consent
+ 0.08 * retention_compliance
+ 0.08 * deletion_sla
+ 0.08 * external_approval
+ 0.09 * audit_coverage
+ 0.06 * human_review
+ 0.04 * app["incident_response_ready"]
+ 0.03 * app["dpa_ready"]
+ 0.05 * app["business_metric_defined"]
)
return {
    "name": app["name"],
    "privacy_score": round(privacy_score, 3),
    "privacy_gate": all(gates.values()),
    "metrics": {
        "pii_redaction": round(pii_redaction, 3),
        "sensitive_block": round(sensitive_block, 3),
        "unauthorized_rate": round(unauthorized_rate, 3),
        "log_redaction": round(log_redaction, 3),
        "training_consent": round(training_consent, 3),
        "retention": round(retention_compliance, 3),
        "deletion_sla": round(deletion_sla, 3),
        "external_approval": round(external_approval, 3),
        "audit_coverage": round(audit_coverage, 3),
    },
    "failed_gates": [name for name, ok in gates.items() if not ok],
}

```

```
apps = [
```

```

{
  "name": "enterprise_rag_assistant",
  "pii_detected": 120,
  "pii_redacted": 118,
  "sensitive_events": 80,
  "sensitive_blocked": 78,
  "access_checks": 600,
  "unauthorized_hits": 0,
  "logs_total": 1000,
  "logs_redacted": 950,
  "training_candidates": 100,
  "training_consent_ok": 100,
  "retention_items": 200,
  "retention_ok": 190,
  "deletion_requests": 20,
  "deletion_sla_met": 19,
  "external_transfers": 10,
  "external_approved": 10,
  "audit_events": 1000,
  "audit_complete": 960,
  "high_risk_total": 50,
  "high_risk_reviewed": 50,
  "incident_response_ready": 1.0,
  "dpa_ready": 1.0,
  "business_metric_defined": 1.0,
},
{
  "name": "contract_review_copilot",
  "pii_detected": 100,
  "pii_redacted": 95,
  "sensitive_events": 65,
  "sensitive_blocked": 58,
  "access_checks": 400,
  "unauthorized_hits": 1,
  "logs_total": 400,
  "logs_redacted": 330,
  "training_candidates": 40,
  "training_consent_ok": 40,
  "retention_items": 190,
  "retention_ok": 170,
  "deletion_requests": 16,
  "deletion_sla_met": 12,
  "external_transfers": 6,
  "external_approved": 5,
  "audit_events": 400,
  "audit_complete": 350,
  "high_risk_total": 35,
  "high_risk_reviewed": 30,
  "incident_response_ready": 1.0,
  "dpa_ready": 1.0,
  "business_metric_defined": 1.0,
},
{
  "name": "customer_voice_agent",
  "pii_detected": 190,

```

```

        "pii_redacted": 150,
        "sensitive_events": 90,
        "sensitive_blocked": 60,
        "access_checks": 500,
        "unauthorized_hits": 2,
        "logs_total": 800,
        "logs_redacted": 500,
        "training_candidates": 80,
        "training_consent_ok": 60,
        "retention_items": 200,
        "retention_ok": 120,
        "deletion_requests": 20,
        "deletion_sla_met": 8,
        "external_transfers": 40,
        "external_approved": 25,
        "audit_events": 800,
        "audit_complete": 500,
        "high_risk_total": 40,
        "high_risk_reviewed": 15,
        "incident_response_ready": 0.0,
        "dpa_ready": 1.0,
        "business_metric_defined": 1.0,
    },
    {
        "name": "generic_chat_portal",
        "pii_detected": 140,
        "pii_redacted": 70,
        "sensitive_events": 70,
        "sensitive_blocked": 30,
        "access_checks": 300,
        "unauthorized_hits": 8,
        "logs_total": 900,
        "logs_redacted": 360,
        "training_candidates": 200,
        "training_consent_ok": 30,
        "retention_items": 260,
        "retention_ok": 80,
        "deletion_requests": 30,
        "deletion_sla_met": 5,
        "external_transfers": 55,
        "external_approved": 20,
        "audit_events": 900,
        "audit_complete": 350,
        "high_risk_total": 60,
        "high_risk_reviewed": 5,
        "incident_response_ready": 0.0,
        "dpa_ready": 0.0,
        "business_metric_defined": 0.0,
    },
]

results = [audit_privacy_governance(app) for app in apps]
ranked = sorted(
    [(r["name"], r["privacy_score"], r["privacy_gate"]) for r in results],
    key=lambda item: item[1],

```

```

    reverse=True,
)
privacy_pass = [r["name"] for r in results if r["privacy_gate"]]
needs_rework = {
    r["name"]: r["failed_gates"]
    for r in results
    if not r["privacy_gate"]
}

print("ranked=", ranked)
print("privacy_pass=", privacy_pass)
print("sample_metrics=", results[0]["metrics"])
print("needs_rework=", needs_rework)

```

一组典型输出是：

```

ranked= [('enterprise_rag_assistant', 0.98, True), ('contract_review_copilot', 0.907, False), ('customer_voice_agent', 0.85, True)]
privacy_pass= ['enterprise_rag_assistant']
sample_metrics= {'pii_redaction': 0.983, 'sensitive_block': 0.975, 'unauthorized_rate': 0.0, 'log_redaction': 0.95, 'trace_redaction': 0.95}
needs_rework= {'contract_review_copilot': ['sensitive_block', 'permission_isolation', 'log_redaction', 'retention', 'trace_redaction']}

```

这个 demo 的重点是：隐私合规不是“加一个提示词”或“写一份政策”就结束。enterprise_rag_assistant 通过门禁，是因为 PII、敏感数据、权限、日志、训练授权、保留删除、外部传输、审计、人审和协议都过线；contract_review_copilot 虽然业务价值可能高，但只要出现越权、日志脱敏、删除 SLA 或外部传输审批问题，就不能直接进入高风险企业流程；generic_chat_portal 说明通用入口如果没有数据流和治理门禁，隐私风险会快速累积。

9.19 本章小结

隐私、合规与治理是大模型产品进入企业和高风险行业的基础。它覆盖 PII、企业敏感数据、数据最小化、权限隔离、日志治理、训练数据使用、外部 API、输出责任、审计、删除和组织治理。

下一章会进入上线运营与反馈闭环，讨论大模型产品上线后如何监控、收集反馈、分析失败、持续迭代和稳定运营。

第十章：上线运营与反馈闭环

0. 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做公式和 demo 精修。联网资料主要核对七类口径：OpenAI Evals / graders 资料提醒我们，线上反馈要回流为可复现评估、打分器和回归报告；OpenAI 生产最佳实践和 latency optimization 资料提醒我们，生产系统要持续监控错误、延迟、重试、限流、成本和用户体验；OpenAI Agents tracing 资料提醒我们，Agent 和工具型产品要保留可审计 trace；OpenAI safety best practices 提醒我们，上线后仍要做安全监控、滥用检测和事故响应；Google SRE 的 SLI / SLO / error budget 口径提醒我们，平均值不足以描述可用性和长尾体验；NIST AI RMF Generative AI Profile 提醒我们，生成式 AI 上线后要持续治理、监控、度量和人类监督；OWASP LLM Top 10 提醒我们，敏感信息泄露、提示注入、过度代理和资源消耗风险会在线上持续出现。

本章不替代企业 SRE、数据安全、隐私合规或真实增长实验设计。这里聚焦产品和工程面试口径：如何把大模型产品上线后的质量、成本、延迟、安全、反馈、回归评估和版本治理做成可观测、可回滚、可迭代的闭环。

大模型产品上线不是终点，而是持续迭代的开始。模型会遇到训练时没见过的问题，用户会用意想不到的方式提问，知识库会过期，成本会变化，模型版本会更新，安全风险会暴露。没有运营和反馈闭环的大模型产品，很快会从“看起来智能”变成“不稳定、不可信、没人用”。

本章系统讲上线运营与反馈闭环：灰度发布、质量监控、用户反馈、A/B test、失败样本分析、数据闭环、模型和 prompt 迭代、成本监控、安全监控和持续运营。

10.1 上线为什么不是终点

传统软件上线后，主要关注 bug 和性能。大模型产品上线后，还要关注模型行为。

例如：

1. 用户问法是否符合预期。
2. 模型是否幻觉。
3. RAG 是否召回正确文档。
4. Agent 是否误用工具。
5. 成本是否超预算。
6. 拒答是否过多或过少。
7. 用户是否真的采纳输出。

面试回答：

大模型产品上线后必须持续运营，因为模型行为会受到用户输入、知识库更新、模型版本、prompt、工具和业务环境影响。需要监控/

10.2 灰度发布

大模型产品不应该一次全量上线。

灰度发布可以按：

1. 用户群体。
2. 业务线。
3. 地区。
4. 功能模块。
5. 风险等级。
6. 流量比例。

灰度阶段要观察：

1. 使用率。
2. 成功率。
3. 用户反馈。
4. 错误类型。
5. 成本。
6. 延迟。
7. 安全事件。

灰度不是形式，而是为了在风险可控范围内发现真实问题。

10.3 质量监控

质量监控要覆盖模型输出和任务结果。

常见指标：

1. 答案正确率。
2. 用户满意度。
3. 任务完成率。
4. 引用准确率。
5. 无法回答率。
6. 转人工率。
7. 重试率。
8. 重新生成率。
9. 投诉率。
10. 人工审核通过率。

不同产品指标不同。客服看解决率，RAG 看引用准确率，Agent 看任务完成率，代码助手看采纳率和测试通过率。

10.4 成本监控

大模型产品成本可能悄悄上涨。

需要监控：

1. 平均 token。
2. 输出长度。
3. 模型调用次数。
4. RAG 检索次数。
5. Agent 步数。
6. 工具调用次数。
7. 重试次数。
8. 单用户成本。
9. 单任务成本。
10. 总成本趋势。

成本异常可能来自 prompt 变长、RAG top-k 过大、Agent 无限循环、用户滥用或模型升级。

10.5 延迟监控

延迟影响体验。

要监控：

1. 首 token 延迟。
2. 总生成时间。
3. 检索耗时。
4. 工具调用耗时。
5. 排队时间。
6. 超时率。
7. P50、P95、P99 延迟。

平均延迟不够，长尾延迟更影响用户体验。尤其是企业工作流中，P95 和 P99 很重要。

10.6 用户反馈

用户反馈是优化的重要来源。

反馈形式：

1. 点赞点踩。
2. 选择错误类型。
3. 输入期望答案。
4. 标记引用错误。
5. 请求转人工。
6. 手动编辑结果。
7. 投诉和客服记录。

简单点赞点踩信息量有限。更好的反馈要能定位问题：是检索错、理解错、生成错、引用错、权限错，还是用户期望不同。

10.7 失败样本分析

失败样本是产品改进的燃料。

失败类型可以分为：

1. 意图理解错误。
2. 检索失败。
3. 引用错误。

4. 生成幻觉。
5. 工具调用失败。
6. 权限问题。
7. 拒答不当。
8. 格式不符合要求。
9. 延迟过高。
10. 用户需求不清。

每类失败对应不同优化手段。不能把所有失败都归因于“模型不够强”。

10.8 A/B Test

A/B test 用于比较不同方案。

可以测试：

1. 不同模型。
2. 不同 prompt。
3. 不同 RAG 策略。
4. 不同交互设计。
5. 不同拒答策略。
6. 不同排序或推荐方案。

指标要提前定义。例如测试 RAG 策略时，看引用准确率、自助解决率和延迟，而不是只看主观感觉。

10.9 离线评估和在线评估

离线评估：用固定评估集测试。

优点：

1. 可重复。
2. 成本低。
3. 适合回归测试。

缺点：

1. 覆盖不了真实用户长尾。
2. 可能过拟合评估集。

在线评估：看真实用户使用数据。

优点：

1. 贴近真实场景。
2. 能发现新问题。

缺点：

1. 风险更高。
2. 归因复杂。
3. 需要灰度和监控。

成熟产品需要离线和在线结合。

10.10 反馈到数据闭环

用户反馈要进入数据闭环。

流程：

线上日志 -> 失败样本 -> 人工标注 -> 评估集更新 -> prompt/RAG/模型优化 -> 回归测试 -> 灰度上线

反馈闭环的关键是：反馈不能停留在表格里，要真正驱动系统改进。

10.11 Prompt 迭代

Prompt 是大模型产品的重要配置。

迭代时要注意：

1. 记录版本。
2. 说明修改原因。
3. 跑回归评估。
4. 灰度上线。
5. 监控线上变化。
6. 能快速回滚。

不要在生产环境里随手改 prompt。Prompt 变化可能影响大量行为。

10.12 模型版本迭代

模型升级可能带来能力提升，也可能带来回归。

要评估：

1. 任务成功率是否提升。
2. 延迟是否变化。
3. 成本是否变化。
4. 风格是否变化。
5. 拒答策略是否变化。
6. 安全表现是否变化。
7. 结构化输出是否稳定。

模型升级要像软件版本升级一样管理，而不是直接替换。

10.13 知识库运营

RAG 产品要运营知识库。

工作包括：

1. 新文档入库。
2. 过期文档下线。
3. 文档负责人维护。
4. 权限同步。
5. 缺失问题补文档。
6. 高访问文档优化。
7. 冲突文档合并。

知识库不运营，RAG 效果会逐渐下降。

10.14 安全监控

上线后要持续监控安全。

包括：

1. 敏感信息泄露。
2. 越权访问。
3. Prompt injection。
4. 高风险请求。
5. 不合规输出。
6. 工具误用。
7. 异常调用量。
8. 滥用行为。

安全不是上线前测一次，而是持续监控和响应。

10.15 运营看板

大模型产品需要运营看板。

看板可以包括：

1. DAU 和 MAU。
2. 任务量。
3. 成功率。
4. 满意度。
5. 成本。
6. 延迟。
7. 错误类型分布。
8. 热门问题。
9. 低满意度样本。
10. 安全事件。

看板的目标不是好看，而是帮助团队发现问题和决定下一步优化。

10.16 持续迭代优先级

不是所有问题都要立刻修。

优先级可以看：

1. 影响用户数。
2. 业务价值。
3. 安全风险。
4. 发生频率。
5. 修复成本。
6. 是否阻塞核心流程。

高频低风险问题和低频高风险问题都要关注，只是处理方式不同。

10.16.1 关键公式与上线运营指标速查

可以把一次线上请求或任务样本写成：

$$o_i=(u_i,x_i,y_i,z_i,f_i,b_i,v_i,c_i,l_i,r_i)$$

其中 u_i 是用户或租户， x_i 是输入， y_i 是模型输出或 Agent 轨迹结果， z_i 是任务是否成功， f_i 是用户反馈或人工审核标签， b_i 是 bad case 类别， v_i 是模型 / prompt / RAG / 工具版本， c_i 是成本， l_i 是延迟， r_i 是安全、隐私或回滚风险标记。

1. 灰度覆盖率

灰度发布要明确当前流量占比：

$$q_{\{\mathrm{roll}\}}=\frac{N_{\{\mathrm{rollout}\}}}{N_{\{\mathrm{eligible}\}}+\epsilon}$$

其中 N_{rollout} 是进入新版本或新策略的请求数， N_{eligible} 是符合实验条件的请求数。灰度覆盖率太高会放大风险，太低则统计信号不足。

2. 线上任务成功率与采纳率

上线后要看真实任务是否完成，而不是只看模型输出是否流畅：

$$A_{\{\mathrm{live}\}}=\frac{N_{\{\mathrm{success}\}}}{N_{\{\mathrm{task}\}}+\epsilon}, \quad R_{\{\mathrm{adopt}\}}=\frac{N_{\{\mathrm{adopted}\}}}{N_{\{\mathrm{success}\}}+\epsilon}$$

其中 N_{success} 是真实任务成功数, N_{adopted} 是用户采纳、保存、提交或继续使用模型结果的次数。低采纳率可能说明答案虽然正确, 但不符合工作流、格式、信任或责任要求。

3. 反馈采集率与反馈行动率

反馈闭环要区分“收到了反馈”和“真的处理了反馈”：

$$C_{\{\mathrm{fb}\}} = \frac{N_{\{\mathrm{feedback}\}}}{N_{\{\mathrm{task}\}} + \epsilon}, \quad R_{\{\mathrm{fb}\}} = \frac{N_{\{\mathrm{feedback_actioned}\}}}{N_{\{\mathrm{feedback}\}} + \epsilon}$$

其中 N_{feedback} 是点赞点踩、错误类型、编辑、投诉、转人工或人工审核反馈数, $N_{\text{feedback_actioned}}$ 是已经进入评估集、prompt 修复、知识库修复、产品改动或安全回归的反馈数。

4. Bad case 分析覆盖率

失败样本不能只堆在表格里：

$$C_{\{\mathrm{bad}\}} = \frac{N_{\{\mathrm{bad_triaged}\}}}{N_{\{\mathrm{bad}\}} + \epsilon}$$

其中 N_{bad} 是线上失败样本数, $N_{\text{bad_triaged}}$ 是已经标注失败原因、责任模块、影响范围、修复方式和复测状态的样本数。

5. 回归评估通过率与评估切片覆盖率

每次改模型、prompt、检索策略、工具 schema 或安全策略前, 都要跑回归：

$$R_{\{\mathrm{reg}\}} = \frac{N_{\{\mathrm{reg_pass}\}}}{N_{\{\mathrm{reg}\}} + \epsilon}, \quad C_{\{\mathrm{slice}\}} = \frac{N_{\{\mathrm{slice_covered}\}}}{N_{\{\mathrm{slice}\}} + \epsilon}$$

其中 N_{reg} 是回归样本数, $N_{\text{reg_pass}}$ 是通过样本数; N_{slice} 是预期覆盖的业务、风险、语言、长度、权限、知识库和高价值用户切片数。

6. 延迟 SLO 与成本预算门禁

线上体验看长尾, 不只看平均值：

$$G_{\{\mathrm{lat}\}} = \mathbf{1}[L_{95} \leq L_{\{\mathrm{slo}\}}], \quad G_{\{\mathrm{cost}\}} = \mathbf{1}[C_{\{\mathrm{unit}\}} \leq C_{\{\mathrm{budget}\}}]$$

其中 L_{95} 是 P95 延迟, L_{slo} 是延迟目标, C_{unit} 是单任务成本, C_{budget} 是单任务成本预算。模型升级、prompt 变长、RAG top-k 增大、Agent 步数增加和重试率上升都可能让这两个门禁失效。

7. 安全事故率与事故处理覆盖率

上线后要持续监控安全、隐私和滥用事件：

$$R_{\{\mathrm{incident}\}} = \frac{N_{\{\mathrm{incident}\}}}{N_{\{\mathrm{task}\}} + \epsilon}, \quad C_{\{\mathrm{incident}\}} = \frac{N_{\{\mathrm{incident_resolved}\}}}{N_{\{\mathrm{incident}\}} + \epsilon}$$

其中 N_{incident} 是敏感信息泄露、越权访问、提示注入、工具误用、高风险输出、异常调用或滥用事件数, $N_{\text{incident_resolved}}$ 是已经完成止血、定位、修复、通知和复测的事件数。

8. Trace 覆盖率与回滚就绪

Agent、RAG、工具调用和企业应用上线后, 光存输入输出不够, 还要能复盘链路：

$$C_{\{\mathrm{trace}\}} = \frac{N_{\{\mathrm{trace}\}}}{N_{\{\mathrm{trace_required}\}} + \epsilon}, \quad G_{\{\mathrm{rollback}\}} = \mathbf{1}[\mathrm{rollback_plan_ready}]$$

其中 N_{trace} 是实际保留了可审计 trace、版本、检索证据、工具调用和关键指标的请求数, $N_{\text{trace_required}}$ 是按风险策略要求必须保留 trace 的请求数。 G_{rollback} 表示是否已经准备 feature flag、旧版本、数据回滚、配置回滚、人工兜底和事故沟通预案。没有 trace 就很难定位失败, 没有回滚就不适合扩大灰度。

9. 上线运营门禁

一个简化上线运营门禁可以写成：

$$G_{\{\mathrm{ops}\}} = \mathbf{1}[A_{\{\mathrm{live}\}} \geq a_0] \mathbf{1}[R_{\{\mathrm{adopt}\}} \geq d_0] \mathbf{1}[R_{\{\mathrm{fb}\}} \geq f_0]$$

真实项目还要加入 canary、shadow traffic、feature flag、回滚预案、模型 / prompt / RAG / 工具版本记录、trace、人工复核、异常告警、SLO burn rate 和事故复盘。运营门禁强调：上线不是“评估通过一次”，而是“能持续发现问题、修复问题并证明没有引入严重回归”。

10.16.2 最小可运行上线运营审计 demo

下面这个 0 依赖 demo 演示如何把质量、反馈、回归、延迟、成本、安全、trace 和回滚放到同一张运营审计表里。它不是生产监控系统，而是帮助你在面试中讲清楚：上线后不能只看一个平均满意度，而要把每个候选产品形态都放进门禁。

```
from collections import Counter

products = [
    {
        "name": "support_rag_assistant",
        "eligible": 1000,
        "rollout": 180,
        "tasks": 180,
        "success": 154,
        "adopted": 142,
        "feedback": 62,
        "feedback_actioned": 55,
        "bad_cases": 18,
        "bad_triaged": 17,
        "reg_total": 240,
        "reg_pass": 232,
        "slices": 10,
        "slices_covered": 10,
        "p95_latency_ms": 2200,
        "latency_slo_ms": 3000,
        "unit_cost": 0.036,
        "cost_budget": 0.05,
        "incidents": 0,
        "incidents_resolved": 0,
        "traces": 180,
        "traces_required": 180,
        "rollback_ready": True,
        "failure_reasons": [],
    },
    {
        "name": "contract_review_copilot",
        "eligible": 400,
        "rollout": 160,
        "tasks": 160,
        "success": 126,
        "adopted": 95,
        "feedback": 26,
        "feedback_actioned": 12,
        "bad_cases": 29,
        "bad_triaged": 16,
        "reg_total": 180,
        "reg_pass": 160,
        "slices": 9,
        "slices_covered": 7,
        "p95_latency_ms": 4200,
        "latency_slo_ms": 3500,
```

```

    "unit_cost": 0.085,
    "cost_budget": 0.06,
    "incidents": 1,
    "incidents_resolved": 1,
    "traces": 150,
    "traces_required": 160,
    "rollback_ready": True,
    "failure_reasons": [
        "citation_gap",
        "feedback_not_actioned",
        "latency_over_slo",
        "cost_over_budget",
    ],
},
{
    "name": "refund_agent_auto",
    "eligible": 1200,
    "rollout": 360,
    "tasks": 360,
    "success": 295,
    "adopted": 281,
    "feedback": 38,
    "feedback_actioned": 14,
    "bad_cases": 34,
    "bad_triaged": 20,
    "reg_total": 220,
    "reg_pass": 207,
    "slices": 12,
    "slices_covered": 11,
    "p95_latency_ms": 2600,
    "latency_slo_ms": 3000,
    "unit_cost": 0.041,
    "cost_budget": 0.05,
    "incidents": 3,
    "incidents_resolved": 1,
    "traces": 330,
    "traces_required": 360,
    "rollback_ready": False,
    "failure_reasons": [
        "missing_confirmation",
        "incident_unresolved",
        "rollback_not_ready",
        "feedback_not_actioned",
    ],
},
{
    "name": "generic_chat_portal",
    "eligible": 5000,
    "rollout": 5000,
    "tasks": 5000,
    "success": 3100,
    "adopted": 1700,
    "feedback": 120,
    "feedback_actioned": 22,
    "bad_cases": 410,

```

```

        "bad_triaged": 80,
        "reg_total": 90,
        "reg_pass": 72,
        "slices": 8,
        "slices_covered": 3,
        "p95_latency_ms": 5100,
        "latency_slo_ms": 3000,
        "unit_cost": 0.072,
        "cost_budget": 0.04,
        "incidents": 8,
        "incidents_resolved": 3,
        "traces": 900,
        "traces_required": 5000,
        "rollback_ready": False,
        "failure_reasons": [
            "unclear_workflow",
            "weak_eval",
            "no_trace",
            "incident_unresolved",
            "cost_over_budget",
        ],
    },
]

thresholds = {
    "live_success": 0.80,
    "adoption": 0.80,
    "feedback_action": 0.70,
    "bad_triage": 0.80,
    "regression": 0.95,
    "slice": 0.90,
    "trace": 0.95,
    "incident_rate": 0.01,
    "incident_resolved": 0.90,
}

def rate(num, den):
    return 0.0 if den == 0 else num / den

def audit(row):
    metrics = {
        "rollout": rate(row["rollout"], row["eligible"]),
        "live_success": rate(row["success"], row["tasks"]),
        "adoption": rate(row["adopted"], row["success"]),
        "feedback_action": rate(row["feedback_actioned"], row["feedback"]),
        "bad_triage": rate(row["bad_triaged"], row["bad_cases"]),
        "regression": rate(row["reg_pass"], row["reg_total"]),
        "slice": rate(row["slices_covered"], row["slices"]),
        "trace": rate(row["traces"], row["traces_required"]),
        "incident_rate": rate(row["incidents"], row["tasks"]),
        "incident_resolved": (
            1.0
            if row["incidents"] == 0

```

```

        else rate(row["incidents_resolved"], row["incidents"])
    ),
}
gates = {
    "quality_ok": (
        metrics["live_success"] >= thresholds["live_success"]
        and metrics["adoption"] >= thresholds["adoption"]
    ),
    "feedback_ok": (
        metrics["feedback_action"] >= thresholds["feedback_action"]
        and metrics["bad_triage"] >= thresholds["bad_triage"]
    ),
    "regression_ok": (
        metrics["regression"] >= thresholds["regression"]
        and metrics["slice"] >= thresholds["slice"]
    ),
    "latency_ok": row["p95_latency_ms"] <= row["latency_slo_ms"],
    "cost_ok": row["unit_cost"] <= row["cost_budget"],
    "incident_ok": (
        metrics["incident_rate"] <= thresholds["incident_rate"]
        and metrics["incident_resolved"] >= thresholds["incident_resolved"]
    ),
    "trace_ok": metrics["trace"] >= thresholds["trace"],
    "rollback_ok": row["rollback_ready"],
}
failed = [name for name, ok in gates.items() if not ok]
score_parts = [
    metrics["live_success"],
    metrics["adoption"],
    metrics["feedback_action"],
    metrics["bad_triage"],
    metrics["regression"],
    metrics["slice"],
    metrics["trace"],
    1.0 if gates["latency_ok"] else 0.0,
    1.0 if gates["cost_ok"] else 0.0,
    1.0 if gates["incident_ok"] else 0.0,
    1.0 if gates["rollback_ok"] else 0.0,
]
return {
    "name": row["name"],
    "score": round(sum(score_parts) / len(score_parts), 3),
    "gate_pass": not failed,
    "failed_gates": failed,
    "metrics": metrics,
}

```

```

reports = [audit(row) for row in products]
ranked = [
    (r["name"], r["score"], r["gate_pass"])
    for r in sorted(reports, key=lambda x: x["score"], reverse=True)
]
ops_pass = [r["name"] for r in reports if r["gate_pass"]]
needs_rework = {r["name"]: r["failed_gates"] for r in reports if not r["gate_pass"]}

```

```
reason_counts = Counter(reason for row in products for reason in row["failure_reasons"])
```

```
sample = reports[0]["metrics"]
sample_metrics = {
    "live_success": round(sample["live_success"], 3),
    "adoption": round(sample["adoption"], 3),
    "feedback_action": round(sample["feedback_action"], 3),
    "bad_triage": round(sample["bad_triage"], 3),
    "regression": round(sample["regression"], 3),
    "trace": round(sample["trace"], 3),
}
```

```
print("ranked=", ranked)
print("ops_pass=", ops_pass)
print("sample_metrics=", sample_metrics)
print("needs_rework=", needs_rework)
print("top_failure_reasons=", reason_counts.most_common(5))
```

一组可复现输出：

```
ranked= [('support_rag_assistant', 0.961, True), ('refund_agent_auto', 0.682, False), ('contract_review_copilot', 0.651, False), ('incident_unresolved', 0.645, False), ('cost_over_budget', 0.645, False)]
ops_pass= ['support_rag_assistant']
sample_metrics= {'live_success': 0.856, 'adoption': 0.922, 'feedback_action': 0.887, 'bad_triage': 0.944, 'regression': 0.944, 'trace': 0.944}
needs_rework= {'contract_review_copilot': ['quality_ok', 'feedback_ok', 'regression_ok', 'latency_ok', 'cost_ok', 'trace_ok'], 'incident_unresolved': ['quality_ok', 'feedback_ok', 'regression_ok', 'latency_ok', 'cost_ok', 'trace_ok'], 'cost_over_budget': ['quality_ok', 'feedback_ok', 'regression_ok', 'latency_ok', 'cost_ok', 'trace_ok']}
top_failure_reasons= [('feedback_not_actioned', 2), ('cost_over_budget', 2), ('incident_unresolved', 2), ('citation_gap', 2)]
```

这个 demo 的重点是门禁解释：客服 RAG 助手虽然还在小流量灰度，但质量、反馈、回归、成本、延迟、trace 和回滚都过线，可以继续扩大；退款 Agent 的任务成功率看起来不错，但事故处理、回滚和反馈行动不过线，不适合直接全自动放量；通用聊天入口几乎所有运营门禁都失败，应重新定义场景和工作流。

10.17 常见失败模式

1. 上线后没人看数据。
2. 只收集点赞点踩，不分析原因。
3. Prompt 随意改，没有版本管理。
4. 模型升级不做回归。
5. 知识库不维护。
6. 成本异常没人发现。
7. 安全事件没有响应流程。
8. A/B test 指标不清。
9. 反馈没有进入评估集。
10. 只追求模型能力，不运营产品。

大模型产品的长期质量来自持续运营，而不是一次上线。

10.18 面试题：大模型产品上线后如何运营

回答要点：

上线后要监控质量、成本、延迟、安全和用户反馈。质量上看任务成功率、答案正确率、引用准确率和满意度；成本上看 token、模型

10.19 面试题：如何做大模型产品 A/B Test

回答要点：

先明确实验目标和主指标，例如提升自助解决率、降低转人工率或提高引用准确率。然后控制变量，只比较一个关键差异，比如模型、

10.20 本章小结

大模型产品上线后，需要持续监控、反馈、评估和迭代。灰度发布、质量监控、成本监控、用户反馈、失败样本分析、A/B test、知识库运营、安全监控和版本管理，都是产品长期成功的关键。

下一章是第十八册最后的技术产品面试题，会把产品化、场景选择、体验设计、ROI、企业应用、RAG、Agent、多模态、隐私合规和运营闭环串成面试表达体系。

第十一章：技术产品面试题

本章是第十八册的综合面试题。前面章节已经讲过产品化总览、用户需求、产品体验、ROI、企业级应用、RAG、Agent、多模态、隐私合规和上线运营。本章把这些内容整理成面试中可以直接使用的回答。

技术产品类问题的关键，不是把自己包装成产品经理，而是证明你理解技术如何产生业务价值。一个成熟的算法工程师，应该既能讲模型、RAG、Agent、评估，也能讲用户、场景、成本、风险、上线和反馈闭环。

本讲按 `WRITING_PLAN.md` 的第二轮要求做公式和 demo 精修。联网资料主要核对七类口径：OpenAI Evals / graders 用于把线上反馈和失败样本转成可重复评估；OpenAI 生产最佳实践和 latency optimization 用于支撑成本、延迟、版本和生产稳定性回答；OpenAI Agents SDK tracing 用于支撑 Agent trace、工具调用和可审计性；Google SRE SLI / SLO / error budget 用于支撑 P95/P99、SLO 和回滚口径；NIST AI RMF Generative AI Profile 用于支撑生成式 AI 上线后的治理、监控和人类监督；OWASP LLM Top 10 用于支撑提示注入、敏感信息泄露、过度代理和资源消耗等线上风险边界；前面第十八册各章的审计 demo 用于把面试回答落到可计算指标和门禁。

本章不替代真实产品经理面试、商业计划或法务合规审查。这里聚焦算法工程师的技术产品表达：把“我会做模型”升级成“我能证明模型方案在真实用户、成本、风险和运营闭环中值得上线”。

11.1 大模型从 Demo 到产品差在哪里

回答要点：

Demo 主要证明技术能跑通，通常样例少、输入可控、失败可接受。产品要面对真实用户和长尾输入，需要稳定性、成本控制、安全合

11.2 如何判断一个 LLM 场景值不值得做

回答要点：

我会从用户痛点、任务频率、单次价值、数据可得性、可验证性、容错率、成本和风险几个维度评估。优先选择高频或高价值、数据可

11.3 如何判断需求是否真实

回答要点：

我不会只听用户说想要什么，而会看他们现在是否已经花时间或成本解决这个问题，任务发生频率如何，现有方案痛点在哪里，成功

11.4 如何把模型能力转成产品体验

回答要点：

我会先明确用户任务和成功标准，再把模型能力映射到体验指标。例如理解能力对应意图识别和任务完成率，生成能力对应草稿质量和

11.5 为什么高分模型产品体验可能不好

回答要点：

因为 benchmark 只覆盖部分能力，而产品体验还受延迟、稳定性、可控性、交互设计、引用、拒答、风格、成本和错误恢复影响。一

11.6 如何估算大模型项目 ROI

回答要点：

我会先明确收益来源，是节省人工、提升效率、增加收入还是降低风险。然后估算任务量、自动化比例、单次节省时间或增量收入，再

11.7 如何降低大模型产品成本

回答要点：

可以从模型、上下文、调用策略和系统架构优化。简单任务用小模型或规则，大模型只处理复杂任务；控制 prompt 和 RAG 上下文长度

11.8 企业级 LLM 应用和普通应用有什么区别

回答要点：

企业级 LLM 应用更强调数据、权限、系统集成、稳定性、审计和 ROI。普通应用可能只关注用户体验和模型效果，但企业应用需要接入

11.9 如何落地企业知识库问答

回答要点：

我会先做数据治理，整理文档来源、版本、权限和元数据，再构建 RAG 流程，包括文档解析、分块、embedding、检索、rerank、答案

11.10 RAG 产品落地最难的是什么

回答要点：

RAG 落地最难的不只是检索算法，而是企业文档治理、权限控制、知识更新、引用可信和效果评估。文档过期、版本冲突、权限不清、

11.11 如何评估企业 RAG

回答要点：

我会分层评估。检索层看召回率和 rerank 命中率；生成层看答案正确率、引用准确率和证据支持率；产品层看自助解决率、用户满意

11.12 Agent 产品落地最重要的是什么

回答要点：

Agent 产品落地最重要的是控制任务边界和行动风险。Agent 能执行工具和真实动作，所以要明确适用场景、工具权限、状态管理、人

11.13 如何设计企业 Agent 产品

回答要点：

我会先选择高价值但风险可控的任务，定义完成标准。系统上设计 planner、tool registry、executor、state manager、permission

11.14 多模态产品落地难点是什么

回答要点：

多模态产品落地难点包括输入质量不稳定、模型对细节理解可能出错、生成可控性不足、延迟和成本较高、版权和隐私风险更复杂，

11.15 如何落地图像理解产品

回答要点：

我会先明确图像理解服务哪个任务，例如文档解析、质检、拍照问答或设计评审。然后评估输入质量、目标识别难度、OCR 或 layout

11.16 大模型产品如何做隐私保护

回答要点：

我会从数据全链路做隐私保护。首先梳理数据流，明确用户输入、检索文档、工具结果、日志和反馈会流向哪里。然后做数据最小化、

11.17 企业 RAG 如何避免数据泄露

回答要点：

企业 RAG 要在检索和生成全链路做权限控制。检索时只能召回用户有权限的文档，引用也必须是用户可访问的来源；不能把无权限文

11.18 大模型产品上线后如何运营

回答要点：

上线后要监控质量、成本、延迟、安全和用户反馈。质量上看任务成功率、答案正确率、引用准确率和满意度；成本上看 token、模型

11.19 如何做大模型产品 A/B Test

回答要点：

先明确实验目标和主指标，例如提升自助解决率、降低转人工率或提高引用准确率。然后控制变量，只比较一个关键差异，比如模型、

11.20 关键公式与面试评分速查

技术产品面试不是背诵产品词汇，而是用指标和门禁证明判断。一次面试回答可以抽象为：

$a_i = (q_i, T_i, F_i, D_i, R_i, H_i)$

其中 q_i 是问题， T_i 是覆盖的主题集合， F_i 是给出的公式或指标集合， D_i 是能落地的 demo 或项目证据集合， R_i 是风险边界集合， H_i 是 trade-off 集合。

1. 主题覆盖率

$C_{\{\mathrm{topic}\}} = \frac{|T_{\{\mathrm{ans}\}} \cap T_{\{\mathrm{req}\}}|}{|T_{\{\mathrm{req}\}}| + \epsilon}$

T_{req} 至少应覆盖 demo vs product、真实需求、产品指标、ROI、RAG、Agent、多模态、隐私、上线运营和反馈闭环。只讲模型结构而不讲用户、指标和风险，主题覆盖通常不够。

2. 公式覆盖率

$C_{\{\mathrm{formula}\}} = \frac{|F_{\{\mathrm{ans}\}} \cap F_{\{\mathrm{req}\}}|}{|F_{\{\mathrm{req}\}}| + \epsilon}$

高频公式包括 monthly benefit、unit cost、ROI、retrieval recall、citation accuracy、task success、unauthorized rate、input quality rate、ops gate、trace coverage、feedback action rate 和 privacy gate。公式不要求每题都写满，但要能在关键题上把判断量化。

3. Demo 证据覆盖率

$C_{\{\mathrm{demo}\}} = \frac{|D_{\{\mathrm{ans}\}} \cap D_{\{\mathrm{req}\}}|}{|D_{\{\mathrm{req}\}}| + \epsilon}$

这里的 demo 指本册前面可运行的审计脚本或项目证据，例如 ROI audit、RAG product audit、Agent product audit、multimodal product audit、privacy audit 和 ops audit。面试中能说出“我会用什么表和什么脚本验证”，比只说“持续优化”更可信。

4. 风险与 trade-off 覆盖率

$C_{\{\mathrm{risk}\}} = \frac{|R_{\{\mathrm{ans}\}} \cap R_{\{\mathrm{req}\}}|}{|R_{\{\mathrm{req}\}}| + \epsilon}, \quad C_{\{\mathrm{trade}\}} = \frac{|H_{\{\mathrm{ans}\}} \cap H_{\{\mathrm{req}\}}|}{|H_{\{\mathrm{req}\}}| + \epsilon}$

风险至少包括数据泄露、过度代理、隐私、版权、事故响应和采用率不确定性；trade-off 至少包括质量 vs 成本、召回 vs 延迟、自动化 vs 控制、延迟 vs 质量、上线速度 vs 安全。

5. 技术产品面试门禁

$$G_{\{\mathrm{prod_int}\}}=\mathbf{1}[C_{\{\mathrm{topic}\}}\geq t_0]\mathbf{1}[C_{\{\mathrm{formula}\}}\geq f_0]\mathbf{1}[C_{\{\mathrm{m}}}$$

其中 N_{weak} 是低分问题数。这个门禁的用途不是给自己制造焦虑，而是避免面试回答停留在“我知道这些词”，帮助你定位缺公式、缺 demo、缺风险边界或缺 trade-off 的题。

11.21 最小可运行技术产品面试复盘 demo

下面这个 0 依赖 demo 用 toy 面试回答记录，检查技术产品类回答是否覆盖主题、公式、demo、风险和 trade-off。

```
answers = [
  {
    "question": "demo_to_product",
    "topics": {"demo_to_product", "real_user", "monitoring", "feedback_loop", "business_metric"},
    "formulas": set(),
    "demos": set(),
    "risks": {"long_tail"},
    "tradeoffs": {"quality_cost"},
  },
  {
    "question": "roi",
    "topics": {"roi", "unit_economics", "adoption", "ops_cost"},
    "formulas": {"monthly_benefit", "unit_cost", "roi"},
    "demos": {"roi_audit"},
    "risks": {"adoption_uncertainty"},
    "tradeoffs": {"quality_cost"},
  },
  {
    "question": "enterprise_rag",
    "topics": {"rag", "document_governance", "permission", "citation", "evaluation"},
    "formulas": {"retrieval_recall", "citation_accuracy"},
    "demos": {"rag_product_audit"},
    "risks": {"data_leakage"},
    "tradeoffs": {"recall_latency"},
  },
  {
    "question": "agent_product",
    "topics": {"agent", "tool_permission", "trace", "human_confirmation", "rollback"},
    "formulas": {"task_success", "unauthorized_rate"},
    "demos": {"agent_product_audit"},
    "risks": {"excessive_agency"},
    "tradeoffs": {"automation_control"},
  },
  {
    "question": "multimodal_product",
    "topics": {"multimodal", "input_quality", "evidence", "safety"},
    "formulas": {"input_quality_rate"},
    "demos": set(),
    "risks": {"privacy", "copyright"},
    "tradeoffs": {"latency_quality"},
  },
  {
    "question": "online_ops",
    "topics": {"online_monitoring", "feedback_loop", "regression", "rollback", "trace"},
    "formulas": {"ops_gate", "trace_coverage", "feedback_action_rate"},
    "demos": {"ops_audit"},
    "risks": {"incident_response"},
  }
]
```

```

        "tradeoffs": {"speed_safety"},
    },
    {
        "question": "privacy",
        "topics": {"privacy", "data_flow", "log_governance"},
        "formulas": set(),
        "demos": set(),
        "risks": {"pii"},
        "tradeoffs": set(),
    },
]

required = {
    "topics": {
        "demo_to_product", "real_user", "business_metric", "roi", "unit_economics",
        "rag", "document_governance", "permission", "citation", "evaluation",
        "agent", "tool_permission", "trace", "human_confirmation", "multimodal",
        "input_quality", "privacy", "online_monitoring", "feedback_loop", "rollback",
    },
    "formulas": {
        "monthly_benefit", "unit_cost", "roi", "retrieval_recall",
        "citation_accuracy", "task_success", "unauthorized_rate", "input_quality_rate",
        "ops_gate", "trace_coverage", "feedback_action_rate", "privacy_gate",
    },
    "demos": {
        "roi_audit", "rag_product_audit", "agent_product_audit",
        "multimodal_product_audit", "privacy_audit", "ops_audit",
    },
    "risks": {
        "data_leakage", "excessive_agency", "privacy",
        "copyright", "incident_response", "adoption_uncertainty",
    },
    "tradeoffs": {
        "quality_cost", "recall_latency", "automation_control",
        "latency_quality", "speed_safety",
    },
}

```

```

def union(field):
    values = set()
    for answer in answers:
        values.update(answer[field])
    return values

def coverage(field):
    got = union(field)
    need = required[field]
    return round(len(got & need) / len(need), 3), sorted(need - got)

summary = {}
missing = {}
for field in required:

```

```

summary[field + "_coverage"], missing[field] = coverage(field)

question_scores = {}
for answer in answers:
    topic_hit = len(answer["topics"] & required["topics"]) / max(1, len(answer["topics"]))
    formula_hit = 1.0 if answer["formulas"] else 0.0
    demo_hit = 1.0 if answer["demos"] else 0.0
    risk_hit = 1.0 if answer["risks"] else 0.0
    tradeoff_hit = 1.0 if answer["tradeoffs"] else 0.0
    score = 0.4 * topic_hit + 0.2 * formula_hit + 0.2 * demo_hit + 0.1 * risk_hit + 0.1 * tradeoff_hit
    question_scores[answer["question"]] = round(score, 3)

weak_questions = [q for q, score in question_scores.items() if score < 0.75]
gates = {
    "topic_ok": summary["topics_coverage"] >= 0.85,
    "formula_ok": summary["formulas_coverage"] >= 0.75,
    "demo_ok": summary["demos_coverage"] >= 0.70,
    "risk_ok": summary["risks_coverage"] >= 0.85,
    "tradeoff_ok": summary["tradeoffs_coverage"] >= 0.85,
    "weak_question_ok": len(weak_questions) <= 1,
}
interview_ready = all(gates.values())
revision_plan = []
if missing["formulas"]:
    revision_plan.append(" 补公式: " + ", ".join(missing["formulas"][:3]))
if missing["demos"]:
    revision_plan.append(" 补 demo: " + ", ".join(missing["demos"][:3]))
if weak_questions:
    revision_plan.append(" 重练问题: " + ", ".join(weak_questions[:3]))

print("summary=", summary)
print("missing=", missing)
print("question_scores=", question_scores)
print("weak_questions=", weak_questions)
print("gates=", gates)
print("interview_ready=", interview_ready)
print("revision_plan=", revision_plan)

```

一组可复现输出:

```

summary= {'topics_coverage': 1.0, 'formulas_coverage': 0.917, 'demos_coverage': 0.667, 'risks_coverage': 1.0, 'tradeoffs_coverage': 1.0}
missing= {'topics': [], 'formulas': ['privacy_gate'], 'demos': ['multimodal_product_audit', 'privacy_audit'], 'risks': []}
question_scores= {'demo_to_product': 0.52, 'roi': 0.8, 'enterprise_rag': 1.0, 'agent_product': 1.0, 'multimodal_product_audit': 0.667, 'privacy_audit': 0.667}
weak_questions= ['demo_to_product', 'multimodal_product_audit', 'privacy_audit']
gates= {'topic_ok': True, 'formula_ok': True, 'demo_ok': False, 'risk_ok': True, 'tradeoff_ok': True, 'weak_question_ok': False}
interview_ready= False
revision_plan= ['补公式: privacy_gate', '补demo: multimodal_product_audit, privacy_audit', '重练问题: demo_to_product, multimodal_product_audit, privacy_audit']

```

这个 demo 故意让 interview_ready=False。它暴露的是：主题覆盖已经够了，但 privacy 题缺公式和 demo，多模态题缺可运行证据，demo-to-product 题停留在概念层。真实复盘时，下一步不是继续背概念，而是补 privacy_gate 公式、隐私治理 demo、多模态产品审计 demo，并把弱题重新按 “用户价值 -> 技术方案 -> 指标公式 -> demo 证据 -> 风险边界 -> trade-off” 回答一遍。

11.22 技术产品面试回答模板

通用模板:

我会从用户价值、技术可行性、成本收益、风险控制和上线运营五个方面看。先确认用户痛点和成功指标，再判断大模型是否是合适用于 RAG：

RAG 项目我会重点看文档治理、权限、检索、引用、评估和反馈闭环。因为企业 RAG 的难点通常不是接一个向量库，而是让答案基于用于 Agent：

Agent 项目我会重点看任务边界、工具权限、人工确认、失败恢复和审计。Agent 能执行真实动作，所以产品化时不能只追求自主性，用于 ROI：

ROI 我会把收益和成本都拆开。收益看节省人工、提升效率、增加收入或降低风险；成本看模型、检索、工具、人审、研发、运维和合

11.23 高频追问

追问：什么时候不该用大模型？

当规则或传统系统能稳定低成本解决，任务强确定、低延迟、低容错，或者数据和权限无法治理时，不应强行用大模型。大模型适合

追问：产品上线后指标下降怎么办？

先定位是模型问题、检索问题、数据变化、prompt 变化、用户流量变化还是成本延迟问题。需要看日志、失败样本、版本变更和用户

追问：如何平衡安全和体验？

安全和体验不是简单对立。低风险场景可以更自动化，高风险场景需要引用、人审和确认。好的拒答要说明原因并提供替代帮助，权限

11.24 本章小结

技术产品面试考察的是你是否理解大模型从能力到价值的完整链路。好的回答不只讲模型，也讲用户需求、场景选择、产品体验、成本收益、企业系统、RAG/Agent 落地、隐私合规和上线运营。

到这里，第十八册《大模型产品化、商业化与落地》的正文第一版完成，第二轮也已经把产品化、场景选择、体验、ROI、企业应用、RAG、Agent、多模态、隐私治理、上线运营和技术产品面试串成了可计算、可复盘、可面试表达的闭环。本册的主线是：从“技术能做”走向“产品值得做、用户愿意用、业务能受益、风险可控制、系统可运营”。